

Where irrigation exists is globally contested

3. Main analysis

Arnald Puy

Contents

1	Preliminary	3
2	Uncertainty and sensitivity analysis	5
2.1	Composite figure	9
2.2	Does coarsening reduce disagreement?	18
2.3	UA/SA by excluding mapping paradigms	22
2.4	UA/SA by weighting newer datasets more	28
2.5	UA/SA on mapping variables	32
3	The maximum threshold for existence disagreement: $\tau_{\max}$	44
3.1	Composite figure	52
3.2	How many grid cells disagree?	56
3.3	How much irrigated land lies in disagreement?	57
4	Consequences for countries	59
4.1	Identification of irrigation hotspots	59
4.2	National rankings	62
4.3	Shift in country ranks	70
4.4	Fraction of irrigated areas lost	78
5	Consequences for crop production	88
5.1	Extract data from EarthStat	88
5.2	Plot crop heatmap	92
5.3	Plot breadbasket regions	95
5.4	Plot global	99
6	Consequences for irrigation-induced evapotranspiration	101
6.1	Extract CESM data	101
6.2	Merge figures	108
7	Final figures	110
8	Is disagreement caused by different irrigation conceptualizations?	115
9	Assessment of non-identifiability	123

10 Irrigation masks	126
11 Session information	127

1 Preliminary

```
# Load libraries -----

sensobol::load_packages(c("data.table", "scales", "cowplot", "tidyverse", "sensobol",
                          "here", "terra", "rnatrualearth", "sf", "countrycode",
                          "wesanderson", "benchmarkme", "parallel"))

# Create custom theme -----

theme_AP <- function() {
  theme_bw() +
    theme(panel.grid.major = element_blank(),
          panel.grid.minor = element_blank(),
          legend.background = element_rect(fill = "transparent", color = NA),
          legend.key = element_rect(fill = "transparent", color = NA),
          legend.key.width = unit(0.4, "cm"),
          legend.key.height = unit(0.5, "lines"),
          legend.key.spacing.y = unit(0, "lines"),
          legend.box.spacing = unit(0, "pt"),
          legend.spacing.y = unit(0.1, "cm"),
          legend.text = element_text(size = 6),
          legend.title = element_text(size = 7),
          axis.text.x = element_text(size = 7),
          axis.text.y = element_text(size = 7),
          axis.title.x = element_text(size = 7.3),
          axis.title.y = element_text(size = 7.3),
          axis.title = element_text(size = 10),
          plot.title = element_text(size = 8),
          strip.text.x = element_text(size = 7.4),
          strip.text.y = element_text(size = 7.4),
          strip.background = element_rect(fill = "white"),
          strip.text = element_text(margin = margin(t = 1.5, b = 1.5)))
}

# Source all .R files in the "functions" folder -----

r_functions <- list.files(path = here("functions"),
                          pattern = "\\R$", full.names = TRUE)

lapply(r_functions, source)

# Set seed -----

seed <- 123
```

```

# Load countries as sf (medium scale is a good balance) -----
countries_sf <- ne_countries(scale = "medium", returnclass = "sf")

# Keep only what is needed -----
countries_sf <- countries_sf[, c("name", "geometry")]

# Define colors and palettes and plotting datasets -----

tau_palette <- c("Agreement"= "#2ECC71",
  "Low disagreement" = "#F7DC6F",
  "Moderate disagreement" = "#F39C12",
  "High disagreement" = "#E74C3C",
  "Extreme disagreement" = "red")

res_palette <- c("0.2deg" = "lightblue",
  "0.4deg" = "#117A65",
  "1deg" = "darkblue")

```


2 Uncertainty and sensitivity analysis

```
# LOAD ALL DATASETS #####

# List all resolution files -----

files <- list.files("./datasets/irrigated_areas_regridded",
                    pattern = "irrigated_areas_regridded_.*\\.csv",
                    full.names = TRUE)

# Load them, bind them and label them -----

dt <- rbindlist(lapply(files, function(f) {
  x <- fread(f)
  res <- sub(".*_(\\d+)\\.csv", "\\1", f)
  x[, resolution:= paste0(as.numeric(res) / 10, "deg")]
  x
}))

# Load cropland mask -----

crop_path <- "./irrigated_area_datasets/GirrEO/asap_mask_crop_v02.tif"
crop_native <- rast(crop_path)

# Retrieve only cells within cropland -----

dt <- filter_long_dt_by_crop_mask_aggregated(dt = dt, crop_native = crop_native,
                                             rule = "any")

# DEFINE TAU GRID #####

# Vector with the datasets used -----

dataset_names <- unique(dt$dataset)
new_dataset_names <- c("GAEZ+2015", "GIAM", "GMIA", "GRIPC", "LUH2", "Meier",
                      "MIRCA 2000", "MIRCA OS", "Nagaraj", "SPAM2010")

# Define a dense grid of tau values from 1 ha (10^0) to 1e5 ha (10^5) -----

log_tau_min <- 0
log_tau_max <- 5
log_step <- 0.1

taus_ha_nonzero <- 10^(seq(log_tau_min, log_tau_max, by = log_step))
taus_ha <- c(0, taus_ha_nonzero)
taus_mha <- taus_ha * 1e-6
```

```

# Labels: "any_positive", then "1ha", "1.26ha", ..., "1e+05ha"---
tau_labels <- ifelse(taus_ha == 0, "any_positive", sprintf("%gha", signif(taus_ha, 3)))

# Define tau grid -----

tau_grid <- data.table(tau_label = tau_labels, tau_ha = taus_ha, tau_mha = taus_mha)

# TRANSFORM THE SAMPLE MATRIX #####

# Define the settings -----

N <- 2^13
params <- c("resolution", "tau", "exclusion")
matrices <- c("A", "B", "AB")
R <- 10^3
boot <- TRUE

# Resolution vector -----

resolution_vec <- c("0.2deg", "0.4deg", "1deg")
n_resolution_vec <- length(resolution_vec)

# Tau vector -----

tau_vec <- tau_grid$tau_ha
n_tau_vec <- length(tau_vec)

# Dataset exclusion vector -----

dataset_vec <- c("all", dataset_names)
n_dataset_vec <- length(dataset_vec)

# Create the sample matrix -----

mat <- data.table(sobol_matrices(matrices = matrices, N = N, params = params))

# Transform resolution column -----

mat[, resolution:= pmin(n_resolution_vec, 1L + floor(resolution * n_resolution_vec))]
mat[, resolution:= resolution_vec[resolution]]

# Transform tau column -----

mat[, tau:= pmin(n_tau_vec, 1L + floor(tau * n_tau_vec))]
mat[, tau:= tau_vec[tau]]

```

```

# Transform exclusion column -----

mat[, exclusion:= pmin(n_dataset_vec, 1L + floor(exclusion * n_dataset_vec))]
mat[, exclusion:= dataset_vec[exclusion]]

# PRE-COMPUTE ALL COMBINATIONS #####

# Compute all possibilities by looping over resolutions and tau values -----

# Set parallel backend -----

resolutions <- sort(unique(dt$resolution))
ncores <- max(1, parallel::detectCores() - 1)

# loop over resolutions and scenarios in parallel

tau_results <- rbindlist(
  parallel::mclapply(resolutions, function(res) {

    dt_res_long <- dt[resolution == res]

    # datasets actually present at this resolution
    ds_res <- sort(unique(dt_res_long$dataset))

    rbindlist(
      lapply(dataset_vec, function(scn) {

        # which datasets to use -----

        if (scn == "all") {

          use_ds <- ds_res

        } else {

          use_ds <- setdiff(ds_res, scn)

        }

        # if only one dataset left, skip -----
        if (length(use_ds) < 2L) {

          return(NULL)

        }

        # reshape to wide -----

        dt_res_wide <- dcast(dt_res_long[dataset %in% use_ds],

```

```

    lon + lat + country + code + continent ~ dataset,
    value.var = "mha", fill = 0
  )

  dt_res_wide[, `:=`(A_min = do.call(pmin, c(.SD, list(na.rm = TRUE))),
    A_max = do.call(pmax, c(.SD, list(na.rm = TRUE)))),
    .SDcols = use_ds]

  # compute tau -----

  out <- tau_grid[, compute_tau_fun(dt_res_wide, tau_mha, use_ds),
    .(tau_label, tau_ha, tau_mha)]

  # tag with resolution + scenario -----

  out[, `:=`(resolution = res, scenario = scn)]

  out
  }),
  use.names = TRUE, fill = TRUE
)
}, mc.cores = ncores),
use.names = TRUE, fill = TRUE)

# ARRANGE OUTPUT #####

# Rename tau_ha to tau to match mat -----

setnames(tau_results, "tau_ha", "tau")

# Join all three disagreement metrics onto mat -----

mat[tau_results, on = .(resolution, tau, exclusion = scenario),
  `:=`(frac_disagree = i.frac_disagree,
    frac_major_disagree = i.frac_major_disagree,
    frac_minor_disagree = i.frac_minor_disagree,
    share_existence = i.share_existence,
    share_marginal = i.share_marginal))]

# Reshape just these three for plotting -----

mat_long <- melt(mat, id.vars = c("resolution", "tau", "exclusion"),
  measure.vars = c("frac_disagree", "frac_major_disagree",
    "frac_minor_disagree", "share_existence",
    "share_marginal"),
  variable.name = "agreement", value.name = "frac_value")

```

2.1 Composite figure

```
# PREPARE DATA TO PLOT UNCERTAINTY #####

# Calculate mean and quantiles by tau x agreement -----

mat_sum <- mat_long[, .(frac_mean = mean(frac_value, na.rm = TRUE),
    frac_median = median(frac_value, na.rm = TRUE),
    frac_lwr = quantile(frac_value, 0.025, na.rm = TRUE),
    frac_upr = quantile(frac_value, 0.975, na.rm = TRUE)),
    .(tau, agreement)]

# Get values at tau = 0, 1, 10, 500, 1000, 1600, 2000, 5000, 10000, 12000 -----

target_tau <- c(0, 1, 10, 500, 1000, 1600, 2000, 5000, 10000, 12000)

mat_sum[, {dt_sub <- .SD
idx <- sapply(target_tau, function(x) {
  which.min(abs(log10(tau + 1e-12) - log10(x + 1e-12)))
})
dt_sub[idx]
},
agreement
] %>%
[, .(agreement, tau, frac_mean, frac_lwr, frac_upr)]
```

##	agreement	tau	frac_mean	frac_lwr	frac_upr
##	<fctr>	<num>	<num>	<num>	<num>
## 1:	frac_disagree	0.0000	0.7725147940	0.5965542885	0.9048508038
## 2:	frac_disagree	1.0000	0.7641095924	0.5935795736	0.8913352054
## 3:	frac_disagree	10.0000	0.7458169499	0.5872583044	0.8574304934
## 4:	frac_disagree	501.1872	0.5696037035	0.4990357022	0.6046632124
## 5:	frac_disagree	1000.0000	0.5101819636	0.4207578610	0.5563956371
## 6:	frac_disagree	1584.8932	0.4773297398	0.3893680378	0.5323500248
## 7:	frac_disagree	1995.2623	0.4499272243	0.3558259018	0.5173525037
## 8:	frac_disagree	5011.8723	0.3573094252	0.2308683309	0.4640555280
## 9:	frac_disagree	10000.0000	0.2758302271	0.1276267473	0.4086514626
## 10:	frac_disagree	12589.2541	0.2498516315	0.1079319283	0.3922905305
## 11:	frac_major_disagree	0.0000	0.5459749341	0.3824987605	0.6639383467
## 12:	frac_major_disagree	1.0000	0.5345828870	0.3787803669	0.6428472244
## 13:	frac_major_disagree	10.0000	0.5066465761	0.3694843827	0.5942080415
## 14:	frac_major_disagree	501.1872	0.3422024147	0.2610007097	0.3931581557
## 15:	frac_major_disagree	1000.0000	0.2967305803	0.2158021415	0.3670054536
## 16:	frac_major_disagree	1584.8932	0.2702110404	0.1959761780	0.3446950917
## 17:	frac_major_disagree	1995.2623	0.2537648037	0.1781096677	0.3362667328
## 18:	frac_major_disagree	5011.8723	0.1922679985	0.1100610979	0.2802429351
## 19:	frac_major_disagree	10000.0000	0.1453529051	0.0641605209	0.2356222112

```
## 20: frac_major_disagree 12589.2541 0.1288693376 0.0554201253 0.2146752603
## 21: frac_minor_disagree      0.0000 0.2265398599 0.1985622211 0.2863424569
## 22: frac_minor_disagree      1.0000 0.2295267055 0.1991819534 0.2922131021
## 23: frac_minor_disagree     10.0000 0.2391703737 0.2035200793 0.3034221002
## 24: frac_minor_disagree    501.1872 0.2274012888 0.1963311849 0.2613710001
## 25: frac_minor_disagree   1000.0000 0.2134513833 0.1893901834 0.2318943438
## 26: frac_minor_disagree   1584.8932 0.2071186994 0.1860436292 0.2213102108
## 27: frac_minor_disagree   1995.2623 0.1961624206 0.1772688000 0.2109764073
## 28: frac_minor_disagree   5011.8723 0.1650414267 0.1208072330 0.1941001487
## 29: frac_minor_disagree  10000.0000 0.1304773220 0.0634662264 0.1818294497
## 30: frac_minor_disagree  12589.2541 0.1209822939 0.0525118030 0.1823252355
## 31:      share_existence      0.0000 1.0000000000 1.0000000000 1.0000000000
## 32:      share_existence      1.0000 0.9998842092 0.9993735644 1.0000000000
## 33:      share_existence     10.0000 0.9987395427 0.9958634953 0.9994211288
## 34:      share_existence    501.1872 0.9265090021 0.8442199776 0.9606578698
## 35:      share_existence   1000.0000 0.8878074738 0.7835751053 0.9334183673
## 36:      share_existence   1584.8932 0.8549958612 0.7312409812 0.9107164472
## 37:      share_existence   1995.2623 0.8301866190 0.6886404631 0.8912397072
## 38:      share_existence   5011.8723 0.7100854672 0.4888273553 0.7785060526
## 39:      share_existence  10000.0000 0.5726446689 0.2750825646 0.6224474129
## 40:      share_existence  12589.2541 0.5234532434 0.2116215677 0.5748619682
## 41:      share_marginal      0.0000 0.0000000000 0.0000000000 0.0000000000
## 42:      share_marginal      1.0000 0.0001157908 0.0000000000 0.0006264356
## 43:      share_marginal     10.0000 0.0012604573 0.0005788712 0.0041365047
## 44:      share_marginal    501.1872 0.0734909979 0.0393421302 0.1557800224
## 45:      share_marginal   1000.0000 0.1121925262 0.0665816327 0.2164248947
## 46:      share_marginal   1584.8932 0.1450041388 0.0892835528 0.2687590188
## 47:      share_marginal   1995.2623 0.1698133810 0.1087602928 0.3113595369
## 48:      share_marginal   5011.8723 0.2899145328 0.2214939474 0.5111726447
## 49:      share_marginal  10000.0000 0.4273553311 0.3775525871 0.7249174354
## 50:      share_marginal  12589.2541 0.4765467566 0.4251380318 0.7883784323
##          agreement          tau      frac_mean      frac_lwr      frac_upr
##          <fctr>          <num>          <num>          <num>          <num>
```

```
# Plot uncertainty -----
```

```
agreement_cols <- c(frac_disagree = "black",
                    frac_major_disagree = "#B2182B",
                    frac_minor_disagree = "#2166AC" )

plot_uncertainty1 <- mat_sum[!agreement %in% c("share_existence", "share_marginal")] %>%
  ggplot(.,
    aes(x = tau, y = frac_median, colour = agreement, fill = agreement)) +
  geom_ribbon(aes(ymin = frac_lwr, ymax = frac_upr), alpha = 0.18, colour = NA) +
  geom_line(linewidth = 1.1) +
  scale_x_log10(labels = label_log(digits = 2)) +
  scale_colour_manual(values = agreement_cols,
                      breaks = c("frac_disagree",
```

```

        "frac_major_disagree",
        "frac_minor_disagree"),
    labels = c("Total",
               "Major",
               "Minor"),
    name = "Disagreement") +
scale_fill_manual(values = agreement_cols,
                  breaks = c("frac_disagree",
                             "frac_major_disagree",
                             "frac_minor_disagree"),
                  labels = c("Total",
                             "Major",
                             "Minor"),
                  name = "Disagreement") +
labs(x = expression(tau~"(ha)"), y = "Fraction of cells") +
theme_AP() +
theme(legend.position = c(0.65, 0.77))

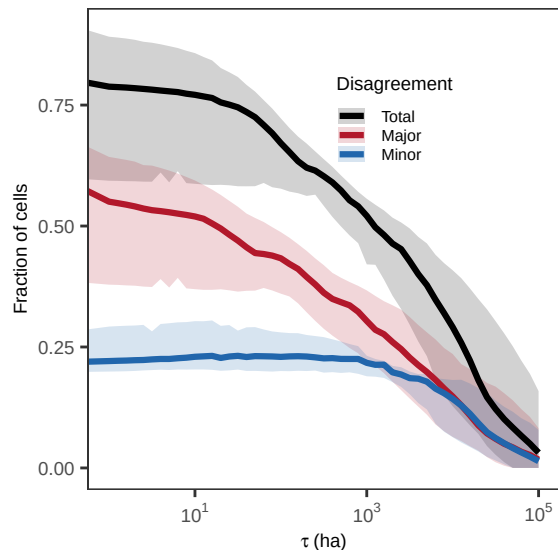
```

plot_uncertainty1

```

## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.

```



```

# Set colors for existential and marginal disagreement -----
exist_cols <- c("share_existence" = "#1f78b4", "share_marginal" = "#e66101")

plot_agreement1 <- mat_sum[agreement %in% c("share_existence", "share_marginal")] %>%
  ggplot(., aes(x = tau, y = frac_median, colour = agreement, fill = agreement)) +
  geom_ribbon(aes(ymin = frac_lwr, ymax = frac_upr), alpha = 0.18, colour = NA) +
  geom_line(linewidth = 1.1) +
  scale_colour_manual(values = exist_cols,

```

```

        labels = c(share_existence = "Existential",
                    share_marginal = "Marginal"),
        name = "Disagreement") +
scale_fill_manual(values = exist_cols,
                  labels = c(share_existence = "Existential",
                              share_marginal = "Marginal"),
                  name = "Disagreement") +
scale_x_log10(labels = label_log(digits = 2)) +
labs(x = expression(tau~"(ha)"), y = "Fraction of \ndisagreeing cells") +
theme_AP() +
theme(legend.position = c(0.37, 0.7))

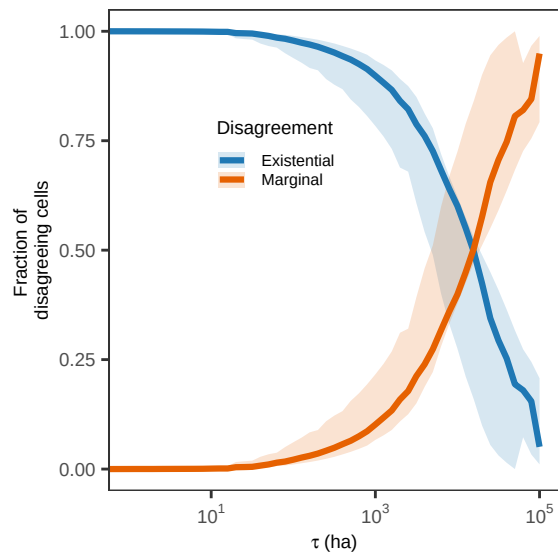
```

plot_agreement1

```

## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.

```



```

# SENSITIVITY ANALYSIS #####

```

```

# Sobol' indices -----

```

```

ind <- sobol_indices(matrices = matrices, N = N, params = params, Y
                    = mat[, frac_disagree], R = R, boot = TRUE)

```

```

print(ind)

```

```

##
## First-order estimator: saltelli | Total-order estimator: jansen
##
## Total number of model runs: 40960
##

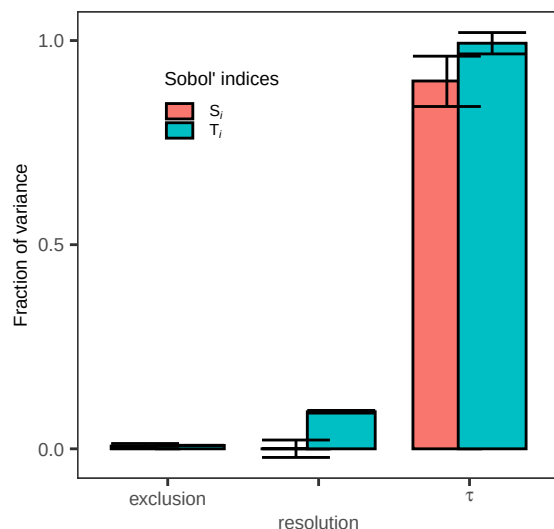
```



```
## Sum of first order indices: 0.9079082
##      original      bias    std.error    low.ci    high.ci
##      <num>      <num>      <num>      <num>      <num>
## 1: 0.0001506532 -3.275755e-05 0.0108794512 -0.0211399217 0.021506743
## 2: 0.9010276594 9.404230e-04 0.0314407547 0.8384644897 0.961709983
## 3: 0.0067298400 -2.710929e-06 0.0031939823 0.0004724605 0.012992641
## 4: 0.0907715303 2.194818e-06 0.0016776382 0.0874812249 0.094057446
## 5: 0.9935216113 6.024186e-05 0.0133459914 0.9673037070 1.019619032
## 6: 0.0083001208 8.542363e-06 0.0001909142 0.0079173935 0.008665763
##      sensitivity parameters
##      <char>      <char>
## 1:      Si resolution
## 2:      Si      tau
## 3:      Si exclusion
## 4:      Ti resolution
## 5:      Ti      tau
## 6:      Ti exclusion
```

```
plot_indices1 <- plot(ind) +
  labs(x = "", y = "Fraction of variance") +
  theme_AP() +
  scale_x_discrete(labels = c(exclusion = "exclusion",
                              resolution = "resolution",
                              tau = expression(tau)),
    guide = guide_axis(n.dodge = 2)) +
  theme(legend.position = c(0.3, 0.8))
```

```
plot_indices1
```

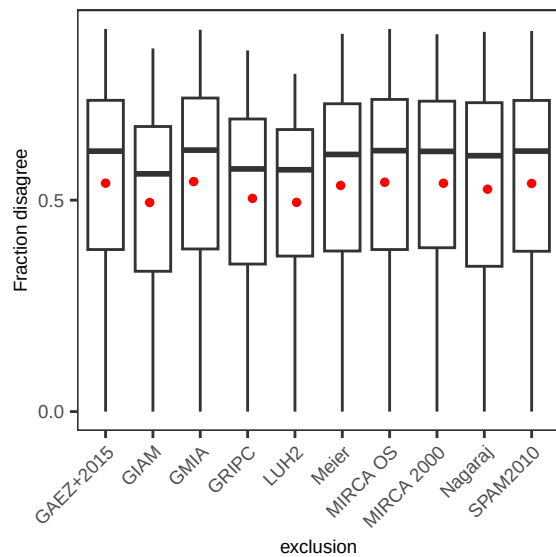


```
# SCATTERPLOTS OF OUTPUT AGAINST INPUT #####
```

```
label_map <- setNames(new_dataset_names, dataset_names)
```

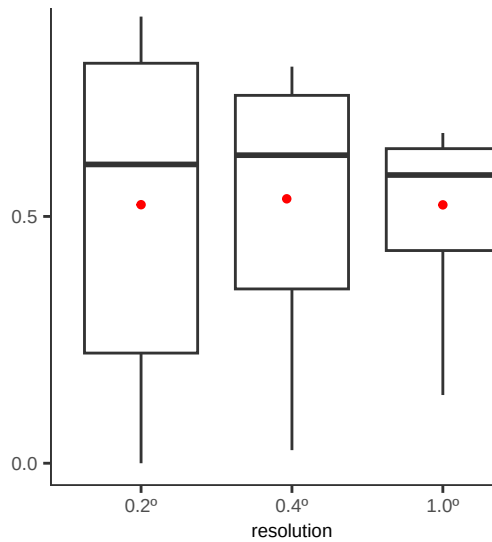
```
a1 <- ggplot(mat[!exclusion == "all", c("exclusion", "frac_disagree")],
             aes(exclusion, frac_disagree)) +
  geom_boxplot() +
  stat_summary_bin(fun = "mean", geom = "point",
                  colour = "red", size = 1) +
  scale_x_discrete(labels = label_map) +
  scale_y_continuous(breaks = breaks_pretty(n = 3)) +
  theme_AP() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(x = "exclusion", y = "Fraction disagree")
```

a1



```
b1 <- ggplot(mat[1:N, c("resolution", "frac_disagree")],
             aes(resolution, frac_disagree)) +
  geom_boxplot() +
  scale_x_discrete(labels = c(`0.2deg` = "0.2°",
                              `0.4deg` = "0.4°",
                              `1deg` = "1.0°")) +
  scale_y_continuous(breaks = breaks_pretty(n = 3)) +
  stat_summary_bin(fun = "mean", geom = "point",
                  colour = "red", size = 1) +
  theme_AP() +
  labs(x = "resolution", y = "")
```

b1

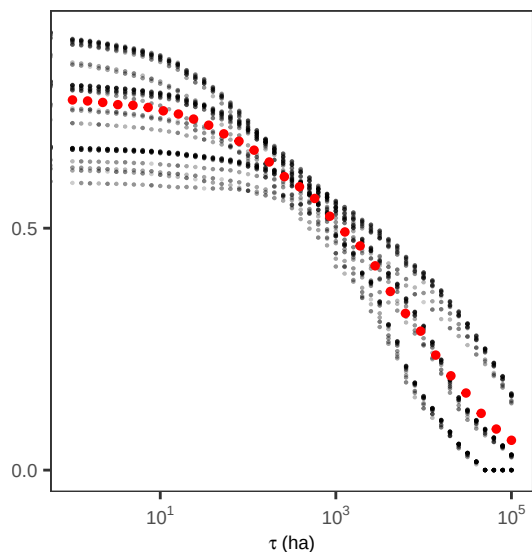


```
c1 <- ggplot(mat[1:N, c("tau", "frac_disagree")],
  aes(tau, frac_disagree)) +
  geom_point(size = 0.1, alpha = 0.1) +
  stat_summary_bin(fun = "mean", geom = "point",
    colour = "red", size = 1) +
  scale_y_continuous(breaks = breaks_pretty(n = 3)) +
  theme_AP() +
  scale_x_log10(labels = label_log(digits = 2)) +
  labs(x = expression(tau~"("ha)"), y = "")
```

c1

Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
log-10 transformation introduced infinite values.

Warning: Removed 157 rows containing non-finite outside the scale range
(`stat\_summary\_bin()`).



```
# PLOT DATASETS #####
```

```
tmp <- dt[resolution == "0.2deg"] %>%
  .[, .(mha = sum(mha)), dataset] %>%
  .[, continent:= "GLOBAL"]

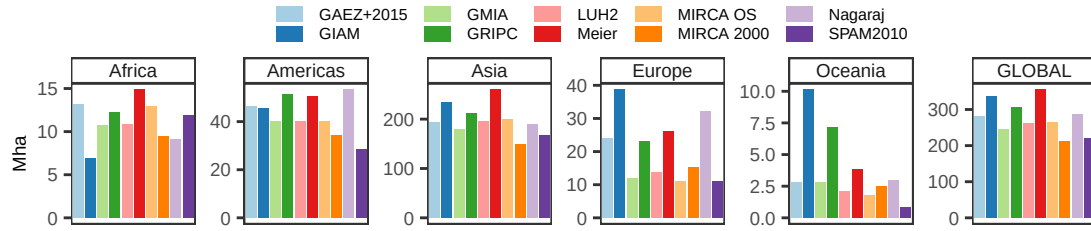
dt[resolution == "0.2deg"] %>%
  .[, .(mha = sum(mha)), .(continent, dataset)] %>%
  rbind(., tmp)
```

##	continent	dataset	mha
##	<char>	<char>	<num>
## 1:	Americas	gaez_v4	46.2015416
## 2:	Europe	gaez_v4	23.8489672
## 3:	Africa	gaez_v4	13.1101914
## 4:	Asia	gaez_v4	194.2821806
## 5:	Oceania	gaez_v4	2.8450580
## 6:	Americas	giam	45.5089223
## 7:	Europe	giam	38.6557015
## 8:	Africa	giam	6.9337475
## 9:	Asia	giam	233.9728781
## 10:	Oceania	giam	10.1185604
## 11:	Americas	gmia	40.2113502
## 12:	Europe	gmia	11.9504917
## 13:	Africa	gmia	10.7517031
## 14:	Asia	gmia	180.0792101
## 15:	Oceania	gmia	2.8368249
## 16:	Americas	gripc	51.2475536
## 17:	Europe	gripc	23.0350045
## 18:	Africa	gripc	12.2038719
## 19:	Asia	gripc	212.3461235
## 20:	Oceania	gripc	7.1657891
## 21:	Americas	luh2	40.0523677
## 22:	Europe	luh2	13.7826118
## 23:	Africa	luh2	10.8576084
## 24:	Asia	luh2	195.5457561
## 25:	Oceania	luh2	2.1157916
## 26:	Americas	meier	50.4449965
## 27:	Europe	meier	26.0018526
## 28:	Africa	meier	14.8522635
## 29:	Asia	meier	259.4777712
## 30:	Oceania	meier	3.8551223
## 31:	Americas	mirca2000	34.4909117
## 32:	Europe	mirca2000	15.3232002
## 33:	Africa	mirca2000	9.4163731
## 34:	Asia	mirca2000	148.8331835
## 35:	Oceania	mirca2000	2.4615961

```
## 36: Americas mirca_os 40.1775798
## 37: Europe mirca_os 10.9262841
## 38: Africa mirca_os 12.9762031
## 39: Asia mirca_os 199.6637856
## 40: Oceania mirca_os 1.7670877
## 41: Americas nagaraj 53.2348690
## 42: Europe nagaraj 32.0035173
## 43: Africa nagaraj 9.0617581
## 44: Asia nagaraj 188.5240888
## 45: Oceania nagaraj 2.9588799
## 46: Americas spam 28.3824799
## 47: Europe spam 10.9216630
## 48: Africa spam 11.8448877
## 49: Asia spam 167.5769329
## 50: Oceania spam 0.8283964
## 51: GLOBAL gaez_v4 280.2879388
## 52: GLOBAL giam 335.1898098
## 53: GLOBAL gmia 245.8295799
## 54: GLOBAL gripc 305.9983426
## 55: GLOBAL luh2 262.3541356
## 56: GLOBAL meier 354.6320061
## 57: GLOBAL mirca2000 210.5252647
## 58: GLOBAL mirca_os 265.5109403
## 59: GLOBAL nagaraj 285.7831131
## 60: GLOBAL spam 219.5543599
## continent dataset mha
## <char> <char> <num>
```

```
dt[resolution == "0.2deg"] %>%
  .[, .(mha = sum(mha)), .(continent, dataset)] %>%
  rbind(., tmp) %>%
  .[, continent := factor(continent, levels = c("Africa", "Americas",
                                                "Asia", "Europe", "Oceania",
                                                "GLOBAL"))] %>%

  ggplot(., aes(dataset, mha, fill = dataset)) +
  geom_bar(stat = "identity") +
  scale_fill_brewer(palette = "Paired", label = label_map, name = "") +
  facet_wrap(~continent, ncol = 6, scales = "free_y") +
  theme_AP() +
  labs(x = "", y = "Mha") +
  theme(legend.position = "top") +
  theme(axis.text.x = element_blank(),
        axis.ticks.x = element_blank())
```



MERGE UA/SA PLOTS

```
top <- plot_grid(plot_uncertainty1, plot_agreement1, plot_indices1,
  ncol = 3, labels = "auto", rel_widths = c(0.4, 0.3, 0.3))
```

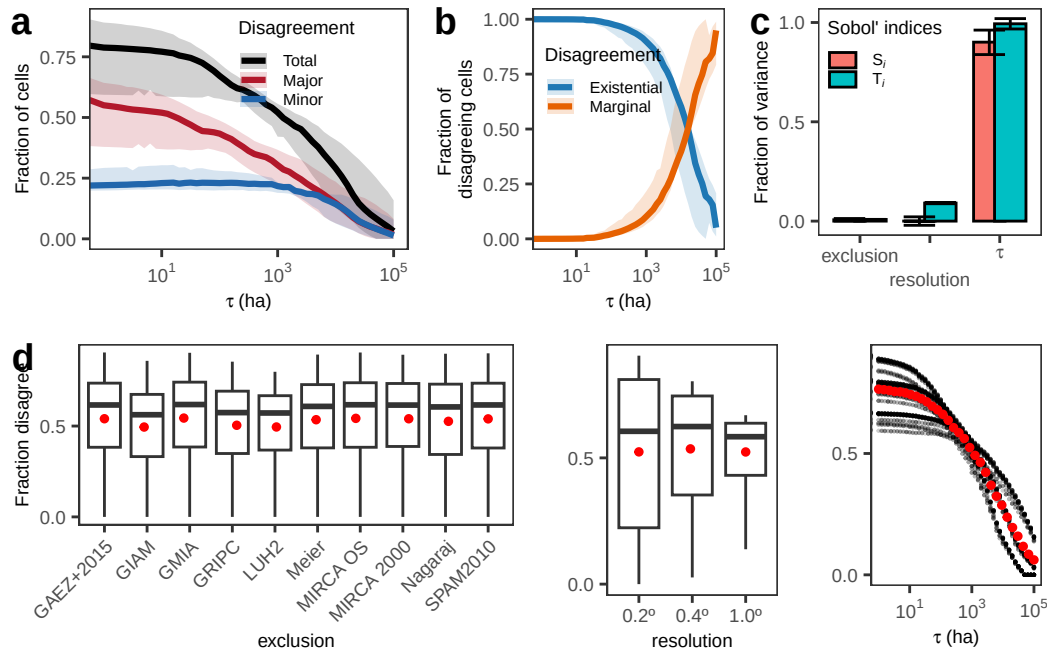
```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.
## log-10 transformation introduced infinite values.
## log-10 transformation introduced infinite values.
```

```
bottom <- plot_grid(a1, b1, c1, ncol = 3, rel_widths = c(0.5, 0.25, 0.25), labels = "d")
```

```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.
```

```
## Warning: Removed 157 rows containing non-finite outside the scale range
## (`stat_summary_bin()`).
```

```
plot_grid(top, bottom, ncol = 1)
```



2.2 Does coarsening reduce disagreement?

DOES ZOOMING OUT REDUCE DISAGREEMENT?

```

effect_resolution_dt <- tau_results[scenario == "all", .(tau, frac_disagree, resolution)]

# Order resolution -----

effect_resolution_dt[, resolution:= factor(resolution,
                                           levels = c("0.2deg", "0.4deg", "1deg"))]

# To wide table -----

wide_dt <- dcast(effect_resolution_dt, tau ~ resolution, value.var = "frac_disagree")

# Rename -----

setnames(wide_dt, old = c("0.2deg", "0.4deg", "1deg"), new = c("res_0.2", "res_0.4", "res_1"))

# Compute absolute change (fine - coarse) -----

wide_dt[, abs_change:= res_0.2 - res_1]

# Compute elasticity (log-log) -----

# Area ratio from 0.2 to 1
area_ratio <- 25
wide_dt[, elasticity:= ifelse(res_0.2 > 0 & res_1 > 0,
                             log(res_1 / res_0.2) / log(area_ratio), NA_real_)]

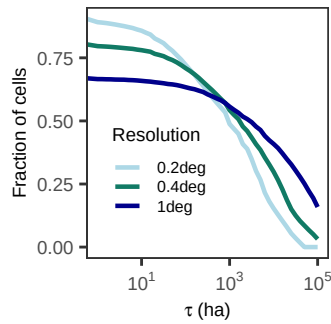
# Plot disagreement curves -----

p1 <- ggplot(effect_resolution_dt, aes(tau, frac_disagree, colour = resolution)) +
  geom_line(linewidth = 0.8) +
  scale_x_log10(labels = label_log(digits = 2)) +
  scale_color_manual(values = res_palette) +
  labs(x = expression(tau~"(ha)"),
       y = "Fraction of cells",
       colour = "Resolution") +
  theme_AP() +
  theme(legend.position = c(0.3, 0.35))

p1

## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation
## introduced infinite values.

```

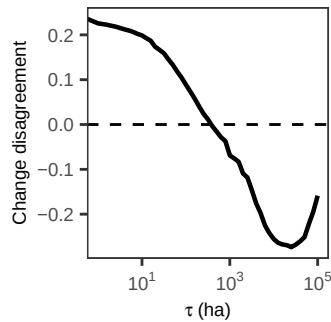


Plot absolute change -----

```
p2 <- ggplot(wide_dt, aes(tau, abs_change)) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_line(linewidth = 0.8) +
  scale_x_log10(labels = label_log(digits = 2)) +
  labs(x = expression(tau~"(ha)"),
       y = "Change disagreement") +
  theme_AP()
```

p2

Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation
introduced infinite values.

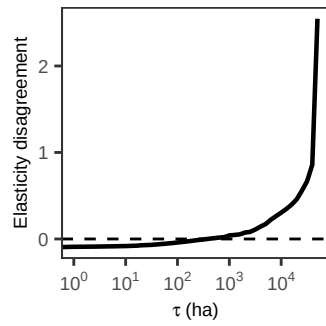


Plot elasticity -----

```
p3 <- ggplot(wide_dt[!is.na(elasticity)],
  aes(x = tau, y = elasticity)) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_line(linewidth = 0.8) +
  scale_x_log10(labels = label_log(digits = 2)) +
  labs(x = expression(tau~"(ha)"),
       y = "Elasticity disagreement") +
  theme_AP()
```

p3

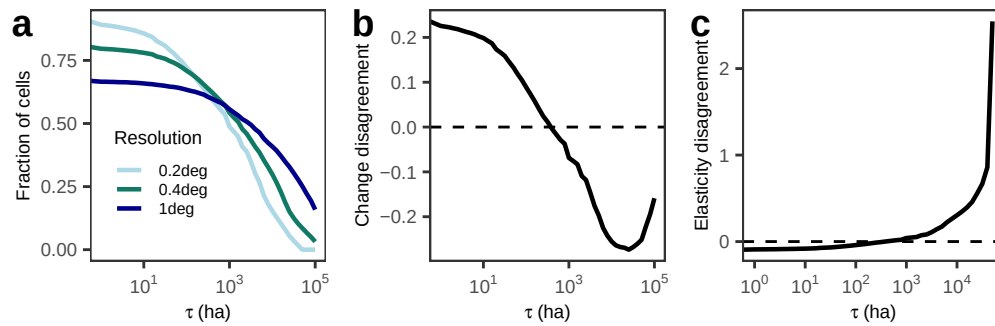
Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation
introduced infinite values.



```
# Merge -----

plot_grid(p1, p2, p3, ncol = 3, labels = "auto")
```

```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.
## log-10 transformation introduced infinite values.
```



2.3 UA/SA by excluding mapping paradigms

##	agreement	tau	frac_mean	frac_median	frac_lwr
##	<fctr>	<num>	<num>	<num>	<num>
## 1:	frac_disagree	0.0000	0.7430530248	0.7755168771	0.5669310858
## 2:	frac_disagree	1.0000	0.7333800619	0.7621964390	0.5613534953
## 3:	frac_disagree	10.0000	0.7137910743	0.7146835560	0.5469757065
## 4:	frac_disagree	501.1872	0.5389292065	0.5637516586	0.4301616935
## 5:	frac_disagree	1000.0000	0.4806388477	0.5117712159	0.3618199772
## 6:	frac_disagree	1584.8932	0.4489805921	0.4738165904	0.3457740612
## 7:	frac_disagree	1995.2623	0.4239791101	0.4483877459	0.3116302034
## 8:	frac_disagree	5011.8723	0.3349236070	0.3436038030	0.2032739840
## 9:	frac_disagree	10000.0000	0.2572035781	0.2772682201	0.1073456352
## 10:	frac_disagree	12589.2541	0.2340514119	0.2518989889	0.0908059987
## 11:	frac_major_disagree	0.0000	0.5031937420	0.4994215001	0.3299454636
## 12:	frac_major_disagree	1.0000	0.4896659751	0.4891594145	0.3238720873
## 13:	frac_major_disagree	10.0000	0.4602356204	0.4695407214	0.3087506197
## 14:	frac_major_disagree	501.1872	0.3058621258	0.3168921978	0.2007899528
## 15:	frac_major_disagree	1000.0000	0.2646720627	0.2671663565	0.1717144444
## 16:	frac_major_disagree	1584.8932	0.2411082172	0.2412093164	0.1586694233
## 17:	frac_major_disagree	1995.2623	0.2268283381	0.2210372755	0.1458326905
## 18:	frac_major_disagree	5011.8723	0.1711008105	0.1681171085	0.0920017280
## 19:	frac_major_disagree	10000.0000	0.1291880616	0.1322249610	0.0532755269
## 20:	frac_major_disagree	12589.2541	0.1151898186	0.1158005936	0.0453837134
## 21:	frac_minor_disagree	0.0000	0.2398592828	0.2369856222	0.2027764006
## 22:	frac_minor_disagree	1.0000	0.2437140868	0.2374814080	0.2047595439
## 23:	frac_minor_disagree	10.0000	0.2535554539	0.2493837718	0.2100892414
## 24:	frac_minor_disagree	501.1872	0.2330670807	0.2306643530	0.2063708478
## 25:	frac_minor_disagree	1000.0000	0.2159667850	0.2171889934	0.1901055328
## 26:	frac_minor_disagree	1584.8932	0.2078723749	0.2062469013	0.1787816289
## 27:	frac_minor_disagree	1995.2623	0.1971507720	0.1975706495	0.1633633483
## 28:	frac_minor_disagree	5011.8723	0.1638227966	0.1626339353	0.1112722560
## 29:	frac_minor_disagree	10000.0000	0.1280155164	0.1342340109	0.0540701083
## 30:	frac_minor_disagree	12589.2541	0.1188615933	0.1276221138	0.0454222853
## 31:	share_existence	0.0000	1.0000000000	1.0000000000	1.0000000000
## 32:	share_existence	1.0000	0.9997672523	0.9998374407	0.9993735644
## 33:	share_existence	10.0000	0.9982965766	0.9988512349	0.9958634953
## 34:	share_existence	501.1872	0.9168285618	0.9332604417	0.8442199776
## 35:	share_existence	1000.0000	0.8703566888	0.8809219223	0.7835751053
## 36:	share_existence	1584.8932	0.8326718403	0.8422081094	0.7312409812
## 37:	share_existence	1995.2623	0.8046116385	0.8164020867	0.6886404631
## 38:	share_existence	5011.8723	0.6665122263	0.6842837274	0.4888273553
## 39:	share_existence	10000.0000	0.5232850394	0.5448511469	0.2750825646
## 40:	share_existence	12589.2541	0.4719699653	0.4993270525	0.2116215677
## 41:	share_marginal	0.0000	0.0000000000	0.0000000000	0.0000000000
## 42:	share_marginal	1.0000	0.0002327477	0.0001625593	0.0000000000
## 43:	share_marginal	10.0000	0.0017034234	0.0011487651	0.0004279575
## 44:	share_marginal	501.1872	0.0831714382	0.0667395583	0.0382878568

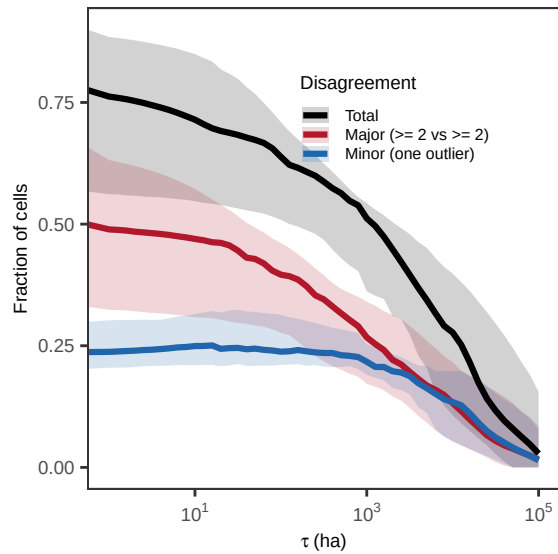
```

## 45:      share_marginal  1000.0000 0.1296433112 0.1190780777 0.0644334054
## 46:      share_marginal  1584.8932 0.1673281597 0.1577918906 0.0856892626
## 47:      share_marginal  1995.2623 0.1953883615 0.1835979133 0.1052967454
## 48:      share_marginal  5011.8723 0.3334877737 0.3157162726 0.2163691902
## 49:      share_marginal 10000.0000 0.4767149606 0.4551488531 0.3759911779
## 50:      share_marginal 12589.2541 0.5280300347 0.5006729475 0.4259939860
##          agreement      tau    frac_mean  frac_median    frac_lwr
##          <fctr>      <num>      <num>      <num>      <num>
##          frac_upr
##          <num>
##  1: 0.8997747400
##  2: 0.8846468356
##  3: 0.8495463943
##  4: 0.5901705317
##  5: 0.5430094199
##  6: 0.5178482895
##  7: 0.5040902330
##  8: 0.4510411502
##  9: 0.3983639068
## 10: 0.3808874566
## 11: 0.6584379918
## 12: 0.6317153701
## 13: 0.5720677631
## 14: 0.3672533466
## 15: 0.3403569658
## 16: 0.3210213188
## 17: 0.3134605850
## 18: 0.2595438770
## 19: 0.2165344571
## 20: 0.1985622211
## 21: 0.2993566205
## 22: 0.3024038016
## 23: 0.3153176783
## 24: 0.2842595735
## 25: 0.2525534607
## 26: 0.2416298948
## 27: 0.2325569697
## 28: 0.2086018840
## 29: 0.1979424888
## 30: 0.1979424888
## 31: 1.0000000000
## 32: 1.0000000000
## 33: 0.9995720425
## 34: 0.9617121432
## 35: 0.9355665946
## 36: 0.9143107374
## 37: 0.8947032546
## 38: 0.7836308098

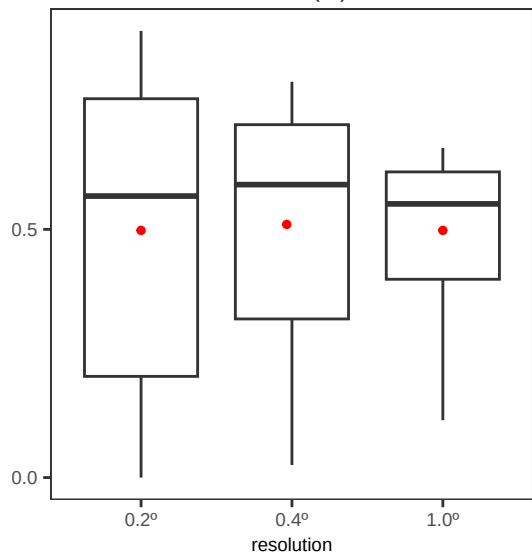
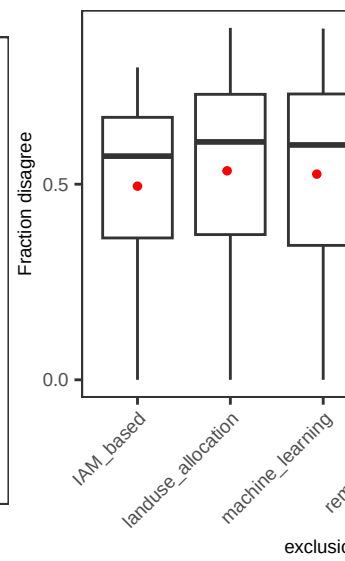
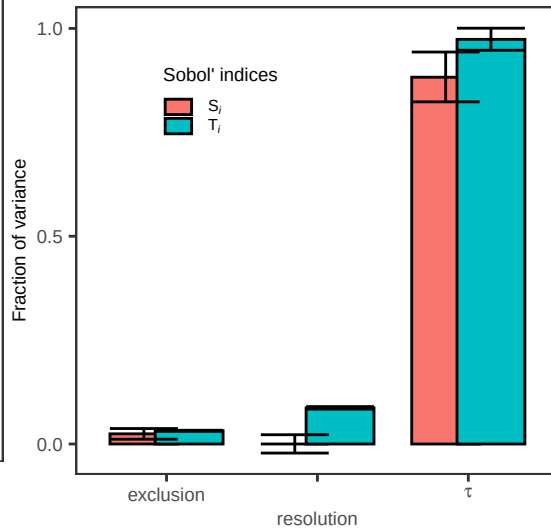
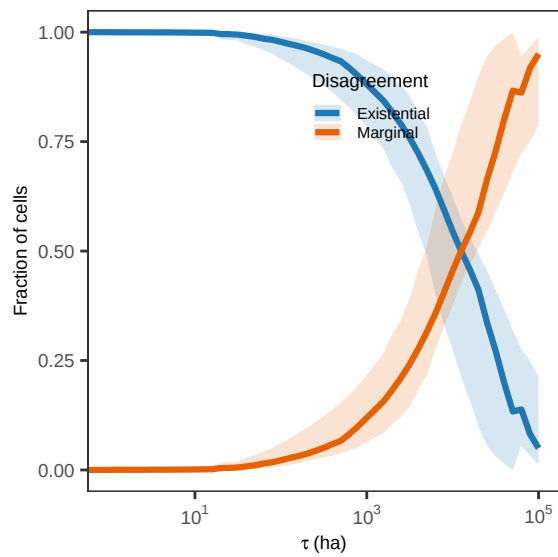
```

```
## 39: 0.6240088221
## 40: 0.5740060140
## 41: 0.0000000000
## 42: 0.0006264356
## 43: 0.0041365047
## 44: 0.1557800224
## 45: 0.2164248947
## 46: 0.2687590188
## 47: 0.3113595369
## 48: 0.5111726447
## 49: 0.7249174354
## 50: 0.7883784323
##          frac_upr
##          <num>
```

```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.
```

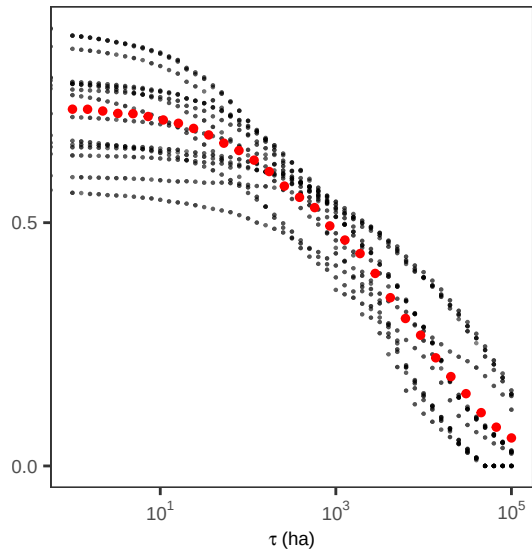


```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.
```



```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.
```

```
## Warning: Removed 157 rows containing non-finite outside the scale range
## (`stat_summary_bin()`).
```



```
# MERGE UA/SA PLOTS #####
```

```
top <- plot_grid(plot_uncertainty, plot_agreement, plot_indices2,
                 ncol = 3, labels = "auto", rel_widths = c(0.4, 0.3, 0.3))
```

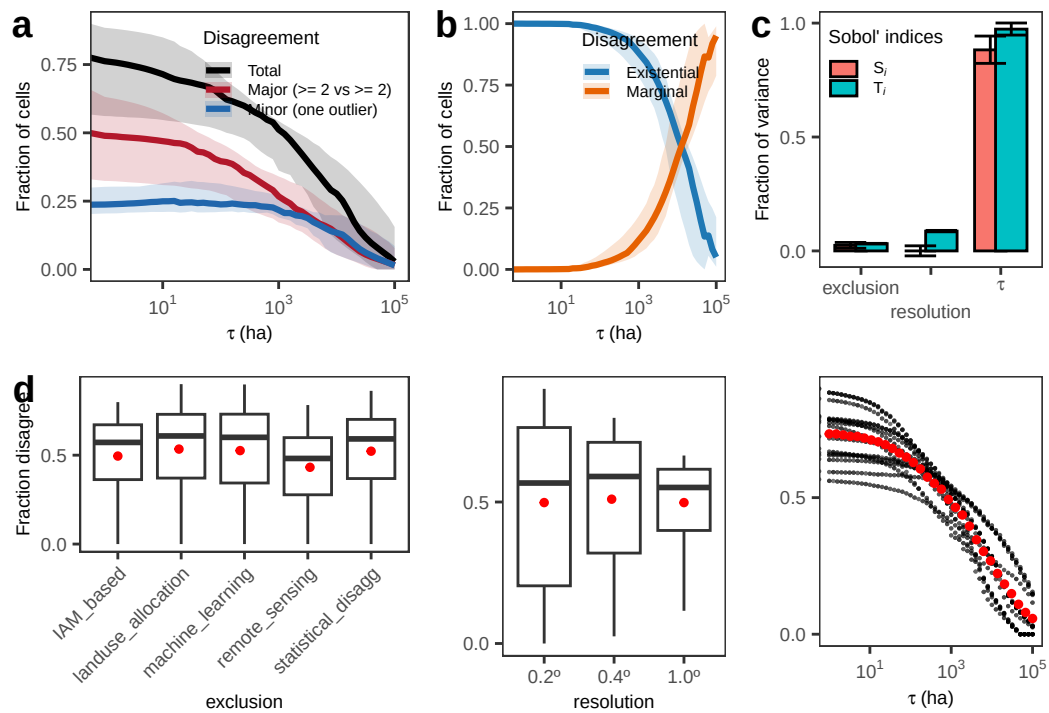
```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.
## log-10 transformation introduced infinite values.
## log-10 transformation introduced infinite values.
```

```
bottom <- plot_grid(a2, b2, c2, ncol = 3, rel_widths = c(0.4, 0.3, 0.3), labels = "d")
```

```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.
```

```
## Warning: Removed 157 rows containing non-finite outside the scale range
## (`stat_summary_bin()`).
```

```
plot_grid(top, bottom, ncol = 1)
```

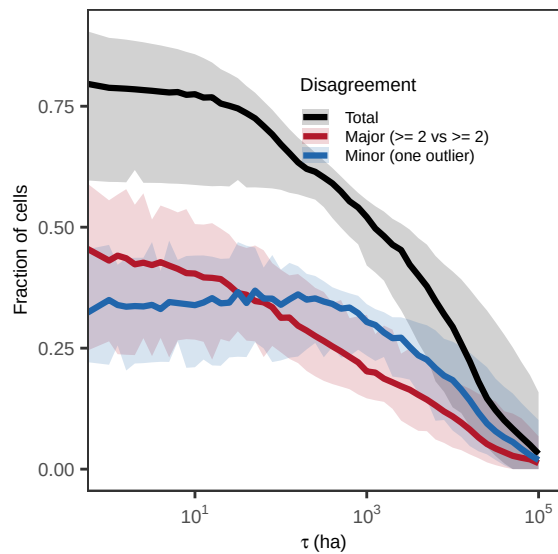


2.4 UA/SA by weighting newer datasets more

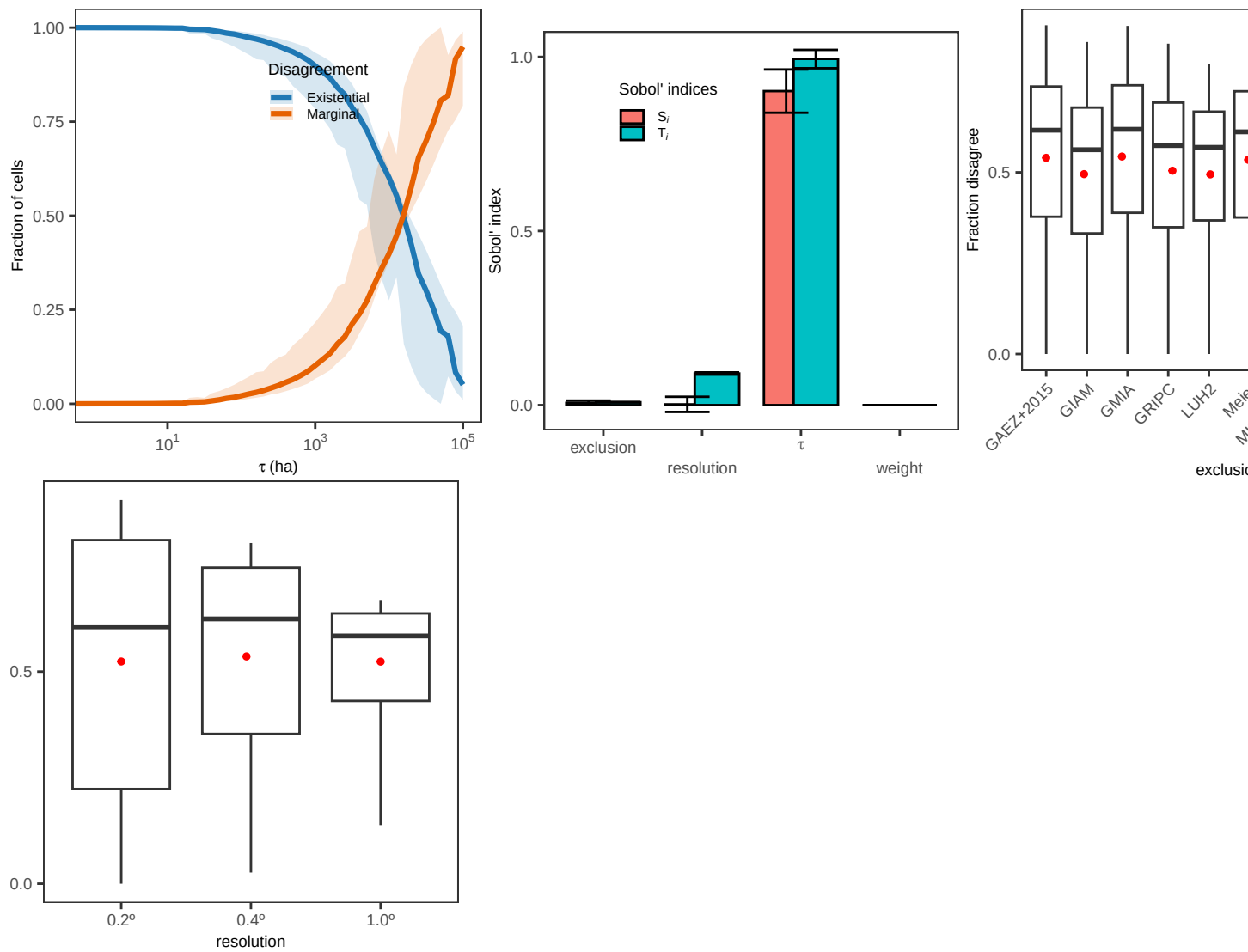
##	tau	agreement	frac_mean	frac_median	frac_lwr
##	<num>	<fctr>	<num>	<num>	<num>
## 1:	10	frac_disagree	0.7447800994	7.748378e-01	0.5872583044
## 2:	100	frac_disagree	0.6666677100	6.712284e-01	0.5772186415
## 3:	1	frac_disagree	0.7650881937	7.880427e-01	0.5935795736
## 4:	0	frac_disagree	0.7716544945	7.962423e-01	0.5965542885
## 5:	1000	frac_disagree	0.5103718881	5.209518e-01	0.4207578610
## 6:	10	frac_major_disagree	0.3946974528	4.041954e-01	0.2676003966
## 7:	100	frac_major_disagree	0.3177491105	3.127169e-01	0.2413907489
## 8:	1	frac_major_disagree	0.4229582141	4.310257e-01	0.2640772186
## 9:	0	frac_major_disagree	0.4404037495	4.551409e-01	0.2451660882
## 10:	1000	frac_major_disagree	0.2097105925	2.020327e-01	0.1462294026
## 11:	10	frac_minor_disagree	0.3500826466	3.388696e-01	0.2105850273
## 12:	100	frac_minor_disagree	0.3489185996	3.395040e-01	0.2245909767
## 13:	1	frac_minor_disagree	0.3421299796	3.495649e-01	0.2156668319
## 14:	0	frac_minor_disagree	0.3312507450	3.234318e-01	0.2209779375
## 15:	1000	frac_minor_disagree	0.3006612956	3.040855e-01	0.2345066931
## 16:	10	share_existence	0.9987395006	9.990102e-01	0.9958634953
## 17:	100	share_existence	0.9749187906	9.785986e-01	0.9388543231
## 18:	1	share_existence	0.9998888226	9.999368e-01	0.9993735644
## 19:	0	share_existence	1.0000000000	1.000000e+00	1.0000000000
## 20:	1000	share_existence	0.8881529264	8.982820e-01	0.7835751053
## 21:	10	share_marginal	0.0012604994	9.897843e-04	0.0006042296
## 22:	100	share_marginal	0.0250812094	2.140140e-02	0.0104125963
## 23:	1	share_marginal	0.0001111774	6.324311e-05	0.0000000000
## 24:	0	share_marginal	0.0000000000	0.000000e+00	0.0000000000
## 25:	1000	share_marginal	0.1118470736	1.017180e-01	0.0665816327
##	tau	agreement	frac_mean	frac_median	frac_lwr
##	<num>	<fctr>	<num>	<num>	<num>
##	frac_upr				
##	<num>				
## 1:	0.8574304934				
## 2:	0.7225445120				
## 3:	0.8913352054				
## 4:	0.9047505169				
## 5:	0.5563956371				
## 6:	0.5064418393				
## 7:	0.4121183158				
## 8:	0.5555861863				
## 9:	0.5899728454				
## 10:	0.3187902826				
## 11:	0.4413451381				
## 12:	0.4123195332				
## 13:	0.4634183973				
## 14:	0.4544157127				
## 15:	0.3409766981				


```
## 16: 0.9993957704
## 17: 0.9895874037
## 18: 1.0000000000
## 19: 1.0000000000
## 20: 0.9334183673
## 21: 0.0041365047
## 22: 0.0611456769
## 23: 0.0006264356
## 24: 0.0000000000
## 25: 0.2164248947
##          frac_upr
##          <num>
```

```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.
```

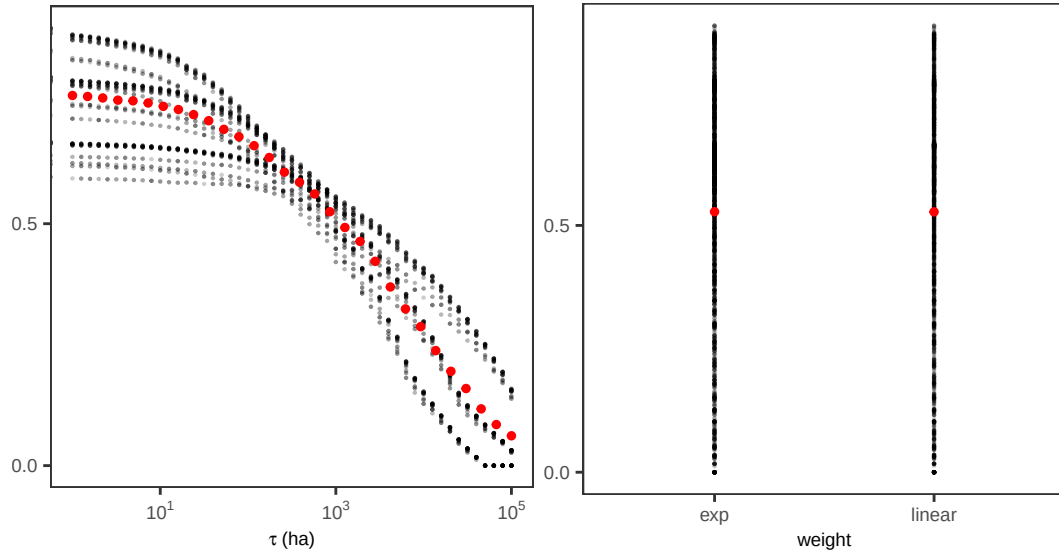


```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.
```



```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.
```

```
## Warning: Removed 157 rows containing non-finite outside the scale range
## (`stat_summary_bin()`).
```



```
# MERGE UA/SA PLOTS #####
```

```
top <- plot_grid(plot_uncertainty, plot_agreement, plot_indices3,
  ncol = 3, labels = "auto", rel_widths = c(0.4, 0.3, 0.3))
```

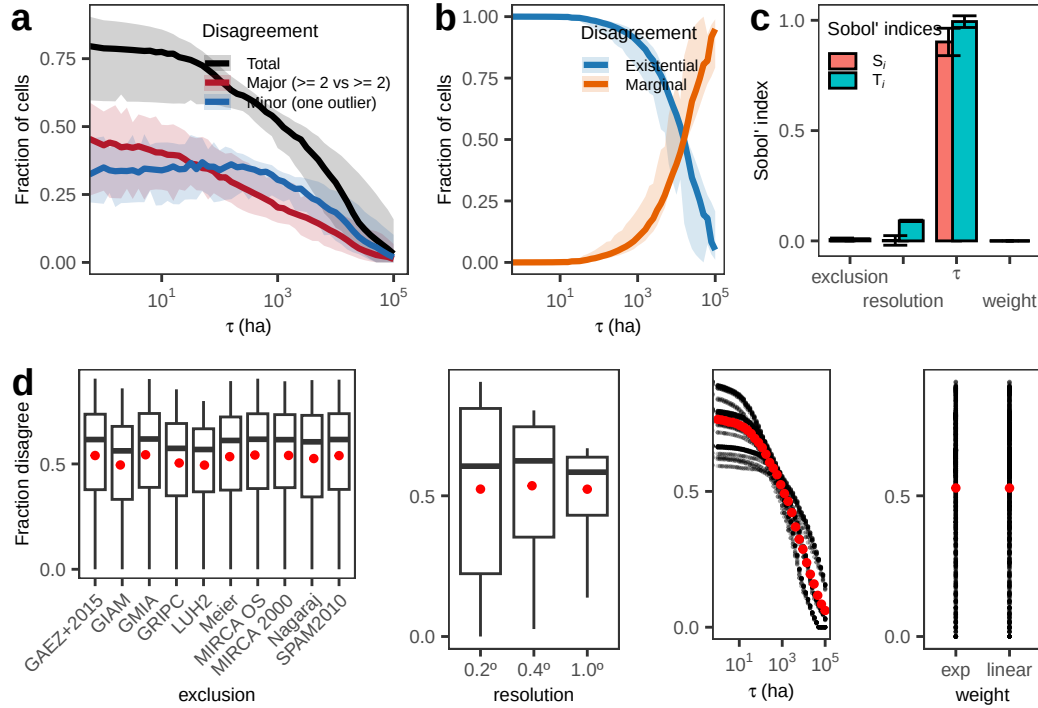
```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.
## log-10 transformation introduced infinite values.
## log-10 transformation introduced infinite values.
```

```
bottom <- plot_grid(a3, b3, c3, d3, ncol = 4, rel_widths = c(0.35, 0.25, 0.2, 0.2), labels = "auto")
```

```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.
```

```
## Warning: Removed 157 rows containing non-finite outside the scale range
## (`stat_summary_bin()`).
```

```
plot_grid(top, bottom, ncol = 1)
```



2.5 UA/SA on mapping variables

```
# MAPPING PARADIGM EXCLUSION UA/SA #####

ds_groups <- list(AEI = c("meier", "nagaraj", "gaez_v4", "spam", "luh2"),
                 AAI = c("giam", "gmia", "gripc", "mirca2000", "mirca_os"))

mapping_paradigm_vec <- names(ds_groups)
n_mapping_paradigm_vec <- length(mapping_paradigm_vec)

# Create the sample matrix -----

params <- c("resolution", "tau", "exclusion")
mat <- data.table(sobol_matrices(matrices = matrices, N = N, params = params))

# Transform resolution column -----

mat[, resolution:= pmin(n_resolution_vec, 1L + floor(resolution * n_resolution_vec))]
mat[, resolution:= resolution_vec[resolution]]

# Transform tau column -----

mat[, tau:= pmin(n_tau_vec, 1L + floor(tau * n_tau_vec))]
mat[, tau:= tau_vec[tau]]

# Transform exclusion column -----
```

```

mat[, exclusion:= pmin(n_mapping_paradigm_vec, 1L + floor(exclusion * n_mapping_paradigm_vec))
mat[, exclusion:= mapping_paradigm_vec[exclusion]]

# PRE-COMPUTE ALL COMBINATIONS #####

# Compute all possibilities by looping over resolutions and tau values -----

# Set parallel backend -----

resolutions <- sort(unique(dt$resolution))
ncores <- max(1, parallel::detectCores() - 1)

# ds_groups: named list of dataset keys to exclude by mapping category
# mat: data.table with columns resolution, tau (in ha), exclusion (category name)

idcols <- c("lon", "lat", "country", "code", "continent")

tau_results <- rbindlist(
  parallel::mclapply(resolutions, function(res) {

    dt_res_long <- dt[resolution == res]
    if (nrow(dt_res_long) == 0L) return(NULL)

    # scenarios to run for this resolution
    mat_res <- mat[resolution == res]
    if (nrow(mat_res) == 0L) return(NULL)

    # datasets present at this resolution
    ds_res <- sort(unique(dt_res_long$dataset))

    # ---- CAST ONCE per resolution ----

    dt_res_wide_all <- dcast(dt_res_long,
                           lon + lat + country + code + continent ~ dataset,
                           value.var = "mha", fill = 0)

    # loop exclusion groups (few) instead of mat rows (many)
    excl_levels <- unique(mat_res$exclusion)

    rbindlist(lapply(excl_levels, function(excl_grp) {

      # datasets to exclude for this group
      excl_ds <- ds_groups[[excl_grp]]
      if (is.null(excl_ds)) excl_ds <- character(0)

      use_ds <- setdiff(ds_res, excl_ds)
      if (length(use_ds) < 2L) return(NULL)
    })
  })

```

```

# taus used for thsiu (resolution, exclusion-group)
taus_ha <- unique(mat_res[exclusion == excl_grp, tau])
taus_mha <- taus_ha * 1e-6

# ---- compute A_min/A_max once per exclusion group ----
dt_w <- dt_res_wide_all[, c(idcols, use_ds), with = FALSE]

dt_w[, `:=`(A_min = do.call(pmin, c(.SD, list(na.rm = TRUE))),
           A_max = do.call(pmax, c(.SD, list(na.rm = TRUE))),
           .SDcols = use_ds)]

# ---- now only loop over taus
out_grp <- rbindlist(lapply(seq_along(taus_mha), function(k) {

  tau_ha <- taus_ha[k]
  tau_mha <- taus_mha[k]

  out <- compute_tau_fun(dt_w, tau_mha, use_ds)

  out[, `:=`(
    tau_label = paste0(format(tau_ha, scientific = FALSE, trim = TRUE), "ha"),
    tau_ha = tau_ha,
    tau_mha = tau_mha,
    resolution = res,
    exclusion = excl_grp
  )]

  out
}), use.names = TRUE, fill = TRUE)

out_grp
}), use.names = TRUE, fill = TRUE)

}, mc.cores = ncores),
use.names = TRUE, fill = TRUE
)

# ARRANGE OUTPUT #####

# Rename tau_ha to tau to match mat -----

setnames(tau_results, "tau_ha", "tau")

# Join all three disagreement metrics onto mat -----

mat[tau_results, on = .(resolution, tau, exclusion = exclusion),
  `:=`(frac_disagree = i.frac_disagree,

```

```

    frac_major_disagree = i.frac_major_disagree,
    frac_minor_disagree = i.frac_minor_disagree,
    share_existence = i.share_existence,
    share_marginal = i.share_marginal)]

# Reshape just these three for plotting -----

mat_long <- melt(mat, id.vars = c("resolution", "tau", "exclusion"),
  measure.vars = c("frac_disagree", "frac_major_disagree",
    "frac_minor_disagree", "share_existence",
    "share_marginal"),
  variable.name = "agreement", value.name = "frac_value")

# PREPARE DATA TO PLOT UNCERTAINTY #####

# Calculate mean and quantiles by tau x agreement -----

mat_sum <- mat_long[, .(frac_mean = mean(frac_value, na.rm = TRUE),
  frac_median = median(frac_value, na.rm = TRUE),
  frac_lwr = quantile(frac_value, 0.025, na.rm = TRUE),
  frac_upr = quantile(frac_value, 0.975, na.rm = TRUE)),
  .(tau, agreement)]

mat_sum[, {dt_sub <- .SD
  idx <- sapply(target_tau, function(x) {
    which.min(abs(log10(tau + 1e-12) - log10(x + 1e-12)))
  })
  dt_sub[idx]
},
agreement
]

```

##	agreement	tau	frac_mean	frac_median	frac_lwr
##	<fctr>	<num>	<num>	<num>	<num>
## 1:	frac_disagree	0.0000	0.632763385	0.6411539816	0.4969013386
## 2:	frac_disagree	1.0000	0.626999490	0.6402736556	0.4967773922
## 3:	frac_disagree	10.0000	0.606632456	0.6313949394	0.4939266237
## 4:	frac_disagree	501.1872	0.442056757	0.4188292653	0.4095025766
## 5:	frac_disagree	1000.0000	0.390105738	0.3894397620	0.3426651649
## 6:	frac_disagree	1584.8932	0.360526361	0.3355802606	0.2924753911
## 7:	frac_disagree	1995.2623	0.339223484	0.3515121468	0.2679050822
## 8:	frac_disagree	5011.8723	0.264034661	0.2756426380	0.1703027124
## 9:	frac_disagree	10000.0000	0.201851184	0.2132652548	0.1015444194
## 10:	frac_disagree	12589.2541	0.181480564	0.1767443030	0.0863702287
## 11:	frac_major_disagree	0.0000	0.315958611	0.3597766487	0.1761279127
## 12:	frac_major_disagree	1.0000	0.308318422	0.2884702450	0.1798463064

```

## 13: frac_major_disagree      10.0000 0.284691982 0.2813270285 0.1813336639
## 14: frac_major_disagree      501.1872 0.182193883 0.1683201160 0.1477149998
## 15: frac_major_disagree     1000.0000 0.159155914 0.1683192861 0.1287839047
## 16: frac_major_disagree     1584.8932 0.143161615 0.1294833744 0.1065587682
## 17: frac_major_disagree     1995.2623 0.134430265 0.1443976202 0.0949177647
## 18: frac_major_disagree     5011.8723 0.100141504 0.1012877911 0.0557904157
## 19: frac_major_disagree    10000.0000 0.075214507 0.0770410986 0.0366356034
## 20: frac_major_disagree    12589.2541 0.066654489 0.0598118618 0.0314283951
## 21: frac_minor_disagree        0.0000 0.316804774 0.3207734259 0.2500000000
## 22: frac_minor_disagree        1.0000 0.318681069 0.3295744083 0.2516113039
## 23: frac_minor_disagree       10.0000 0.321940474 0.3500679109 0.2547099653
## 24: frac_minor_disagree       501.1872 0.259862874 0.2538357060 0.2374814080
## 25: frac_minor_disagree      1000.0000 0.230949824 0.2211204760 0.2138812602
## 26: frac_minor_disagree      1584.8932 0.217364746 0.2147375567 0.1859166230
## 27: frac_minor_disagree      1995.2623 0.204793220 0.2071145265 0.1729873176
## 28: frac_minor_disagree      5011.8723 0.163893157 0.1743548468 0.1145122967
## 29: frac_minor_disagree     10000.0000 0.126636677 0.1362241561 0.0635819422
## 30: frac_minor_disagree     12589.2541 0.114826075 0.1169324413 0.0526969482
## 31:      share_existence         0.0000 1.0000000000 1.0000000000 1.0000000000
## 32:      share_existence         1.0000 0.997429207 0.9996449844 0.9927644711
## 33:      share_existence        10.0000 0.990364277 0.9854812471 0.9796737767
## 34:      share_existence       501.1872 0.848206846 0.9383355414 0.7107206519
## 35:      share_existence      1000.0000 0.778994714 0.6961660551 0.6343482906
## 36:      share_existence      1584.8932 0.725210004 0.8147204317 0.5797872340
## 37:      share_existence      1995.2623 0.686576017 0.5903933691 0.5496726445
## 38:      share_existence      5011.8723 0.516795763 0.4410986404 0.3880684907
## 39:      share_existence     10000.0000 0.363638334 0.3178324366 0.2300267719
## 40:      share_existence     12589.2541 0.315040530 0.2801972062 0.1763865546
## 41:      share_marginal         0.0000 0.0000000000 0.0000000000 0.0000000000
## 42:      share_marginal         1.0000 0.002570793 0.0003550156 0.0002296211
## 43:      share_marginal        10.0000 0.009635723 0.0145187529 0.0009438414
## 44:      share_marginal       501.1872 0.151793154 0.0616644586 0.0468247000
## 45:      share_marginal      1000.0000 0.221005286 0.3038339449 0.0996180777
## 46:      share_marginal      1584.8932 0.274789996 0.1852795683 0.1437288136
## 47:      share_marginal      1995.2623 0.313423983 0.4096066309 0.1815937941
## 48:      share_marginal      5011.8723 0.483204237 0.5589013596 0.3539786710
## 49:      share_marginal     10000.0000 0.636361666 0.6821675634 0.5255298651
## 50:      share_marginal     12589.2541 0.684959470 0.7198027938 0.5815244826
##      agreement      tau      frac_mean      frac_median      frac_lwr
##      <fctr>      <num>      <num>      <num>      <num>
##      frac_upr
##      <num>
## 1: 0.739917302
## 2: 0.726077699
## 3: 0.692334989
## 4: 0.498038131
## 5: 0.464055528
## 6: 0.442736738

```



```

## 7: 0.435423897
## 8: 0.375061973
## 9: 0.322508676
## 10: 0.301685672
## 11: 0.395709260
## 12: 0.379223625
## 13: 0.337519672
## 14: 0.222772775
## 15: 0.201660882
## 16: 0.193232524
## 17: 0.191497273
## 18: 0.157535944
## 19: 0.124938027
## 20: 0.114154685
## 21: 0.361087111
## 22: 0.359783380
## 23: 0.359166230
## 24: 0.280490828
## 25: 0.262394646
## 26: 0.249504214
## 27: 0.243926624
## 28: 0.217526029
## 29: 0.197570649
## 30: 0.187530987
## 31: 1.000000000
## 32: 0.999770379
## 33: 0.999056159
## 34: 0.953175300
## 35: 0.900381922
## 36: 0.856271186
## 37: 0.818406206
## 38: 0.646021329
## 39: 0.474470135
## 40: 0.418475517
## 41: 0.000000000
## 42: 0.007235529
## 43: 0.020326223
## 44: 0.289279348
## 45: 0.365651709
## 46: 0.420212766
## 47: 0.450327356
## 48: 0.611931509
## 49: 0.769973228
## 50: 0.823613445
##      frac_upr
##      <num>

```

```

# Plot uncertainty -----

agreement_cols <- c(frac_disagree = "black",
                    frac_major_disagree = "#B2182B",
                    frac_minor_disagree = "#2166AC" )

plot_uncertainty <- mat_sum[!agreement %in% c("share_existence", "share_marginal")] %>%
  ggplot(.,
    aes(x = tau, y = frac_median, colour = agreement, fill = agreement)) +
  geom_ribbon(aes(ymin = frac_lwr, ymax = frac_upr), alpha = 0.18, colour = NA) +
  geom_line(linewidth = 1.1) +
  scale_x_log10(labels = label_log(digits = 2)) +
  scale_colour_manual(values = agreement_cols,
    breaks = c("frac_disagree",
               "frac_major_disagree",
               "frac_minor_disagree"),
    labels = c("Total",
               "Major (\u2265 2 vs \u2265 2)",
               "Minor (one outlier)"),
    name = "Disagreement") +
  scale_fill_manual(values = agreement_cols,
    breaks = c("frac_disagree",
               "frac_major_disagree",
               "frac_minor_disagree"),
    labels = c("Total",
               "Major (\u2265 2 vs \u2265 2)",
               "Minor (one outlier)"),
    name = "Disagreement") +
  labs(x = expression(tau~"(ha)"), y = "Fraction of cells") +
  theme_AP() +
  theme(legend.position = c(0.65, 0.77))

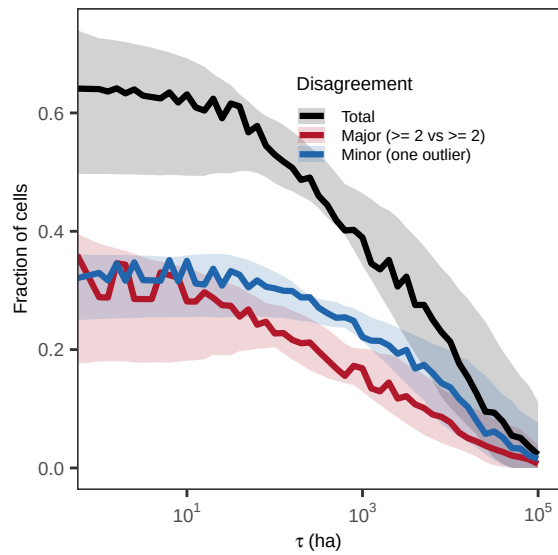
plot_uncertainty

```

```

## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.

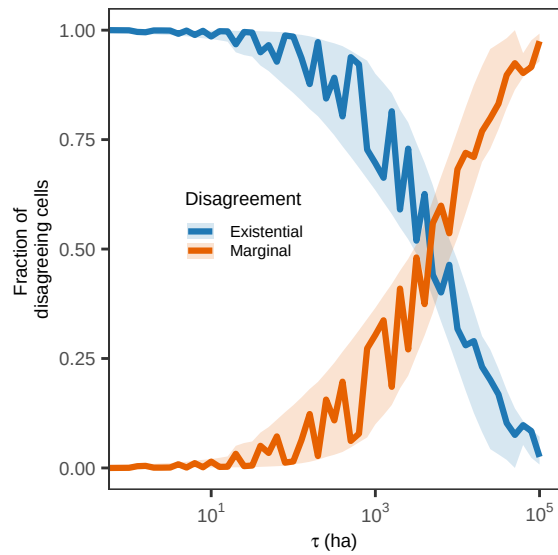
```



```
plot_agreement <- mat_sum[agreement %in% c("share_existence", "share_marginal")] %>%
  ggplot(., aes(x = tau, y = frac_median, colour = agreement, fill = agreement)) +
  geom_ribbon(aes(ymin = frac_lwr, ymax = frac_upr), alpha = 0.18, colour = NA) +
  geom_line(linewidth = 1.1) +
  scale_colour_manual(values = exist_cols,
                      labels = c(share_existence = "Existential",
                                share_marginal = "Marginal"),
                      name = "Disagreement") +
  scale_fill_manual(values = exist_cols,
                    labels = c(share_existence = "Existential",
                              share_marginal = "Marginal"),
                    name = "Disagreement") +
  scale_x_log10(labels = label_log(digits = 2)) +
  labs(x = expression(tau~"(ha)"), y = "Fraction of \ndisagreeing cells") +
  theme_AP() +
  theme(legend.position = c(0.3, 0.55))

plot_agreement
```

```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.
```



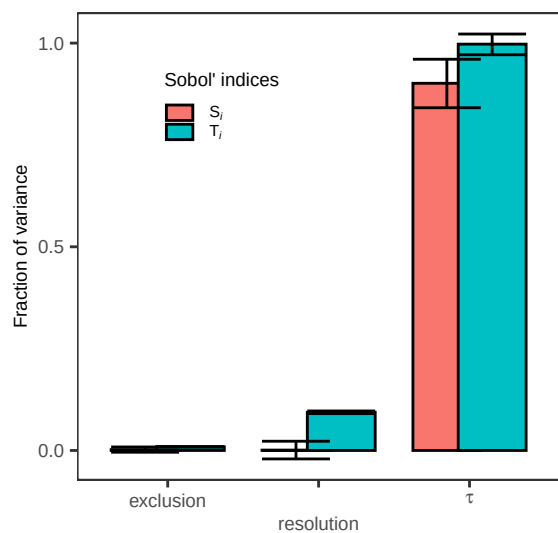
```
# SENSITIVITY ANALYSIS #####
```

```
# Sobol' indices -----
```

```
ind <- sobol_indices(matrices = matrices, N = N, params = params, Y
                     = mat[, frac_disagree], R = R, boot = TRUE)
```

```
plot_indices2 <- plot(ind) +
  theme_AP() +
  labs(x = "", y = "Fraction of variance") +
  scale_x_discrete(labels = c(exclusion = "exclusion",
                              resolution = "resolution",
                              tau = expression(tau)),
                  guide = guide_axis(n.dodge = 2)) +
  theme(legend.position = c(0.3, 0.8))
```

```
plot_indices2
```

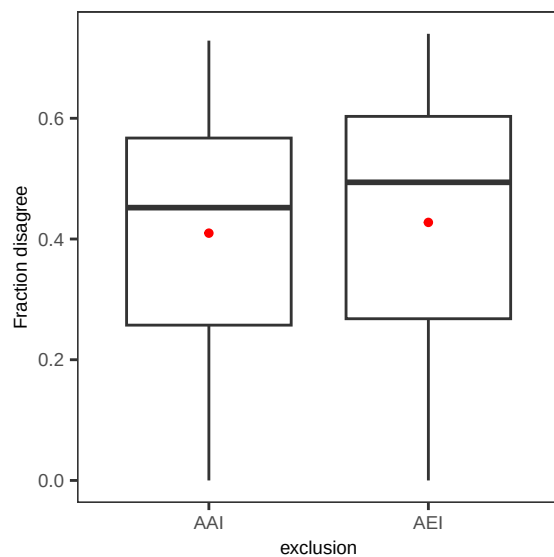


```
# SCATTERPLOTS OF OUTPUT AGAINST INPUT #####
```

```
label_map <- setNames(new_dataset_names, dataset_names)

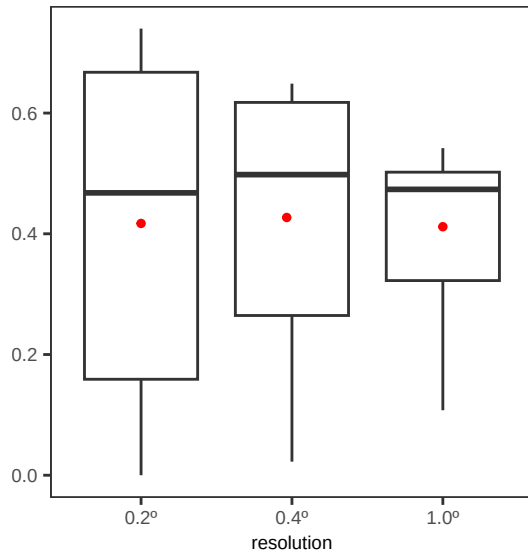
a2<- ggplot(mat[!exclusion == "all", c("exclusion", "frac_disagree")],
            aes(exclusion, frac_disagree)) +
  geom_boxplot() +
  stat_summary_bin(fun = "mean", geom = "point",
                  colour = "red", size = 1) +
  scale_x_discrete(labels = label_map) +
  scale_y_continuous(breaks = breaks_pretty(n = 3)) +
  theme_AP() +
  labs(x = "exclusion", y = "Fraction disagree")
```

a2



```
b2 <- ggplot(mat[1:N, c("resolution", "frac_disagree")],
            aes(resolution, frac_disagree)) +
  geom_boxplot() +
  scale_x_discrete(labels = c(`0.2deg` = "0.2°",
                              `0.4deg` = "0.4°",
                              `1deg` = "1.0°")) +
  scale_y_continuous(breaks = breaks_pretty(n = 3)) +
  stat_summary_bin(fun = "mean", geom = "point",
                  colour = "red", size = 1) +
  theme_AP() +
  labs(x = "resolution", y = "")
```

b2

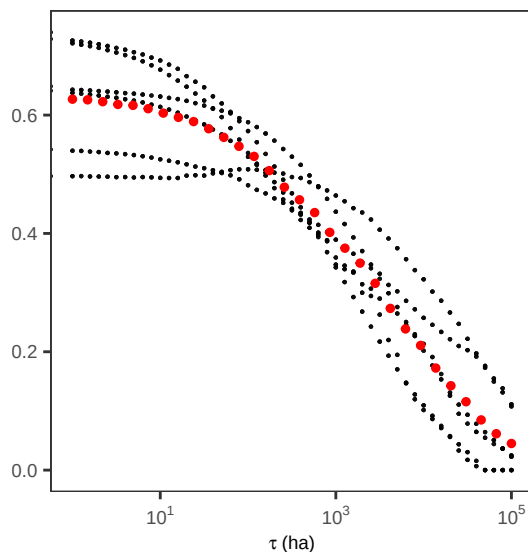


```
c2 <- ggplot(mat[1:N, c("tau", "frac_disagree")],
             aes(tau, frac_disagree)) +
  geom_point(size = 0.1, alpha = 0.1) +
  stat_summary_bin(fun = "mean", geom = "point",
                  colour = "red", size = 1) +
  scale_y_continuous(breaks = breaks_pretty(n = 3)) +
  theme_AP() +
  scale_x_log10(labels = label_log(digits = 2)) +
  labs(x = expression(tau~"(ha)"), y = "")
```

c2

```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.
```

```
## Warning: Removed 157 rows containing non-finite outside the scale range
## (`stat_summary_bin()`).
```



```
# MERGE UA/SA PLOTS #####
```

```
top <- plot_grid(plot_uncertainty, plot_agreement, plot_indices2,
  ncol = 3, labels = "auto", rel_widths = c(0.4, 0.3, 0.3))
```

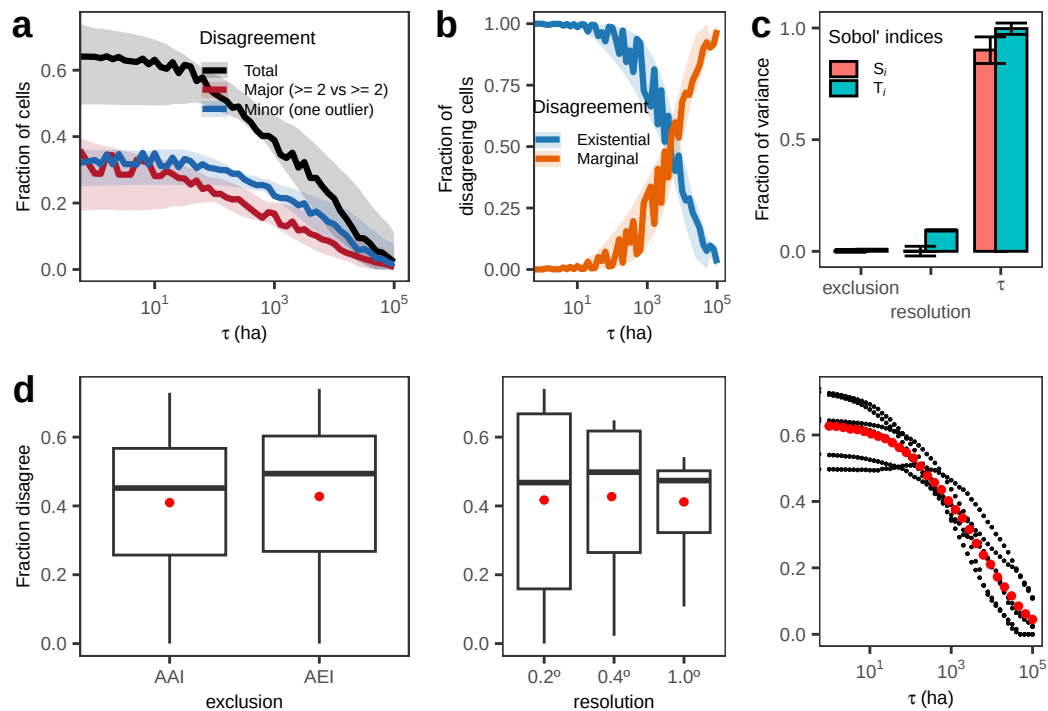
```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced .
## log-10 transformation introduced infinite values.
## log-10 transformation introduced infinite values.
## log-10 transformation introduced infinite values.
```

```
bottom <- plot_grid(a2, b2, c2, ncol = 3, rel_widths = c(0.4, 0.3, 0.3), labels = "d")
```

```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced .
## log-10 transformation introduced infinite values.

## Warning: Removed 157 rows containing non-finite outside the scale range
## (`stat_summary_bin()`).
```

```
plot_grid(top, bottom, ncol = 1)
```



3 The maximum threshold for existence disagreement: $\tau_{\max}$

```
# COMPUTE TAU MAX #####
#####

## Compute tau_max_results -----

tau_max_results <- rbindlist(
  parallel::mclapply(resolutions, function(res) {

    dt_res_long <- dt[resolution == res]

    # datasets present at this resolution
    ds_res <- sort(unique(dt_res_long$dataset))

    # resolution-specific detectability thresholds (in ha)-----

    zero_tol_ha <- switch(res,
      "0.2deg" = 500, # 1% of cell at 0.2°
      "0.4deg" = 2000, # 1% of cell at 0.4°
      "1deg" = 12000, # 1% of cell at 1°
      stop("Unknown resolution: ", res))

    rbindlist(
      lapply(dataset_vec, function(scn) {

        # which datasets to use -----

        if (scn == "all") {
          use_ds <- ds_res
        } else {
          use_ds <- setdiff(ds_res, scn)
        }

        # if only one dataset left, skip -----

        if (length(use_ds) < 2L) return(NULL)

        # reshape to wide -----

        dt_res_wide <- dcast(
          dt_res_long[dataset %in% use_ds],
          lon + lat + country + code + continent ~ dataset,
          value.var = "mha", fill = 0
        )
      })
    )
  })
)
```



```

    tau_max_dt <- tau_grid[,
      compute_tau_max_existential_fun(dt_res_wide, use_ds, tau_mha, tau_ha,
        zero_tol_ha = zero_tol_ha)
    ]

    tau_max_dt[, `:=`(resolution = res, scenario = scn)]

    tau_max_dt
  }},
  use.names = TRUE, fill = TRUE
)
}, mc.cores = ncores),
use.names = TRUE, fill = TRUE
)

# Add the countries -----

chunk_size <- 500000L
idx <- split(seq_len(nrow(tau_max_results)),
  ceiling(seq_len(nrow(tau_max_results)) / chunk_size))

tau_max_results[, country:= NA_character_]
for (i in idx) {
  tau_max_results[i, country:= lonlat_to_country(lon, lat, countries_sf)]
}

# Add the continents -----

country_to_continent(tau_max_results)

# Retrieve the round with all datasets and exclude the rest -----

tau_max_all <- tau_max_results[scenario == "all"]

# DEFINE PLOTS TAU MAX #####

# Bin the tau max results -----

tau_max_results[, det_thresh:= fcase(
  resolution == "0.2deg", 500,
  resolution == "0.4deg", 2000,
  resolution == "1deg", 12000)]

tau_max_results[, tau_bin:= fcase(
  is.na(tau_max_ha), "Agreement",
  tau_max_ha <= 2 * det_thresh, "Low disagreement",

```

```

    tau_max_ha <= 5 * det_thresh, "Moderate disagreement",
    tau_max_ha <= 10 * det_thresh, "High disagreement",
    default = "Extreme disagreement"
  )]

tau_max_results[, tau_bin:= factor(tau_bin,levels = c("Agreement",
                                                    "Low disagreement",
                                                    "Moderate disagreement",
                                                    "High disagreement",
                                                    "Extreme disagreement"),
                                ordered = TRUE)]

# Proportion of cells disagreeing at 1% threshold-----

tau_max_results[, .N, .(tau_bin, resolution)] %>%
  .[, fraction := N / sum(N), by = resolution] %>%
  .[!tau_bin == "Agreement", sum(fraction), resolution]

##      resolution      V1
##      <char>      <num>
## 1:      0.2deg 0.5508904
## 2:      0.4deg 0.4590500
## 3:      1deg 0.3799635

thresh_df <- unique(tau_max_results[!is.na(det_thresh), .(resolution, det_thresh)])

# ANALYSIS OF DATA -----

# Fraction of high and extreme ontological disagreement
# across resolutions in each continent -----

dt_all <- tau_max_results[scenario == "all"]
dt_all[, N_total := .N, by = .(resolution, continent)]
frac_by_rcb <- dt_all[, .(frac = .N / unique(N_total)),
                      .(resolution, continent, tau_bin)]

frac_by_rcb[, .(min = min(frac), max = max(frac)), .(continent, tau_bin)] %>%
  .[tau_bin %in% c("Extreme disagreement", "High disagreement")] %>%
  .[order(-min)] %>%
  .[, .(min = min(min), max = max(max)), .(continent)]

##      continent      min      max
##      <char>      <num>      <num>
## 1:      Asia 0.10417315 0.3561106
## 2:    Oceania 0.10869565 0.2256921
## 3:     Europe 0.08386916 0.1890844
## 4:  Americas 0.05267170 0.1500000
## 5:     Africa 0.03516442 0.1117143

```

```
# distribution of detectability limits
```

```
tau_max_results[, .N, .(tau_max_ha, resolution, scenario)]
```

```
##      tau_max_ha resolution scenario      N
##      <num>      <char>   <char> <int>
##  1:         NA    0.2deg     all 55250
##  2:   1584.893    0.2deg     all  5919
##  3:   1000.000    0.2deg     all  2969
##  4:   5011.872    0.2deg     all  7187
##  5:   3981.072    0.2deg     all  5088
##  ---
## 588: 31622.777      1deg    spam   291
## 589: 25118.864      1deg    spam   281
## 590:100000.000      1deg    spam   533
## 591: 63095.734      1deg    spam   259
## 592: 79432.823      1deg    spam   257
```

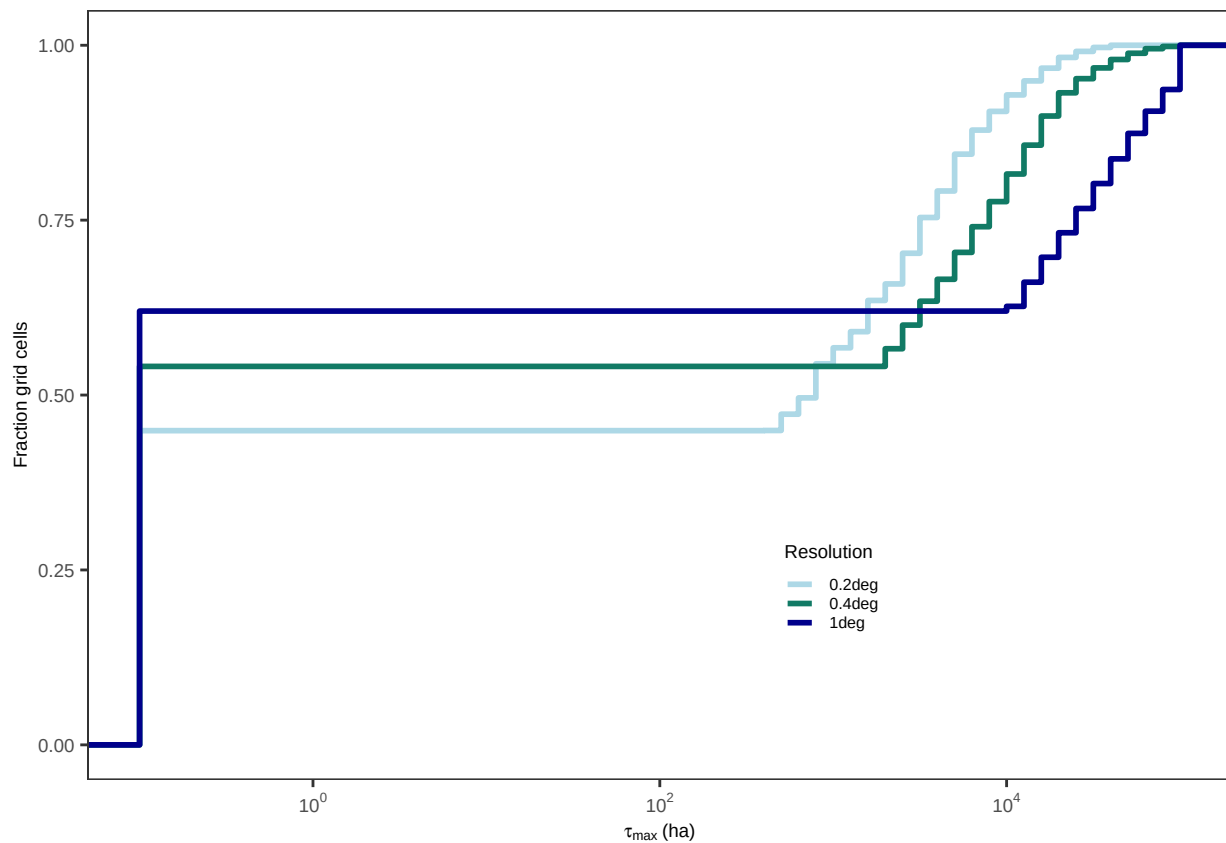
```
tmp <- copy(tau_max_results)
```

```
tmp[is.na(tau_max_ha), tau_max_ha:= 0.1]
```

```
plot_ecdf <- ggplot(tmp, aes(x = tau_max_ha, color = resolution)) +
  stat_ecdf(size = 1) +
  scale_x_log10(labels = label_log(digits = 2)) +
  scale_color_manual(values = res_palette) +
  labs(x = expression(tau[max]~"(ha)"), y = "Fraction grid cells",
       color = "Resolution") +
  theme_AP() +
  theme(legend.position = c(0.65, 0.25))
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
plot_ecdf
```



How much of the world is perfectly detectable vs fundamentally ambiguous?

```
tau_max_results[, .N, by = .(resolution, tau_bin)] %>%
  .[, frac := N / sum(N), by = resolution] %>%
  print()
```

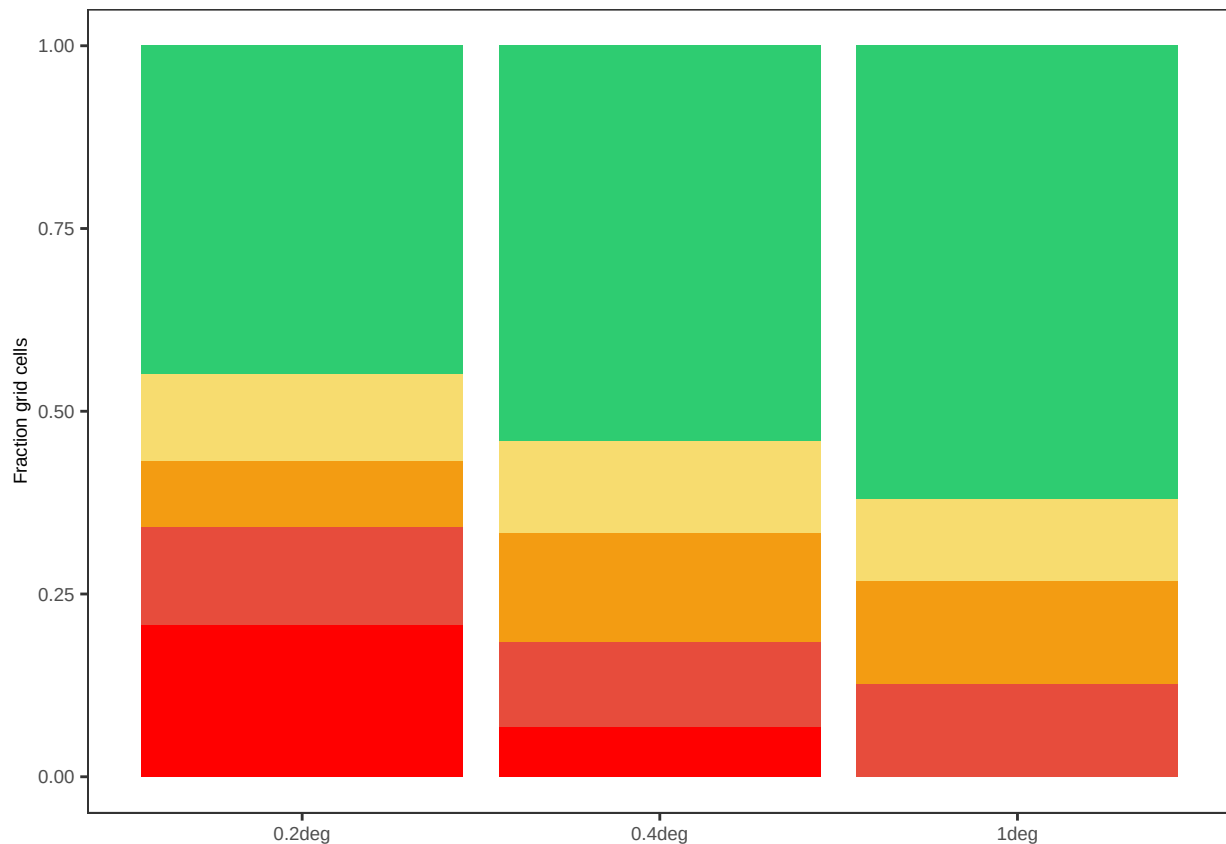
##	resolution	tau_bin	N	frac
##	<char>	<ord>	<int>	<num>
## 1:	0.2deg	Agreement	640389	0.44910962
## 2:	0.2deg	Moderate disagreement	130164	0.09128499
## 3:	0.2deg	Low disagreement	168896	0.11844803
## 4:	0.2deg	Extreme disagreement	297053	0.20832550
## 5:	0.2deg	High disagreement	189406	0.13283185
## 6:	0.4deg	Agreement	236578	0.54095002
## 7:	0.4deg	Moderate disagreement	65807	0.15047172
## 8:	0.4deg	Low disagreement	54457	0.12451925
## 9:	0.4deg	High disagreement	50769	0.11608641
## 10:	0.4deg	Extreme disagreement	29727	0.06797260
## 11:	1deg	Agreement	55027	0.62003651
## 12:	1deg	Low disagreement	9935	0.11194618
## 13:	1deg	Moderate disagreement	12605	0.14203137
## 14:	1deg	High disagreement	11181	0.12598594

```

plot_stacked <- tau_max_results[, .N, by = .(resolution, tau_bin)] %>%
  .[, frac := N / sum(N), by = resolution] |>
  ggplot(aes(resolution, frac, fill = tau_bin)) +
  geom_col() +
  scale_y_continuous() +
  scale_fill_manual(values = tau_palette, name = expression(tau[max])) +
  labs(y = "Fraction grid cells", x = NULL, fill = expression(tau[max])) +
  theme_AP() +
  theme(legend.position = "none")

```

plot_stacked



```

# Boxplots of tau max per resolution -----
plot_box <- ggplot(tau_max_results[!is.na(tau_max_ha) & tau_max_ha > 0],
  aes(resolution, tau_max_ha, fill = resolution)) +
  geom_boxplot(outlier.shape = NA, alpha = 0.85) +
  geom_hline(data = thresh_df, aes(yintercept = det_thresh, color = resolution),
    linetype = "dashed", linewidth = 0.7, show.legend = FALSE) +
  scale_y_log10(labels = scales::label_log(digits = 2),
    name = expression(tau[max] ~ "(ha)")) +
  scale_fill_manual(values = res_palette, guide = "none") +
  scale_color_manual(values = res_palette, guide = "none") +
  labs(

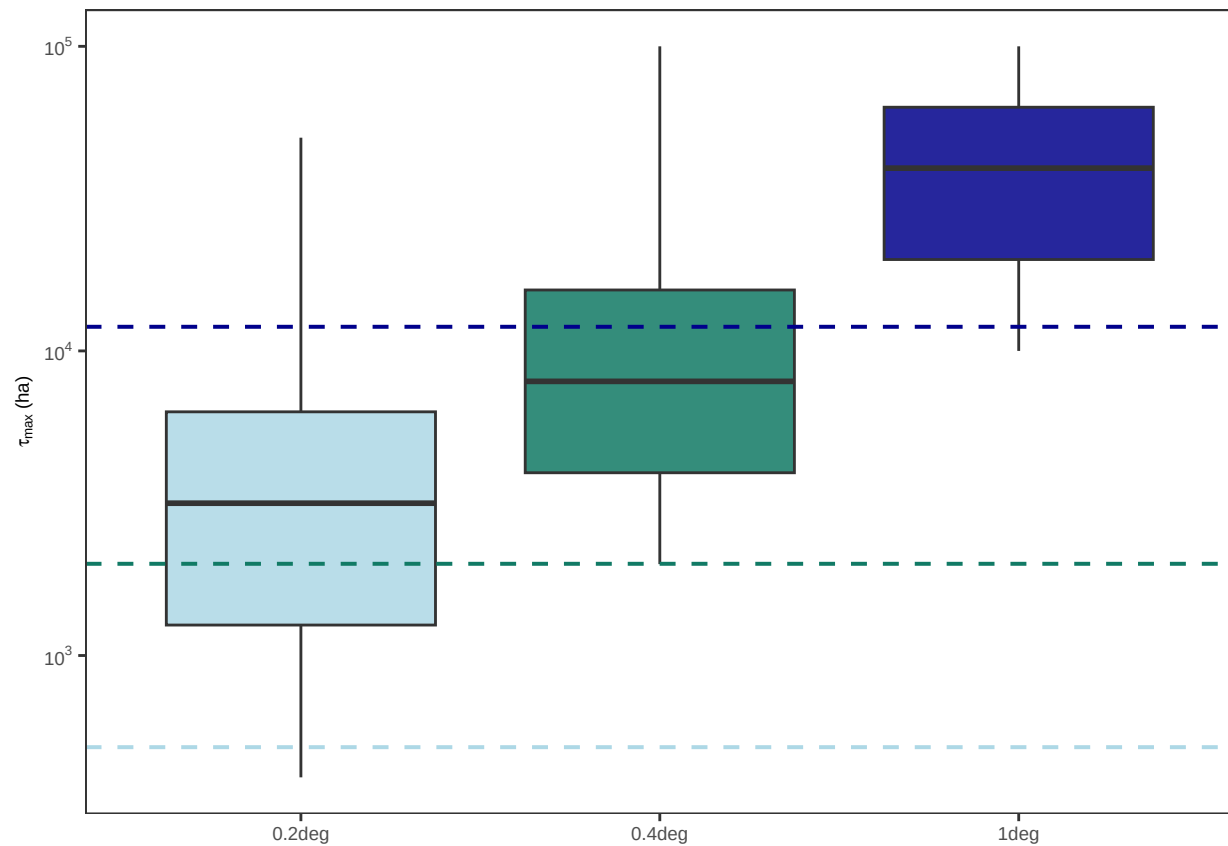
```

```

x = NULL,
) +
theme_AP() +
theme()

```

plot_box



```

# MERGE TAU MAX PLOTS #####

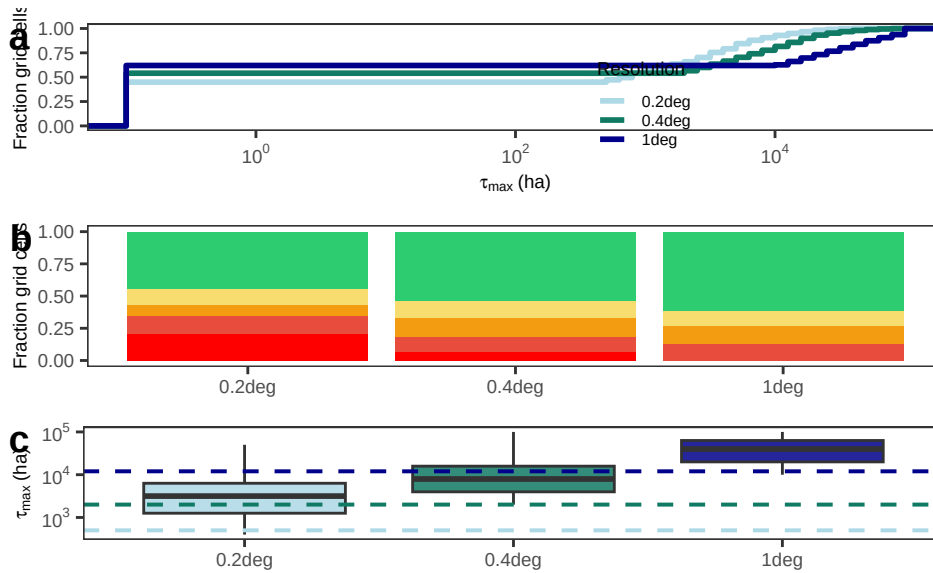
```

```

top <- plot_grid(plot_ecdf, plot_stacked, plot_box, ncol = 1,
  rel_heights = c(0.35, 0.35, 0.3),
  labels = "auto")

```

top

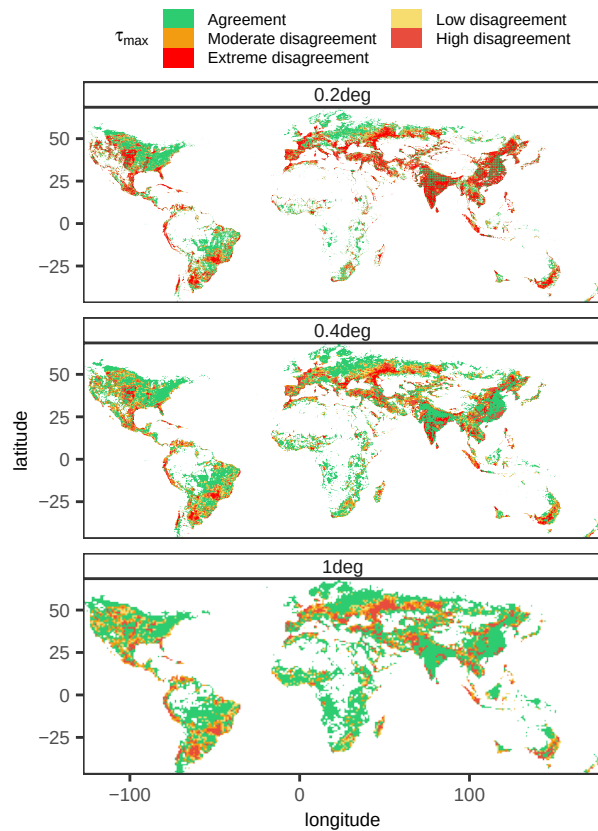


The map

```
plot_raster <- ggplot(tau_max_results, aes(lon, lat, fill = tau_bin)) +
  geom_raster() +
  coord_equal(expand = FALSE) +
  facet_wrap(~ resolution, ncol = 1) +
  scale_fill_manual(values = tau_palette, name = expression(tau[max])) +
  scale_x_continuous(expand = c(0, 0)) +
  scale_y_continuous(expand = c(0, 0)) +
  theme_AP() +
  theme(panel.grid = element_blank(),
        strip.text = element_text(size = 11),
        legend.position = "top") +
  labs(x = "longitude", y = "latitude") +
  guides(fill = guide_legend(nrow = 3, byrow = TRUE))
```

plot_raster

```
## Warning: Raster pixels are placed at uneven horizontal intervals and will be shifted
## i Consider using `geom_tile()` instead.
## Raster pixels are placed at uneven horizontal intervals and will be shifted
## i Consider using `geom_tile()` instead.
```

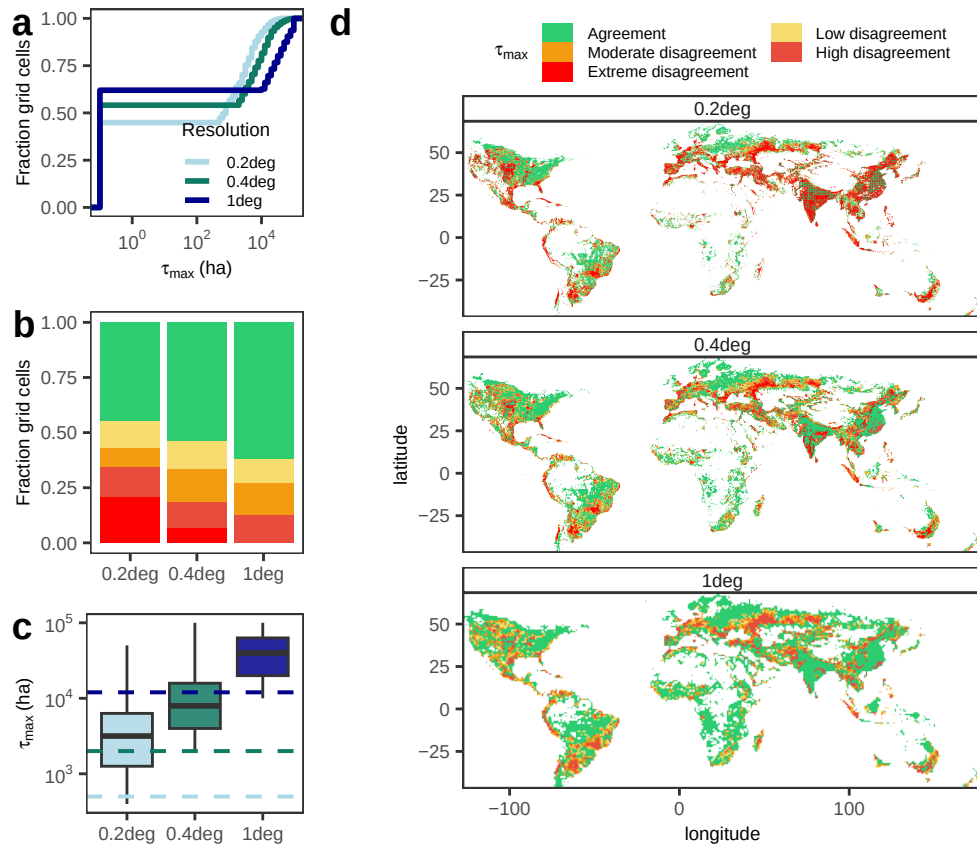


3.1 Composite figure

```
# MERGE PLOTS #####

plot_grid(top, plot_raster, rel_widths = c(0.3, 0.7), ncol = 2, labels = c("", "d"))

## Warning: Raster pixels are placed at uneven horizontal intervals and will be shifted
## i Consider using `geom_tile()` instead.
## Raster pixels are placed at uneven horizontal intervals and will be shifted
## i Consider using `geom_tile()` instead.
```

3.1.1 Which datasets drive deep disagreement?

```
# WHICH DATASETS DRIVE DEEP DISAGREEMENT? -----

# Check leave one-out scenarios -----

loo_stats_full <- tau_max_results[, .(
  median_tau = median(tau_max_ha, na.rm = TRUE),
  p75_tau = quantile(tau_max_ha, 0.75, na.rm = TRUE),
  p90_tau = quantile(tau_max_ha, 0.90, na.rm = TRUE),
  share_gt_1000 = mean(tau_max_ha >= 1000, na.rm = TRUE)),
  scenario] %>%
  .[order(-p90_tau)]

loo_stats_full
```

##	scenario	median_tau	p75_tau	p90_tau	share_gt_1000
##	<char>	<num>	<num>	<num>	<num>
## 1:	all	3981.072	10000.000	19952.62	0.8705251
## 2:	gaez_v4	3981.072	10000.000	19952.62	0.8700602
## 3:	giam	3981.072	7943.282	19952.62	0.8558969
## 4:	gmia	3981.072	10000.000	19952.62	0.8704727
## 5:	gripc	3981.072	7943.282	19952.62	0.8650565

```

## 6:      meier    3981.072 10000.000 19952.62      0.8693391
## 7: mirca2000    3981.072 10000.000 19952.62      0.8721156
## 8:  mirca_os    3981.072 10000.000 19952.62      0.8712516
## 9:   nagaraj    3981.072 10000.000 19952.62      0.8587307
## 10:      spam    3981.072 10000.000 19952.62      0.8702693
## 11:      luh2    3162.278  7943.282 15848.93      0.8537190

# In which regions is each dataset structurally responsible for
# deep existential contradictions? -----

# Compute spatial delta -----

tau_all <- tau_max_results[scenario == "all",.(lon, lat, resolution,
                                              tau_all = tau_max_ha)]

# Keep only datasets of interest--

tau_loo <- tau_max_results[scenario %in% c("giam", "luh2", "nagaraj", "gripc"),
                          .(lon, lat, country, continent, resolution, scenario,
                            tau_loo = tau_max_ha)]

# Merge -----

tau_delta <- merge(tau_loo, tau_all, by = c("lon", "lat", "resolution"),
                  all.x = TRUE)

# Compute difference -----

tau_delta[, delta:= tau_all - tau_loo]

# Let us define "strong amplification" of differences -----
# Differences larger than 1000 ha.

tau_delta[, strong:= delta >= 1000]

# Compute overlap -----

influence_matrix <- dcast(tau_delta[resolution == "0.2deg"],
                        lon + lat ~ scenario, value.var = "strong",
                        fill = FALSE)

influence_matrix[, n_datasets:= rowSums(.SD), .SDcols = c("giam", "luh2", "nagaraj", "gripc")]

table(influence_matrix$n_datasets)

##
##      0      1
## 27483 18737

```

```

# 0: deep disagreement in the cell not driven by these three.
# 1: dataset-specific amplification.
# 2: partial overlap.
# 3: same hotspot amplified by all three.

# Where does amplification concentrate? -----

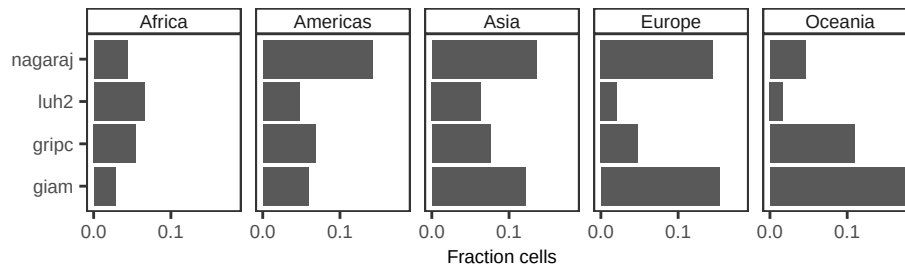
regional_amp <- tau_delta[resolution == "0.2deg",
                        .(median_delta = median(delta, na.rm = TRUE),
                          share_strong = mean(delta >= 1000, na.rm = TRUE)),
                        .(continent, scenario)] %>%
  .[order(-median_delta)]

regional_amp

##      continent scenario median_delta share_strong
##      <char>    <char>         <num>         <num>
## 1:  Americas      giam             0    0.05904022
## 2:  Americas      gripc             0    0.06811812
## 3:  Americas      luh2             0    0.04731494
## 4:  Americas      nagaraj           0    0.14193931
## 5:   Europe      giam             0    0.15351445
## 6:   Europe      gripc             0    0.04808959
## 7:   Europe      luh2             0    0.02085566
## 8:   Europe      nagaraj           0    0.14524721
## 9:   Africa      giam             0    0.02829181
## 10:  Africa      gripc             0    0.05506249
## 11:  Africa      luh2             0    0.06565308
## 12:  Africa      nagaraj           0    0.04397835
## 13:   Asia      giam             0    0.12130263
## 14:   Asia      gripc             0    0.07585118
## 15:   Asia      luh2             0    0.06372931
## 16:   Asia      nagaraj           0    0.13518991
## 17: Oceania      giam             0    0.18177718
## 18: Oceania      gripc             0    0.10928320
## 19: Oceania      luh2             0    0.01693667
## 20: Oceania      nagaraj           0    0.04547121
##      continent scenario median_delta share_strong
##      <char>    <char>         <num>         <num>

ggplot(regional_amp, aes(scenario, share_strong)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  facet_wrap(~continent, ncol = 5) +
  scale_y_continuous(breaks = breaks_pretty(n = 2)) +
  theme_AP() +
  labs(x = "", y = "Fraction cells")

```



```
# Which dataset drives the 22,615 cells? -----
```

```
influence_matrix[, dominant := fifelse(giam, "GIAM",
                                       fifelse(luh2, "LUH2",
                                               fifelse(nagaraj, "Nagaraj",
                                                       fifelse(gripc, "GRIPC",
                                                           NA_character_)))))]
```

```
table(influence_matrix$dominant)
```

```
##
##   GIAM   GRIPC   LUH2  Nagaraj
##  6527   3361   2754   8259
```

3.2 How many grid cells disagree?

```
# DETECTABILITY OF IRRIGATED AREA ACROSS MAPS #####
#####
```

```
# UNWEIGHTED: ALL CELLS COUNT EQUALLY #####
# QUESTION ANSWERED: HOW MANY GRID CELLS DISAGREE? -----
```

```
# Make dataset wide and retrieve only the 0.2deg resolution one -----
```

```
dt_wide <- dt[resolution == "0.2deg"] %>%
  dcast(., lon + lat + country + code + continent ~ dataset, value.var = "mha")
```

```
# Retrieve 0.2deg resolution from tau max dataset and remove unwanted cols -----
```

```
tau_max_results_clean <- tau_max_results[resolution == "0.2deg" & scenario == "all"]
tau_max_results_clean[, resolution:= NULL]
tau_max_results_clean[, scenario:= NULL]
```

```
# Merge -----
```

```
dt2 <- merge(dt_wide, tau_max_results_clean[, .(lon, lat, tau_max_mha)],
             by = c("lon", "lat"), all.x = TRUE)
```

```
setnames(dt2, dataset_names, new_dataset_names)
```

```
# Convert to ha for easier interpretation -----

dt2[, tau_max_ha:= tau_max_mha * 1e6]

# Which percentage of the world is in disagreement on whether there is
# or there is not irrigation? -----

frac_cells_nonid <- dt2[, mean(!is.na(tau_max_mha))]
frac_cells_nonid
```

```
## [1] 0.5737804
```

```
summary(dt2$tau_max_ha)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NA's
##    398.1  1584.9  3162.3  5575.2  6309.6 50118.7   55250
```

3.3 How much irrigated land lies in disagreement?

```
# WEIGHTED: CELLS WITH LARGER IRRIGATION COUNT MORE #####
# QUESTION: HOW MUCH IRRIGATED LAND LIES IN DISAGREEMENT? -----
```

```
# Keep only cells with existential disagreement -----
# (tau_max_ha non-NA)
```

```
dt2_disagree <- dt2[!is.na(tau_max_ha)]
```

```
# Sanity check: no NA tau_max_ha left
summary(dt2_disagree$tau_max_ha)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    398.1  1584.9  3162.3  5575.2  6309.6 50118.7
```

```
# should show NA's = 0
```

```
# Compute total and non-identifiable area per dataset -----
```

```
area_nonid <- rbindlist(lapply(new_dataset_names, function(d) {
```

```
  # cells where this dataset has irrigation AND there is disagreement
```

```
  dt_d <- dt2[get(d) > 0, .(tau_max_ha, area = get(d))]
```

```
  # total irrigated area (for this dataset) in cells with disagreement
```

```
  total_area <- dt_d[, sum(area, na.rm = TRUE)]
```

```
  # irrigated area in cells where tau_max_ha is not NA; that is, in cells
  # that disagree
```

```

nonid_area <- dt_d[!is.na(tau_max_ha), sum(area, na.rm = TRUE)]

data.table(dataset = d, total_area_mha = total_area,
            nonid_area_mha = nonid_area, share_nonid = nonid_area / total_area)
)))

```

```
area_nonid
```

```

##      dataset total_area_mha nonid_area_mha share_nonid
##      <char>      <num>      <num>      <num>
##  1:  GAEZ+2015      280.2879      154.70438    0.5519481
##  2:    GIAM        335.1898      204.36911    0.6097116
##  3:    GMIA        245.8296      131.80237    0.5361534
##  4:   GRIPC        305.9983      171.29686    0.5597967
##  5:    LUH2        262.3541       66.70482    0.2542549
##  6:    Meier        354.6320      209.34845    0.5903259
##  7: MIRCA 2000      210.5253      108.83424    0.5169652
##  8:  MIRCA OS       265.5109      140.13459    0.5277921
##  9:   Nagaraj       285.7831      186.01552    0.6508975
## 10:   SPAM2010      219.5544      112.55164    0.5126368

```

```
# How much tolerance do we need to make datasets agree?-----
```

```

tau_diag <- rbindlist(lapply(new_dataset_names, function(d) {

  dt_d <- dt2_disagree[get(d) > 0, .(tau_max_ha, area = get(d))]

  if (nrow(dt_d) == 0) {
    return(data.table(dataset = d,
                      tau_50 = NA_real_,
                      tau_80 = NA_real_,
                      tau_90 = NA_real_,
                      area_share_1000 = NA_real_ ))
  }

```

```
# area-weighted empirical CDF of tau_max_ha (for this dataset in disagreeing cells)
```

```

tmp <- dt_d[, .(area = sum(area)), by = tau_max_ha][order(tau_max_ha)]
tmp[, cum_area := cumsum(area)]
tmp[, cum_share := cum_area / sum(area)]

```

```

# tau_50: minimum tau required to make 50% of irrigated area identifiable
# tau_80: tau required for 80%
# tau_90: tau required for 90%

```

```

tau_50 <- tmp[cum_share >= 0.5][1, tau_max_ha]
tau_80 <- tmp[cum_share >= 0.8][1, tau_max_ha]

```

```

tau_90 <- tmp[cum_share >= 0.9][1, tau_max_ha]

# share of area that is identifiable under tau_threshold_ha (e.g. 1000 ha)

area_share_1000 <- tmp[tau_max_ha <= 1000, fifelse(.N == 0, 0,
                                                    max(cum_share, na.rm = TRUE))]]

data.table(dataset = d,
            tau_50 = tau_50,
            tau_80 = tau_80,
            tau_90 = tau_90,
            area_share_1000 = area_share_1000)
)))

tau_diag

```

##	dataset	tau_50	tau_80	tau_90	area_share_1000
##	<char>	<num>	<num>	<num>	<num>
## 1:	GAEZ+2015	12589.254	25118.86	31622.78	0.024079566
## 2:	GIAM	15848.932	25118.86	31622.78	0.015560853
## 3:	GMIA	12589.254	25118.86	31622.78	0.020758906
## 4:	GRIPC	15848.932	25118.86	31622.78	0.017073267
## 5:	LUH2	10000.000	19952.62	25118.86	0.007732642
## 6:	Meier	12589.254	25118.86	31622.78	0.021551156
## 7:	MIRCA 2000	12589.254	25118.86	31622.78	0.025574408
## 8:	MIRCA OS	12589.254	25118.86	31622.78	0.022653569
## 9:	Nagaraj	7943.282	19952.62	25118.86	0.018838929
## 10:	SPAM2010	15848.932	25118.86	31622.78	0.019254211

4 Consequences for countries

4.1 Identification of irrigation hotspots

```

# DOES THE DIFFERENT DATASETS IDENTIFY THE SAME TOP IRRIGATION HOTSPOTS? #####

# Identify top 10% hotspots per dataset -----

hotspots <- lapply(new_dataset_names, function(d) {

  tmp <- copy(dt2)
  tmp[, value:= get(d)]

  # rank cells-----

  tmp <- tmp[order(-value)]
  n_hot <- ceiling(0.10 * nrow(tmp))

```

```

tmp[, hotspot:= FALSE]
tmp[1:n_hot, hotspot:= TRUE]

tmp[, .(lon, lat, hotspot, dataset = d)]
})

# name the list elements by dataset -----

names(hotspots) <- sapply(hotspots, function(x) unique(x$dataset))

# extract hotspot cell IDs per dataset -----

hotspot_sets <- lapply(hotspots, function(dt) {
  dt[hotspot == TRUE, .(lon, lat)]})

# Country rankings per dataset -----

country_totals <- rbindlist(lapply(new_dataset_names, function(d) {
  dt2[, .(irrigated_area = sum(get(d), na.rm = TRUE)), country] %>%
    .[, dataset:= d]}))

jaccard_cells <- function(dt1, dt2) {
  set1 <- paste(dt1$lon, dt1$lat)
  set2 <- paste(dt2$lon, dt2$lat)

  length(intersect(set1, set2)) / length(union(set1, set2))
}

# Pairwise jacqard similarity -----

datasets <- names(hotspot_sets)

jaccard_results <- rbindlist(combn(datasets, 2, simplify = FALSE,
  FUN = function(pair) {
    d1 <- pair[1]
    d2 <- pair[2]

    data.table(dataset_1 = d1, dataset_2 = d2,
      jaccard = jaccard_cells(hotspot_sets[[d1]], hotspot_sets[[d2]]))
  })
)

jaccard_results[order(jaccard)]

##      dataset_1 dataset_2  jaccard
##      <char>    <char>    <num>
##  1:      GIAM      LUH2 0.3018328

```


## 2:	GRIPC	LUH2	0.3175789
## 3:	LUH2	Nagaraj	0.3527785
## 4:	LUH2	Meier	0.3611592
## 5:	GAEZ+2015	LUH2	0.3622320
## 6:	LUH2	SPAM2010	0.3861947
## 7:	GMIA	LUH2	0.3943957
## 8:	LUH2	MIRCA OS	0.3944707
## 9:	GIAM	Nagaraj	0.4022392
## 10:	LUH2	MIRCA 2000	0.4062703
## 11:	GAEZ+2015	GIAM	0.4131691
## 12:	GRIPC	Nagaraj	0.4160249
## 13:	GIAM	SPAM2010	0.4306368
## 14:	GIAM	GMIA	0.4361844
## 15:	GIAM	GRIPC	0.4400133
## 16:	GRIPC	SPAM2010	0.4416148
## 17:	GAEZ+2015	GRIPC	0.4525183
## 18:	GIAM	MIRCA 2000	0.4547189
## 19:	GIAM	Meier	0.4570899
## 20:	GRIPC	Meier	0.4571718
## 21:	GMIA	GRIPC	0.4574175
## 22:	GIAM	MIRCA OS	0.4621024
## 23:	GRIPC	MIRCA OS	0.4734868
## 24:	GRIPC	MIRCA 2000	0.4804705
## 25:	Nagaraj	SPAM2010	0.4954144
## 26:	GAEZ+2015	Nagaraj	0.5058373
## 27:	MIRCA OS	Nagaraj	0.5133084
## 28:	MIRCA 2000	Nagaraj	0.5171163
## 29:	Meier	Nagaraj	0.5343552
## 30:	GMIA	Nagaraj	0.5484680
## 31:	GAEZ+2015	SPAM2010	0.5702259
## 32:	GAEZ+2015	MIRCA 2000	0.5815287
## 33:	Meier	SPAM2010	0.5853972
## 34:	Meier	MIRCA 2000	0.5887003
## 35:	Meier	MIRCA OS	0.5982000
## 36:	GAEZ+2015	Meier	0.6025467
## 37:	GAEZ+2015	MIRCA OS	0.6276996
## 38:	GAEZ+2015	GMIA	0.6362259
## 39:	MIRCA 2000	SPAM2010	0.6371558
## 40:	MIRCA OS	SPAM2010	0.6599014
## 41:	MIRCA 2000	MIRCA OS	0.6619231
## 42:	GMIA	Meier	0.6752391
## 43:	GMIA	MIRCA 2000	0.6842721
## 44:	GMIA	SPAM2010	0.6969499
## 45:	GMIA	MIRCA OS	0.7250649
##	dataset_1	dataset_2	jaccard
##	<char>	<char>	<num>

```
jaccard_results[, .(min_jaccard = min(jaccard),
                      median_jaccard = median(jaccard),
                      mean_jaccard = mean(jaccard),
                      max_jaccard = max(jaccard))]
```

```
##      min_jaccard median_jaccard mean_jaccard max_jaccard
##           <num>          <num>         <num>         <num>
## 1:    0.3018328      0.4734868      0.5021623      0.7250649
```

4.2 National rankings

```
# NATIONAL RANKINGS USING ALL CELLS #####
```

```
# Country totals & ranks .-----
```

```
country_totals_all <- compute_country_totals(dt2, new_dataset_names)
```

```
# Pairwise rank correlations -----
```

```
rank_corr_all <- compute_rank_correlations(country_totals_all)
rank_corr_all[order(-spearman)]
```

```
##      dataset_1 dataset_2 kendall spearman
##           <char>      <char>    <num>    <num>
## 1: MIRCA 2000   MIRCA OS 0.8869269 0.9792934
## 2:      GMIA MIRCA 2000 0.8899580 0.9764568
## 3:      GMIA   MIRCA OS 0.8782725 0.9726811
## 4:      Meier MIRCA 2000 0.8613228 0.9701687
## 5:      GMIA      Meier 0.8605534 0.9678578
## 6:      GMIA      LUH2 0.8796674 0.9675411
## 7:      Meier   MIRCA OS 0.8402259 0.9608881
## 8:      GMIA   SPAM2010 0.8229857 0.9543192
## 9:      LUH2 MIRCA 2000 0.8326323 0.9521371
## 10:     LUH2   MIRCA OS 0.8306333 0.9516782
## 11:   MIRCA OS   SPAM2010 0.8140963 0.9469802
## 12:     LUH2   SPAM2010 0.8126740 0.9454728
## 13: MIRCA 2000   SPAM2010 0.8033644 0.9443709
## 14:  GAEZ+2015      Meier 0.8019412 0.9432197
## 15:  GAEZ+2015 MIRCA 2000 0.8032595 0.9423526
## 16:      Meier   SPAM2010 0.7935570 0.9393322
## 17:     LUH2      Meier 0.8078651 0.9361524
## 18:  GAEZ+2015   MIRCA OS 0.7958000 0.9351266
## 19:  GAEZ+2015      GMIA 0.7845397 0.9275645
## 20:      Meier   Nagaraj 0.7802484 0.9145597
## 21: MIRCA 2000   Nagaraj 0.7657801 0.9093070
## 22:      GMIA   Nagaraj 0.7570546 0.9069830
## 23:      GIAM      Meier 0.7434964 0.9066388
## 24:  GAEZ+2015     LUH2 0.7415172 0.9043953
```

```
## 25:  MIRCA OS      Nagaraj 0.7454631 0.9032900
## 26:  GAEZ+2015    SPAM2010 0.7309457 0.9002531
## 27:      GIAM MIRCA 2000 0.7256531 0.8932012
## 28:  Nagaraj     SPAM2010 0.7171736 0.8807849
## 29:      GIAM     Nagaraj 0.7057269 0.8777524
## 30:  GAEZ+2015    Nagaraj 0.7107326 0.8765867
## 31:      GIAM      GMIA 0.7023198 0.8763834
## 32:      GMIA      GRIPC 0.6971987 0.8749677
## 33:      GRIPC MIRCA OS 0.6966780 0.8748337
## 34:      GRIPC MIRCA 2000 0.6958898 0.8692777
## 35:      GIAM MIRCA OS 0.6917915 0.8674594
## 36:      GRIPC     Meier 0.6927134 0.8673210
## 37:      GIAM     SPAM2010 0.6834225 0.8653044
## 38:      GRIPC     SPAM2010 0.6796605 0.8649194
## 39:      LUH2     Nagaraj 0.7021109 0.8635624
## 40:  GAEZ+2015      GIAM 0.6798635 0.8568604
## 41:      GRIPC      LUH2 0.6783125 0.8541425
## 42:      GIAM      LUH2 0.6694764 0.8450648
## 43:      GRIPC     Nagaraj 0.6514905 0.8300450
## 44:  GAEZ+2015      GRIPC 0.6442363 0.8237471
## 45:      GIAM      GRIPC 0.6289032 0.8063916
##      dataset_1 dataset_2 kendall spearman
##      <char>      <char>      <num>      <num>
```

```
# Pairwise top-N overlap (e.g. top 20 irrigators) -----
```

```
topN_overlap_all <- compute_topN_overlap(country_totals_all,
                                          dataset_names = new_dataset_names,
                                          N = 20)
topN_overlap_all[order(-jaccard_topN)]
```

```
##      dataset_1 dataset_2      N intersection_N jaccard_topN
##      <char>      <char> <num>      <int>      <num>
##  1:      GMIA      LUH2     20          19    0.9047619
##  2:      GMIA MIRCA 2000     20          19    0.9047619
##  3:      LUH2     Meier     20          19    0.9047619
##  4:      GIAM MIRCA 2000     20          18    0.8181818
##  5:      GMIA     Meier     20          18    0.8181818
##  6:      LUH2 MIRCA 2000     20          18    0.8181818
##  7:     Meier MIRCA 2000     20          18    0.8181818
##  8:     Meier  MIRCA OS     20          18    0.8181818
##  9:      GIAM      GMIA     20          17    0.7391304
## 10:      GIAM      LUH2     20          17    0.7391304
## 11:      GIAM     Meier     20          17    0.7391304
## 12:      GMIA MIRCA OS     20          17    0.7391304
## 13:      LUH2 MIRCA OS     20          17    0.7391304
## 14:     Meier  Nagaraj     20          17    0.7391304
## 15:     Meier  SPAM2010     20          17    0.7391304
```

```

## 16: MIRCA 2000    MIRCA OS    20          17    0.7391304
## 17: MIRCA 2000    Nagaraj    20          17    0.7391304
## 18:   MIRCA OS    SPAM2010   20          17    0.7391304
## 19:  GAEZ+2015    GRIPC      20          16    0.6666667
## 20:      GIAM      GRIPC      20          16    0.6666667
## 21:      GIAM    MIRCA OS    20          16    0.6666667
## 22:      GIAM    Nagaraj    20          16    0.6666667
## 23:      GIAM    SPAM2010   20          16    0.6666667
## 24:      GMIA    Nagaraj    20          16    0.6666667
## 25:      GRIPC MIRCA 2000   20          16    0.6666667
## 26:      LUH2    Nagaraj    20          16    0.6666667
## 27:      LUH2    SPAM2010   20          16    0.6666667
## 28: MIRCA 2000    SPAM2010   20          16    0.6666667
## 29:   MIRCA OS    Nagaraj    20          16    0.6666667
## 30:   Nagaraj    SPAM2010   20          16    0.6666667
## 31:  GAEZ+2015 MIRCA 2000   20          15    0.6000000
## 32:      GMIA      GRIPC      20          15    0.6000000
## 33:      GMIA    SPAM2010   20          15    0.6000000
## 34:      GRIPC      LUH2      20          15    0.6000000
## 35:      GRIPC      Meier     20          15    0.6000000
## 36:      GRIPC    MIRCA OS    20          15    0.6000000
## 37:  GAEZ+2015      GIAM      20          14    0.5384615
## 38:  GAEZ+2015      GMIA      20          14    0.5384615
## 39:  GAEZ+2015    MIRCA OS    20          14    0.5384615
## 40:  GAEZ+2015    Nagaraj    20          14    0.5384615
## 41:      GRIPC    SPAM2010   20          14    0.5384615
## 42:  GAEZ+2015      LUH2      20          13    0.4814815
## 43:  GAEZ+2015      Meier     20          13    0.4814815
## 44:      GRIPC    Nagaraj    20          13    0.4814815
## 45:  GAEZ+2015    SPAM2010   20          12    0.4285714
##      dataset_1  dataset_2      N intersection_N jaccard_topN
##      <char>      <char> <num>          <int>          <num>

```

```

# NATIONAL RANKINGS USING ONLY "IDENTIFIABLE" CELLS #####

```

```

# Only areas that agree -----

```

```

dt_ident <- dt2[is.na(tau_max_ha)]

```

```

# Country totals & ranks on identifiable cells only -----

```

```

country_totals_ident <- compute_country_totals(dt_ident, new_dataset_names)

```

```

# Pairwise rank correlations (identifiable-only) -----

```

```

rank_corr_ident <- compute_rank_correlations(country_totals_ident)
rank_corr_ident[order(-spearman)]

```

##	dataset_1	dataset_2	kendall	spearman
##	<char>	<char>	<num>	<num>
## 1:	GMIA	MIRCA OS	0.8835329	0.9783609
## 2:	GMIA	MIRCA 2000	0.8814086	0.9765933
## 3:	GMIA	SPAM2010	0.8545396	0.9657292
## 4:	MIRCA 2000	MIRCA OS	0.8595946	0.9652307
## 5:	Meier	MIRCA 2000	0.8459998	0.9635799
## 6:	GMIA	Meier	0.8436327	0.9596853
## 7:	MIRCA OS	SPAM2010	0.8333674	0.9594505
## 8:	LUH2	Nagaraj	0.8977533	0.9561767
## 9:	MIRCA 2000	SPAM2010	0.8250296	0.9536111
## 10:	GAEZ+2015	MIRCA 2000	0.8187728	0.9535817
## 11:	GAEZ+2015	MIRCA OS	0.8204898	0.9521117
## 12:	GAEZ+2015	GMIA	0.8146970	0.9500532
## 13:	Meier	SPAM2010	0.8096173	0.9466166
## 14:	Meier	MIRCA OS	0.8119047	0.9429633
## 15:	GAEZ+2015	Meier	0.7973006	0.9353605
## 16:	GAEZ+2015	SPAM2010	0.7717534	0.9270248
## 17:	GIAM	MIRCA 2000	0.7717169	0.9150786
## 18:	GIAM	MIRCA OS	0.7601485	0.9139648
## 19:	GIAM	SPAM2010	0.7645711	0.9139506
## 20:	GIAM	Meier	0.7714894	0.9111299
## 21:	GIAM	GMIA	0.7609975	0.9067002
## 22:	GIAM	Nagaraj	0.8031987	0.9044408
## 23:	Meier	Nagaraj	0.7968289	0.8993616
## 24:	GAEZ+2015	GIAM	0.7402983	0.8991512
## 25:	Nagaraj	SPAM2010	0.7761437	0.8971564
## 26:	GMIA	Nagaraj	0.7818313	0.8961145
## 27:	MIRCA OS	Nagaraj	0.7756057	0.8950570
## 28:	MIRCA 2000	Nagaraj	0.7814858	0.8935640
## 29:	GMIA	LUH2	0.7711712	0.8901905
## 30:	LUH2	MIRCA 2000	0.7701480	0.8869588
## 31:	GRIPC	Nagaraj	0.7656340	0.8864691
## 32:	LUH2	Meier	0.7698723	0.8856886
## 33:	LUH2	MIRCA OS	0.7580659	0.8823827
## 34:	GRIPC	SPAM2010	0.7197618	0.8816127
## 35:	GMIA	GRIPC	0.7205550	0.8809232
## 36:	LUH2	SPAM2010	0.7535114	0.8787780
## 37:	GAEZ+2015	Nagaraj	0.7444534	0.8763956
## 38:	GRIPC	Meier	0.7150236	0.8731797
## 39:	GRIPC	MIRCA 2000	0.7172403	0.8706369
## 40:	GIAM	GRIPC	0.7091910	0.8680663
## 41:	GRIPC	LUH2	0.7301817	0.8674414
## 42:	GIAM	LUH2	0.7445265	0.8669448
## 43:	GRIPC	MIRCA OS	0.7031575	0.8664374
## 44:	GAEZ+2015	LUH2	0.7097899	0.8530376
## 45:	GAEZ+2015	GRIPC	0.6858624	0.8455006
##	dataset_1	dataset_2	kendall	spearman

```
##          <char>      <char>      <num>      <num>
# Pairwise top-N overlap (identifiable-only) -----

topN_overlap_ident <- compute_topN_overlap(country_totals_ident,
                                           dataset_names = new_dataset_names,
                                           N = 20)
topN_overlap_ident[order(-jaccard_topN)]
```

##	dataset_1	dataset_2	N	intersection_N	jaccard_topN
##	<char>	<char>	<num>	<int>	<num>
## 1:	GIAM	GMIA	20	19	0.9047619
## 2:	GIAM	Meier	20	19	0.9047619
## 3:	GIAM	Nagaraj	20	19	0.9047619
## 4:	GMIA	MIRCA 2000	20	19	0.9047619
## 5:	GMIA	Nagaraj	20	19	0.9047619
## 6:	Meier	MIRCA OS	20	19	0.9047619
## 7:	Nagaraj	SPAM2010	20	19	0.9047619
## 8:	GAEZ+2015	MIRCA OS	20	18	0.8181818
## 9:	GIAM	LUH2	20	18	0.8181818
## 10:	GIAM	MIRCA 2000	20	18	0.8181818
## 11:	GIAM	MIRCA OS	20	18	0.8181818
## 12:	GIAM	SPAM2010	20	18	0.8181818
## 13:	GMIA	LUH2	20	18	0.8181818
## 14:	GMIA	Meier	20	18	0.8181818
## 15:	GMIA	MIRCA OS	20	18	0.8181818
## 16:	GMIA	SPAM2010	20	18	0.8181818
## 17:	GRIPC	LUH2	20	18	0.8181818
## 18:	LUH2	Nagaraj	20	18	0.8181818
## 19:	LUH2	SPAM2010	20	18	0.8181818
## 20:	Meier	MIRCA 2000	20	18	0.8181818
## 21:	Meier	Nagaraj	20	18	0.8181818
## 22:	Meier	SPAM2010	20	18	0.8181818
## 23:	MIRCA 2000	MIRCA OS	20	18	0.8181818
## 24:	MIRCA 2000	Nagaraj	20	18	0.8181818
## 25:	MIRCA OS	Nagaraj	20	18	0.8181818
## 26:	MIRCA OS	SPAM2010	20	18	0.8181818
## 27:	GAEZ+2015	Meier	20	17	0.7391304
## 28:	GAEZ+2015	Nagaraj	20	17	0.7391304
## 29:	GAEZ+2015	SPAM2010	20	17	0.7391304
## 30:	GIAM	GRIPC	20	17	0.7391304
## 31:	GMIA	GRIPC	20	17	0.7391304
## 32:	GRIPC	Meier	20	17	0.7391304
## 33:	GRIPC	MIRCA 2000	20	17	0.7391304
## 34:	GRIPC	Nagaraj	20	17	0.7391304
## 35:	GRIPC	SPAM2010	20	17	0.7391304
## 36:	LUH2	Meier	20	17	0.7391304
## 37:	LUH2	MIRCA 2000	20	17	0.7391304

```

## 38: MIRCA 2000    SPAM2010    20                17    0.7391304
## 39:  GAEZ+2015      GIAM    20                16    0.6666667
## 40:  GAEZ+2015      GMIA    20                16    0.6666667
## 41:  GAEZ+2015      GRIPC   20                16    0.6666667
## 42:  GAEZ+2015 MIRCA 2000    20                16    0.6666667
## 43:      GRIPC    MIRCA OS    20                16    0.6666667
## 44:      LUH2     MIRCA OS    20                16    0.6666667
## 45:  GAEZ+2015      LUH2    20                15    0.6000000
##      dataset_1  dataset_2    N intersection_N jaccard_topN
##      <char>      <char> <num>                <int>        <num>

# HOW DO RANKINGS CHANGE WHEN NON-IDENTIFIABLE CELLS ARE MASKED? #####

# How much does each country's rank shift for each dataset? -----

rank_change <- merge(
  country_totals_all[, .(country, dataset, rank_all = rank)],
  country_totals_ident[, .(country, dataset, rank_ident = rank)],
  by = c("country", "dataset"),
  all.x = TRUE) %>%
  .[, rank_shift := rank_ident - rank_all]

rank_change

# PLOT RESULTS #####

# Arrange data -----

rank_corr_all[, pair := paste(dataset_1, dataset_2, sep = "-")]
rank_corr_ident[, pair := paste(dataset_1, dataset_2, sep = "-")]

rank_corr_long <- rbind(rank_corr_all[, .(pair, spearman, mode = "All cells")],
  rank_corr_ident[, .(pair, spearman, mode = "Identifiable cells")])

# order pairs by Spearman (all cells) so the y-axis is readable -----

pair_order <- rank_corr_all[order(spearman), pair]
rank_corr_long[, pair := factor(pair, levels = pair_order)]

# Plot rank correlation -----

p_rankcorr <- ggplot(rank_corr_long, aes(spearman, pair)) +
  geom_segment(data = dcast(rank_corr_long, pair ~ mode, value.var = "spearman"),
    aes(x = `All cells`, xend = `Identifiable cells`,
      y = pair, yend = pair),
    inherit.aes = FALSE,
    colour = "grey70", linewidth = 0.3) +
  geom_point(aes(colour = mode), size = 2, alpha = 0.5) +

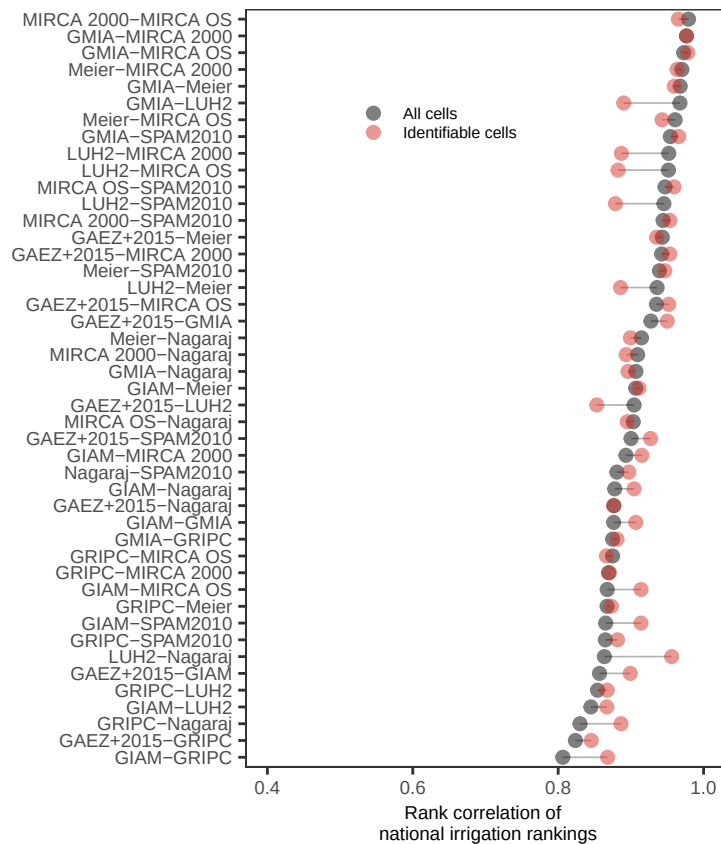
```

```

scale_x_continuous(limits = c(0.4, 1.0)) +
scale_color_manual(values = c("Identifiable cells" = "#d73027",
                              "All cells" = "black")) +
labs(x = "Rank correlation of \n national irrigation rankings",
     y = "",
     colour = NULL) +
theme_AP() +
theme(legend.position = c(0.4, 0.85))

```

p_rankcorr



```

# Only for top N countries -----

topN_overlap_all[, pair:= paste(dataset_1, dataset_2, sep = "-")]
topN_overlap_ident[, pair:= paste(dataset_1, dataset_2, sep = "-")]

topN_long <- rbind(topN_overlap_all[, .(pair, jaccard_topN, mode = "All cells")],
                  topN_overlap_ident[, .(pair, jaccard_topN, mode = "Identifiable cells")])

# order by All-cells Jaccard for consistency -----

pair_order_topN <- topN_overlap_all[order(jaccard_topN), pair]
topN_long[, pair:= factor(pair, levels = pair_order_topN)]

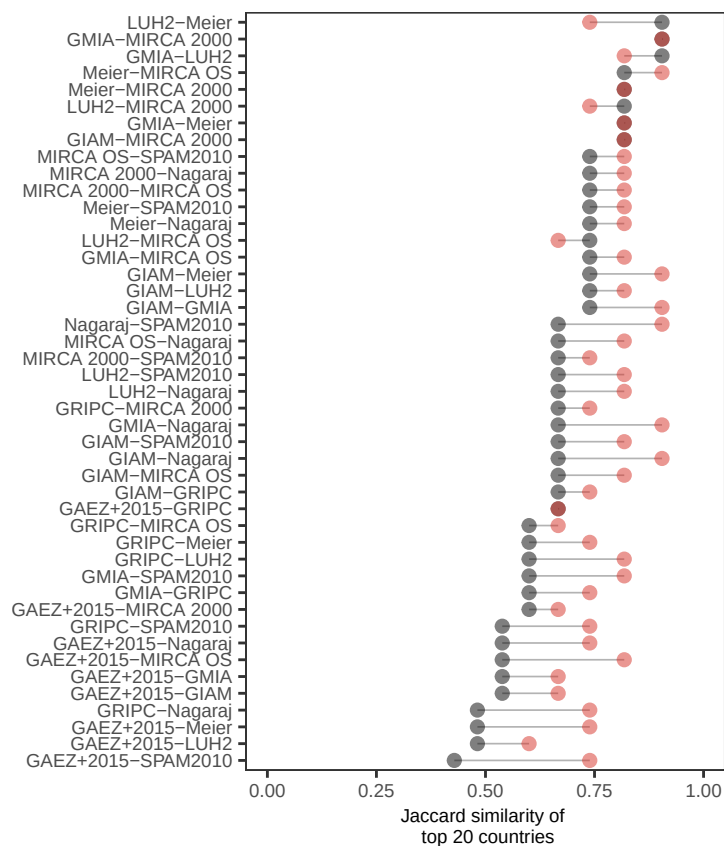
```



```
# Plot -----

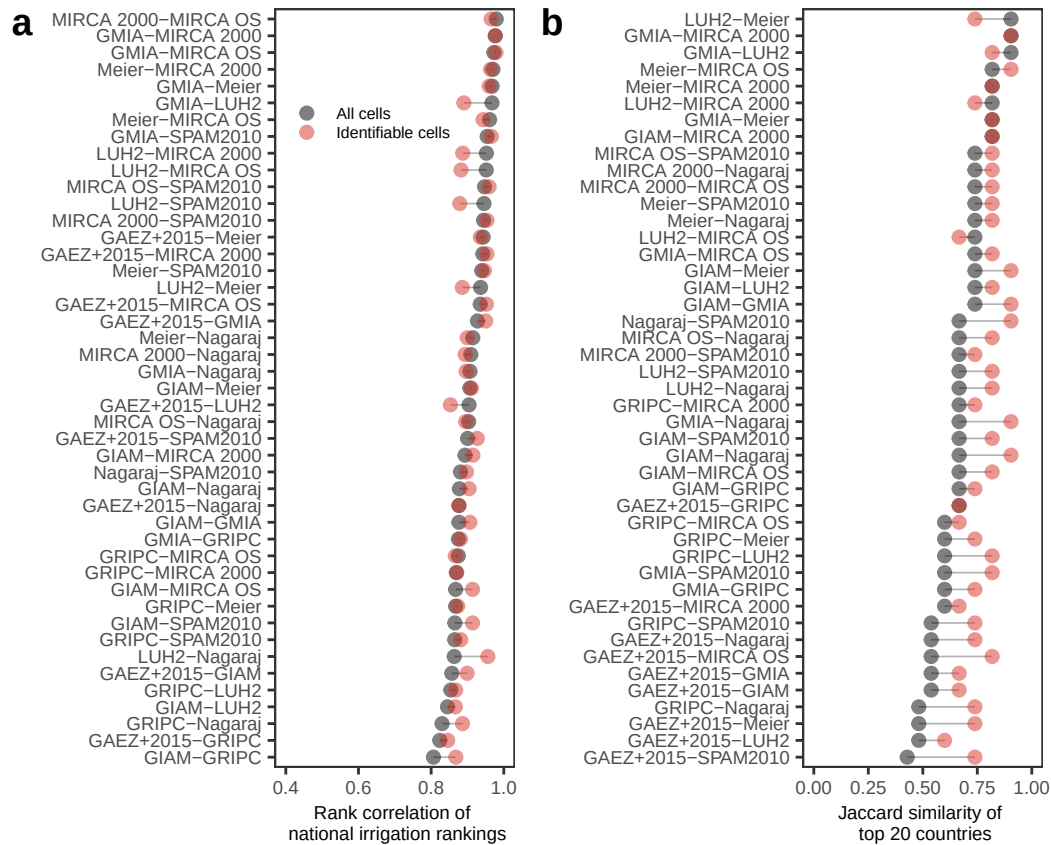
p_topN <- ggplot(topN_long, aes(x = jaccard_topN, y = pair)) +
  geom_segment(data = dcast(topN_long, pair ~ mode, value.var = "jaccard_topN"),
    aes(x = `All cells`, xend = `Identifiable cells`,
      y = pair, yend = pair), inherit.aes = FALSE, colour = "grey70",
    linewidth = 0.3) +
  geom_point(aes(colour = mode), size = 2, alpha = 0.5) +
  scale_x_continuous(limits = c(0, 1)) +
  scale_color_manual(values = c("Identifiable cells" = "#d73027",
    "All cells" = "black")) +
  labs(x = "Jaccard similarity of \n top 20 countries",
    y = "", colour = NULL) +
  theme_AP() +
  theme(legend.position = "none")
```

p_topN



```
# MERGE #####

plot_grid(p_rankcorr, p_topN, ncol = 2, labels = "auto")
```



4.3 Shift in country ranks

```
# SHIFT IN COUNTRY RANKS #####

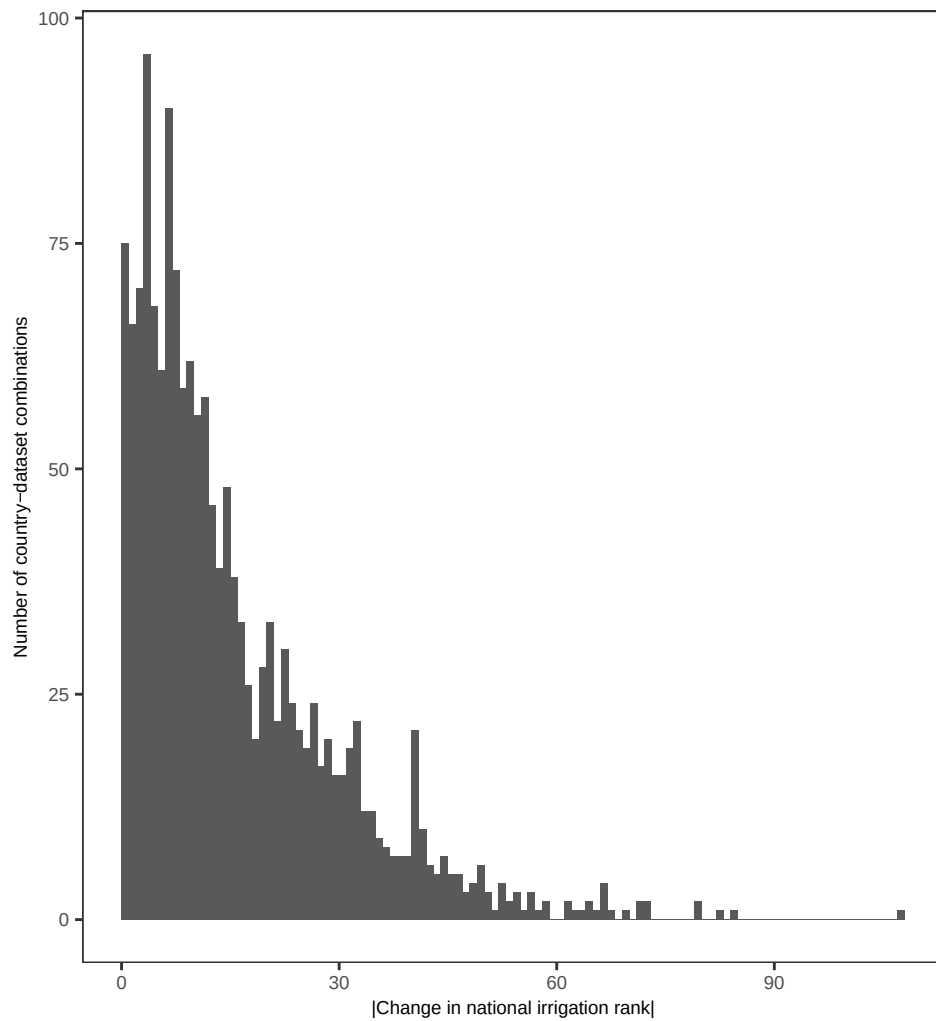
# Total rank shift -----

rank_change[, abs_shift:= abs(rank_shift)]
rank_change_clean <- rank_change[!is.na(abs_shift)]

# Histogram of rank shift -----

p_rankshift_hist <- ggplot(rank_change_clean, aes(abs_shift)) +
  geom_histogram(binwidth = 1, boundary = 0, closed = "left") +
  labs(x = "|Change in national irrigation rank|",
       y = "Number of country-dataset combinations") +
  theme_AP()

p_rankshift_hist
```



```
## Cumulative share -----

rank_shift_stats <- rank_change_clean[, .(share_ge_1 = mean(abs_shift >= 1),
  share_ge_5 = mean(abs_shift >= 5),
  share_ge_10 = mean(abs_shift >= 10),
  share_ge_20 = mean(abs_shift >= 20),
  share_ge_50 = mean(abs_shift >= 50))]

rank_shift_stats

##      share_ge_1 share_ge_5 share_ge_10 share_ge_20 share_ge_50
##      <num>      <num>      <num>      <num>      <num>
## 1:  0.9522293  0.7611465   0.5420382   0.2923567   0.02675159

# FOCUS ON THE SO-CALLED "TOP IRRIGATORS" #####

number_countries <- 20

# Countries consistently classified as top irrigators across datasets -----
```

```
top10_all <- country_totals_all[order(rank), .SD[1:number_countries], dataset] %>%
  .[, .N, by = country] %>%
  .[!country == ""] %>%
  .[N > 5]
```

```
top10_all
```

```
##           country      N
##           <char> <int>
##  1:           China    10
##  2:           India    10
##  3: United States    10
##  4:           Pakistan  10
##  5:           Iran     10
##  6:           Mexico    10
##  7:           Brazil    10
##  8:           Egypt     8
##  9:           Turkey    10
## 10: Uzbekistan    10
## 11:           Italy     8
## 12:           Spain    10
## 13:           Russia     6
## 14: Kazakhstan     8
## 15:           Thailand   8
## 16: Bangladesh     8
## 17:           Vietnam    9
## 18: Indonesia      8
## 19:           Iraq      6
```

Also with the median -----

```
top10_median <- country_totals_all[, .(median_rank = median(rank)), country] %>%
  .[order(median_rank)] %>%
  .[1:number_countries]
```

```
top10_median
```

```
##           country median_rank
##           <char>      <num>
##  1:           China         1.0
##  2:           India         2.0
##  3: United States         3.0
##  4:           Pakistan      4.0
##  5:           Iran         5.0
##  6:           Mexico        7.0
##  7: Indonesia         9.0
##  8:           Thailand      9.0
##  9: Bangladesh       11.0
```

```
## 10:      Brazil      11.5
## 11:      Vietnam     12.0
## 12:      Turkey      12.5
## 13:      Uzbekistan  12.5
## 14:      Spain       13.5
## 15:      Egypt       15.0
## 16:      Italy        15.5
## 17:      Russia      15.5
## 18:      Kazakhstan  18.5
## 19:      Iraq         20.0
## 20: Myanmar (Burma)  20.0
##          country median_rank
##          <char>      <num>
```

```
# See change of ranks in countries when only confirmed irrigation is
# accounted for -----
```

```
rank_focus <- merge(country_totals_all[, .(country, dataset, rank_all = rank)],
                    country_totals_ident[, .(country, dataset, rank_ident = rank)],
                    by = c("country", "dataset"), all = TRUE)

rank_focus[, rank_shift:= rank_ident - rank_all]
rank_focus
```

```
## Key: <country, dataset>
```

```
##          country      dataset rank_all rank_ident rank_shift
##          <char>      <char>      <num>      <num>      <num>
##    1:  Afghanistan GAEZ+2015    23.0         29         6
##    2:  Afghanistan   GIAM      37.0         30        -7
##    3:  Afghanistan   GMIA      24.0         27         3
##    4:  Afghanistan  GRIPC      44.0         31        -13
##    5:  Afghanistan   LUH2      21.0         27         6
##    ---
## 1626: Åland Islands MIRCA 2000   161.5         NA         NA
## 1627: Åland Islands MIRCA OS    161.0         NA         NA
## 1628: Åland Islands   Meier     150.0         NA         NA
## 1629: Åland Islands  Nagaraj    136.0         NA         NA
## 1630: Åland Islands  SPAM2010   150.0         NA         NA
```

```
# Summarize -----
```

```
rank_focus_summary <- rank_focus[!is.na(rank_ident), .(median_shift = median(rank_shift, na.rm
max_drop = max(rank_shift, na.rm = TRUE),
max_rise = min(rank_shift, na.rm = TRUE),
share_drop5 = mean(rank_shift >= 5, na.rm = TRUE),
share_drop10 = mean(rank_shift >= 10, na.rm = TRUE)),
country] %>%
.[order(-median_shift)]
```

```
# Focus on top 20 irrigation hotspots -----
```

```
focus_countries <- top10_median$country
rank_focus_summary[country %in% focus_countries]
```

```
##          country median_shift max_drop max_rise share_drop5 share_drop10
##          <char>         <num>    <num>    <num>         <num>         <num>
##  1:         Brazil         12.5        21         6         1.0         0.9
##  2:         Russia         10.0        21         1         0.8         0.5
##  3:      Indonesia          5.5        12        -2         0.8         0.1
##  4:         Mexico          3.5         7        -1         0.4         0.0
##  5:         Turkey          3.0         9        -3         0.4         0.0
##  6:          Iran          2.5         5        -8         0.1         0.0
##  7:      Kazakhstan          1.0        12        -4         0.3         0.1
##  8:          China          0.0         0         0         0.0         0.0
##  9:          India          0.0         0         0         0.0         0.0
## 10: Myanmar (Burma)          0.0         4        -4         0.0         0.0
## 11:      Pakistan          0.0         0        -1         0.0         0.0
## 12:   United States          0.0         1         0         0.0         0.0
## 13:      Thailand         -1.0         1        -5         0.0         0.0
## 14:          Spain         -1.5         3        -6         0.0         0.0
## 15:          Italy         -3.0        -1        -8         0.0         0.0
## 16:      Vietnam         -3.0         1        -9         0.0         0.0
## 17:   Bangladesh         -5.0        -3       -14         0.0         0.0
## 18:          Iraq         -5.5         6        -9         0.1         0.0
## 19:   Uzbekistan         -5.5        -4       -11         0.0         0.0
## 20:          Egypt         -7.0        -3       -15         0.0         0.0
##          country median_shift max_drop max_rise share_drop5 share_drop10
##          <char>         <num>    <num>    <num>         <num>         <num>
```

```
# Focus on the countries whose ranking drops the most -----
```

```
rank_focus_summary[order(-median_shift)][1:number_countries]
```

```
##          country median_shift max_drop max_rise share_drop5
##          <char>         <num>    <num>    <num>         <num>
##  1:      Netherlands         64.50    108.0     23.0         1.0
##  2:          Hungary         48.50     62.5     10.0         1.0
##  3:      Guatemala         41.25     72.5    -28.0         0.9
##  4: United Arab Emirates         40.50     79.0   -24.5         0.8
##  5:          Eswatini         39.00     71.0    -3.5         0.9
##  6:      Madagascar         39.00     79.5      8.0         1.0
##  7:          Mali         36.00     57.5   -13.0         0.7
##  8:          Germany         30.50     67.5     18.0         1.0
##  9:          Slovakia         30.50     52.0      1.0         0.9
## 10:          Bulgaria         27.00     52.0   -42.0         0.9
## 11:          Jamaica         27.00     40.0      3.5         0.9
## 12:      Costa Rica         26.50     49.0      7.0         1.0
```

```
## 13:      Venezuela      26.50      40.0      11.0      1.0
## 14:      Ethiopia      24.00      71.5       1.0      0.9
## 15:      Nigeria      23.50      65.5       5.0      1.0
## 16:      Ecuador      21.00      33.0      10.0      1.0
## 17:    New Zealand      21.00      31.0     -19.5      0.9
## 18:      Somalia      19.50      66.0     -33.5      0.9
## 19:      Cyprus      19.00      52.0       7.5      1.0
## 20:    Timor-Leste      17.75      48.0      -7.0      0.7
##           country median_shift max_drop max_rise share_drop5
##           <char>      <num>      <num>      <num>      <num>
## share_drop10
##           <num>
## 1:           1.0
## 2:           1.0
## 3:           0.9
## 4:           0.8
## 5:           0.8
## 6:           0.9
## 7:           0.7
## 8:           1.0
## 9:           0.8
## 10:          0.9
## 11:          0.8
## 12:          0.9
## 13:          1.0
## 14:          0.9
## 15:          0.8
## 16:          1.0
## 17:          0.8
## 18:          0.9
## 19:          0.8
## 20:          0.7
## share_drop10
##           <num>
```

```
# Plot the results -----
plot_shift_ranks <- rank_focus[country %in% focus_countries] %>%
  .[, .(country, dataset,
    `All irrigated cells` = rank_all,
    `Confirmed irrigation` = rank_ident)] %>%
  melt(., measure.vars = c("Confirmed irrigation", "All irrigated cells")) %>%
  ggplot(., aes(dataset, value, color = variable,
    group = interaction(country, variable))) +
  geom_line(linewidth = 1) +
  scale_y_reverse() +
  scale_color_manual(values = c("Confirmed irrigation" = "#d73027",
    "All irrigated cells" = "black"),
```

```

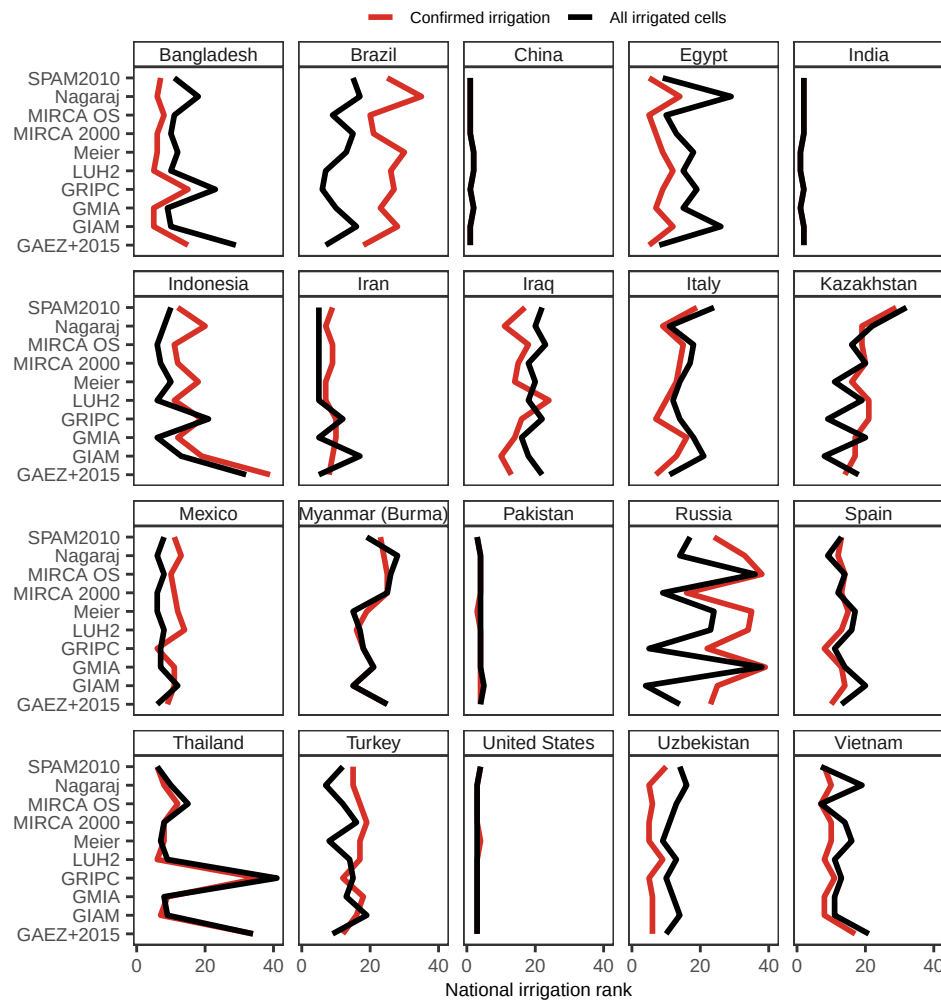
    name = "") +
  scale_y_continuous(breaks = breaks_pretty(n = 3)) +
  facet_wrap(~country) +
  labs(y = "National irrigation rank", x = NULL) +
  coord_flip() +
  theme_AP() +
  theme(legend.position = "top")

```

Scale for y is already present.

Adding another scale for y, which will replace the existing scale.

plot_shift_ranks



COMPUTE AREA RETAINED AND LOST OF TOP 20 COUNTRIES

```

country_loss <- rbindlist(lapply(new_dataset_names, function(d) {

  dt_country <- dt2[country %in% top10_all$country]

  dt_country[, .(dataset = d,

```



```

        total_area = sum(get(d), na.rm = TRUE),
        retained_area = sum(get(d)[is.na(tau_max_ha)], na.rm = TRUE),
        lost_area = sum(get(d)[!is.na(tau_max_ha)], na.rm = TRUE)),
country]
)))

country_loss[, share_lost:= lost_area / total_area]

# Compute mean loss per country across datasets to reorder -----

country_order <- country_loss[, .(mean_share_lost = mean(share_lost, na.rm = TRUE)),
  by = country][order(-mean_share_lost)] # decreasing order

# Set factor levels in that order-----

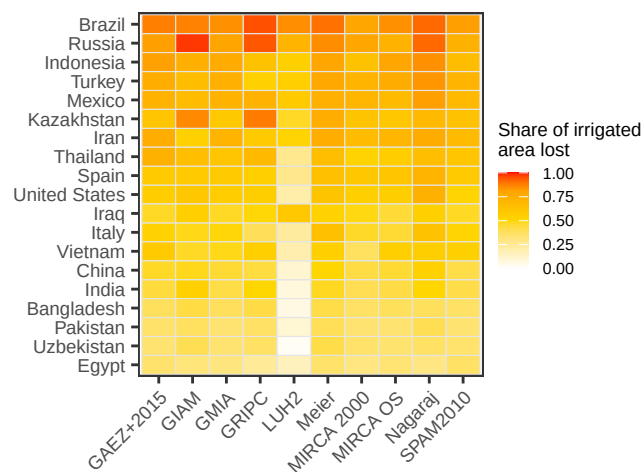
country_loss[, country_f:= factor(country, levels = rev(country_order$country))]

# Plot tileplot -----

plot_tileplot <- ggplot(country_loss, aes(dataset, country_f, fill = share_lost)) +
  geom_tile(color = "grey90") +
  scale_fill_gradientn(colours = c("white", "gold", "orange", "red"),
    values = c(0, 0.5, 0.8, 1),
    limits = c(0, 1),
    name = "Share of irrigated\narea lost") +
  scale_y_discrete(name = NULL) +
  theme_AP() +
  labs(x = "", y = "") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
    legend.position = "right")

plot_tileplot

```



4.4 Fraction of irrigated areas lost

```
# COLLAPSE ASSESSMENT ACROSS A CONTINUUM OF AGREEMENT #####

dt_share_lost <- dcast(dt, lon + lat + country + code + continent + resolution ~ dataset,
  value.var = "mha")

# Define resolution specific thresholds -----

zero_tol_mha_fun <- function(res) {
  switch(res,
    "0.2deg" = 400 / 1e6, # 0.0004 Mha
    "0.4deg" = 1600 / 1e6, # 0.0016 Mha
    "1deg" = 10000 / 1e6, # 0.01 Mha
    stop("Unknown resolution: ", res)
  )
}

# Make copy just in case -----

dt_pres <- copy(dt_share_lost)

# Apply resolution-specific thresholds -----

dt_pres[, (dataset_names) := {
  tol_mha <- zero_tol_mha_fun(resolution[1])
  lapply(.SD, function(x) as.integer(x >= tol_mha))
},
  resolution,
  .SDcols = dataset_names]

# How many datasets say "irrigated" per cell -----

dt_pres[, n_pos := rowSums(.SD, na.rm = TRUE), .SDcols = dataset_names]

n_datasets <- length(dataset_names) # should be 10

# Melt for k-of-n agreement -----

grid_long <- melt(dt_pres, id.vars = c("lon", "lat", "country", "code",
  "continent", "resolution", "n_pos"),
  measure.vars = dataset_names,
  variable.name = "dataset",
  value.name = "present")

# Run calculations for multiple k (10 down to 5) -----

ks <- n_datasets:1
```

```

loss_by_k <- rbindlist(lapply(ks, compute_loss_for_k), use.names = TRUE)

# Summarize results -----

country_loss_by_k <- loss_by_k[, .(min = min(share_lost, na.rm = TRUE),
      max = max(share_lost, na.rm = TRUE),
      mean = mean(share_lost, na.rm = TRUE)),
      .(country, k)] %>%
  .[, allowed_disagree:= 10 - k]

# PLOT RESULTS: THE GLOBAL COLLAPSE CURVE #####

global_k <- country_loss_by_k[, .(mean_loss = mean(mean, na.rm = TRUE),
      median_loss = median(mean, na.rm = TRUE)), k] %>%
  .[order(-k)] %>%
  .[, allowed_disagree:= 10 - k]

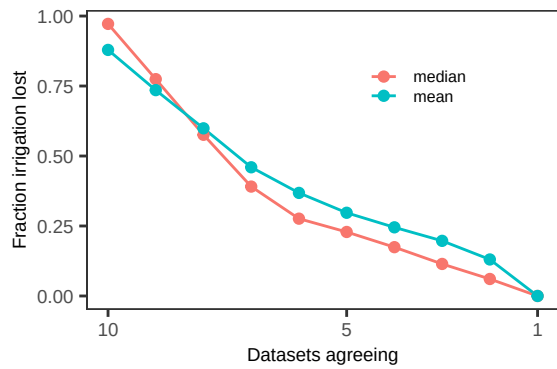
print(global_k)

##           k mean_loss median_loss allowed_disagree
##      <int>      <num>      <num>          <num>
##  1:      10 0.8789739  0.97198163              0
##  2:       9 0.7353161  0.77453888              1
##  3:       8 0.5989455  0.57571490              2
##  4:       7 0.4597504  0.39078182              3
##  5:       6 0.3682308  0.27614700              4
##  6:       5 0.2972035  0.22858468              5
##  7:       4 0.2451639  0.17430556              6
##  8:       3 0.1969250  0.11438299              7
##  9:       2 0.1304492  0.06068063              8
## 10:       1 0.0000000  0.00000000              9

plot_global_k <- global_k %>%
  melt(., measure.vars = c("median_loss", "mean_loss")) %>%
  ggplot(., aes(k, value, color = variable)) +
  geom_line() +
  geom_point() +
  scale_color_discrete(labels = c(median_loss = "median", mean_loss = "mean"),
      name = "") +
  labs(x = "Datasets agreeing", y = "Fraction irrigation lost") +
  theme_AP() +
  scale_x_reverse(breaks = c(10, 5, 1)) +
  theme(legend.position = c(0.7, 0.8))

plot_global_k

```



```
# COUNTRY-LEVEL COLLAPSE PROFILES #####
```

```
# Optional: keep only countries with mean loss < 1 at k=5
```

```
collapse_profiles <- country_loss_by_k[country %in% top10_all$country] %>%
  .[, allowed_disagree:= 10 - k]
```

```
# Under unanimity
```

```
tmp <- collapse_profiles[k == 10 & country %in% top10_all$country] %>%
  .[order(-mean)]
```

```
tmp
```

##	country	k	min	max	mean	allowed_disagree
##	<char>	<int>	<num>	<num>	<num>	<num>
## 1:	Brazil	10	0.88064269	0.9533493	0.9275510	0
## 2:	Russia	10	0.75986842	0.9877311	0.9062287	0
## 3:	Indonesia	10	0.69328494	0.9186718	0.8443967	0
## 4:	Mexico	10	0.73804971	0.8904438	0.8298258	0
## 5:	Kazakhstan	10	0.65354331	0.9279574	0.7943901	0
## 6:	Turkey	10	0.61011905	0.8732463	0.7883040	0
## 7:	United States	10	0.47315809	0.8526610	0.7551067	0
## 8:	Iran	10	0.53748783	0.8364325	0.7399565	0
## 9:	Thailand	10	0.42953020	0.7826087	0.6993478	0
## 10:	Spain	10	0.36942675	0.7899576	0.6815685	0
## 11:	Italy	10	0.46445498	0.7618546	0.6523922	0
## 12:	Vietnam	10	0.38235294	0.7169118	0.6246655	0
## 13:	Iraq	10	0.45000000	0.6653144	0.5841619	0
## 14:	China	10	0.30208040	0.6298919	0.5560644	0
## 15:	India	10	0.15740741	0.6092448	0.5245148	0
## 16:	Pakistan	10	0.37068063	0.5587372	0.4671962	0
## 17:	Egypt	10	0.28909953	0.5575221	0.4548616	0
## 18:	Uzbekistan	10	0.07220217	0.4711934	0.3867142	0
## 19:	Bangladesh	10	0.08823529	0.4213333	0.3637888	0

```
tmp[, .(n_above_90 = sum(mean > 0.9),
  n_above_80 = sum(mean > 0.8),
  n_above_50 = sum(mean > 0.5),
```

```

    pct_above_90 = 100 * mean(mean > 0.9),
    pct_above_80 = 100 * mean(mean > 0.8),
    pct_above_50 = 100 * mean(mean > 0.5))]
```

```

##      n_above_90 n_above_80 n_above_50 pct_above_90 pct_above_80 pct_above_50
##      <int>      <int>      <int>      <num>      <num>      <num>
## 1:           2           4          15      10.52632      21.05263      78.94737
```

```

# Under the most permissive criteria (5/10)
collapse_profiles[k == 5 & country %in% top10_all$country] %>%
  .[order(-mean)]
```

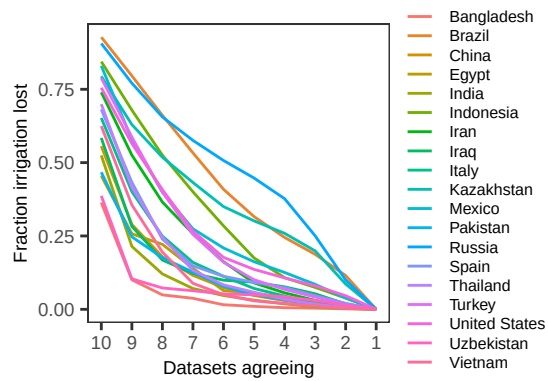
```

##      country      k      min      max      mean allowed_disagree
##      <char> <int>      <num>      <num>      <num>      <num>
## 1:      Russia      5 0.028199566 0.88873950 0.447484937          5
## 2:      Brazil      5 0.069400631 0.62105969 0.316117863          5
## 3: Kazakhstan      5 0.017716535 0.77691363 0.301456424          5
## 4:      Indonesia      5 0.040892193 0.30895091 0.175197788          5
## 5:       Mexico      5 0.031192661 0.35785686 0.162330836          5
## 6: United States      5 0.035277325 0.32532615 0.136061138          5
## 7:       Turkey      5 0.013750955 0.22883406 0.100755304          5
## 8:        Iraq      5 0.000000000 0.32962138 0.092350998          5
## 9:       Spain      5 0.005928854 0.25742574 0.091921916          5
## 10:      Iran      5 0.041157294 0.17183196 0.090127081          5
## 11:      Italy      5 0.001572327 0.20021075 0.071055620          5
## 12: Thailand      5 0.007352941 0.12020460 0.058142125          5
## 13:      Egypt      5 0.004484305 0.16853933 0.055262476          5
## 14: Pakistan      5 0.004280822 0.11967695 0.049999775          5
## 15: Uzbekistan      5 0.000000000 0.11728395 0.049347680          5
## 16:      China      5 0.003233049 0.08483041 0.047069619          5
## 17:      India      5 0.003975265 0.08171255 0.031221534          5
## 18:      Vietnam      5 0.002083333 0.07843137 0.028810410          5
## 19: Bangladesh      5 0.000000000 0.02840909 0.009946725          5
```

```

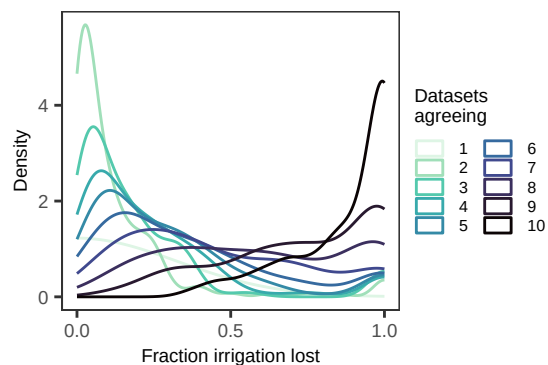
plot_collapse_profiles <- ggplot(collapse_profiles, aes(k, mean, group = country,
  color = country)) +
  scale_color_discrete(name = "") +
  geom_line() +
  scale_x_reverse(breaks = c(10:1)) +
  labs(x = "Datasets agreeing", y = "Fraction irrigation lost") +
  theme_AP()

plot_collapse_profiles
```

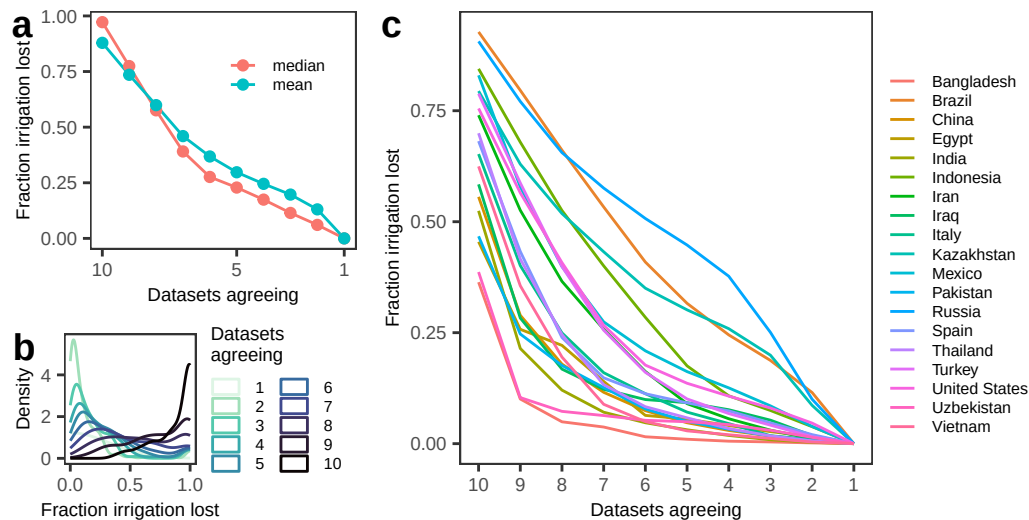


```
# Global distribution by k -----
plot_curves_country <- ggplot(country_loss_by_k, aes(mean, color = factor(k), group = k)) +
  geom_density() +
  labs(x = "Fraction irrigation lost", y = "Density", color = "Datasets \nagreeing") +
  theme_AP() +
  theme(legend.position = "right") +
  scale_x_continuous(breaks = breaks_pretty(n = 3)) +
  scale_color_viridis_d(option = "mako", direction = -1) +
  guides(colour = guide_legend(ncol = 2))
```

plot_curves_country



```
# MERGE #####
left <- plot_grid(plot_global_k, plot_curves_country, ncol = 1, labels = "auto",
  rel_heights = c(0.6, 0.4))
plot_grid(left, plot_collapse_profiles, ncol = 2, labels = c("", "c"),
  rel_widths = c(0.35, 0.65))
```



PLOTS SHOWING K-OF-10 AGREEMENT

Prepare data -----

```
dt_plot <- dt_pres[, .N, .(resolution, n_pos)]
dt_plot[, frac:= N / sum(N), by = resolution]
```

Fraction of cells in the consensus core (>=9/10) -----

```
dt_plot[order(n_pos)] %>%
  .[n_pos >= 9] %>%
  .[, sum(frac), resolution]
```

```
## resolution      V1
##      <char>      <num>
## 1:      1deg 0.2196331
## 2:      0.4deg 0.1945520
## 3:      0.2deg 0.1620483
```

Fraction of cells irrigated by less than half of the datasets .-----

```
dt_plot[order(n_pos)] %>%
  .[n_pos < 5 & n_pos != 0] %>%
  .[, sum(frac), resolution]
```

```
## resolution      V1
##      <char>      <num>
## 1:      0.2deg 0.3057287
## 2:      0.4deg 0.2735299
## 3:      1deg 0.2205007
```

Compute statistics -----

```
dt_summary <- dt_plot[, .(total = sum(N),
```

```

        absence = sum(N[n_pos == 0]),
        contested = sum(N[n_pos >= 1 & n_pos <= 8]),
        fewer_than_half = sum(N[n_pos < 5 & n_pos > 0]),
        presence = sum(N[n_pos >= 9]),
        resolution]

dt_summary[, `:=`(frac_absence = absence / total,
                  frac_contested = contested / total,
                  frac_fewer_than_half = fewer_than_half / total,
                  frac_presence = presence / total)]

dt_summary

##      resolution  total absence contested fewer_than_half presence frac_absence
##      <char>    <int>   <int>    <int>         <int>    <int>         <num>
## 1:      1deg    8068    3473     2823          1779    1772         0.4304660
## 2:      0.2deg 129628  43077    65545          39631   21006         0.3323125
## 3:      0.4deg  39758  14514    17509          10875    7735         0.3650586
##      frac_contested frac_fewer_than_half frac_presence
##      <num>          <num>          <num>
## 1:      0.3499008          0.2205007      0.2196331
## 2:      0.5056392          0.3057287      0.1620483
## 3:      0.4403894          0.2735299      0.1945520

# Probability that a dataset detects irrigation in a given cell -----

dt_k <- dt_pres[, .N, .(resolution, n_pos)]

dt_prob <- dt_k[, .(mean_k = weighted.mean(n_pos, N),
                detection_prob = weighted.mean(n_pos, N) / 10),
                resolution]

dt_prob

##      resolution  mean_k detection_prob
##      <char>    <num>         <num>
## 1:      1deg  3.410387         0.3410387
## 2:      0.2deg 3.426945         0.3426945
## 3:      0.4deg 3.498114         0.3498114

# In how many cells do fewer than half of the datasets detect irrigation? -----

dt_weak <- dt_k[, .(total_cells = sum(N),
  no_irrigation = sum(N[n_pos == 0]),
  any_irrigation = sum(N[n_pos >= 1]),
  weak_detection = sum(N[n_pos >= 1 & n_pos <= 5])), resolution]

dt_weak[, `:=`(frac_weak_all = weak_detection / total_cells,
                frac_weak_given_any = weak_detection / any_irrigation)]

```



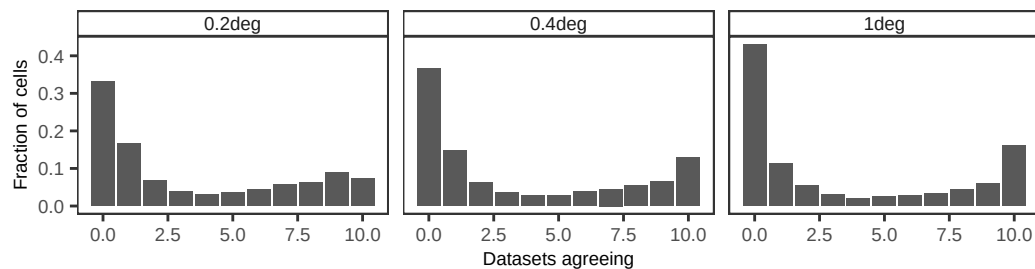
```
dt_weak
```

```
## resolution total_cells no_irrigation any_irrigation weak_detection
##      <char>      <int>      <int>      <int>      <int>
## 1:      1deg      8068      3473      4595      1977
## 2:      0.2deg    129628    43077    86551    44255
## 3:      0.4deg    39758    14514    25244    11993
##      frac_weak_all frac_weak_given_any
##      <num>      <num>
## 1:      0.2450421      0.4302503
## 2:      0.3414000      0.5113170
## 3:      0.3016500      0.4750832
```

```
# Plot distribution -----
```

```
p1 <- plot_agreement <- ggplot(dt_plot, aes(n_pos, frac)) +
  geom_col() +
  facet_wrap(~resolution) +
  labs(x = "Datasets agreeing",
       y = "Fraction of cells") +
  theme_AP()
```

p1



```
# Prepare data for cumulative plot -----
```

```
dt_ecdf <- dt_pres[, .N, .(resolution, n_pos)]
dt_ecdf[, frac := N / sum(N), resolution]
setorder(dt_ecdf, resolution, n_pos)
dt_ecdf[, cdf := cumsum(frac), resolution]

dt_ecdf[, resolution := factor(resolution, levels = c("0.2deg", "0.4deg", "1deg"))]
```

```
# Plot -----
```

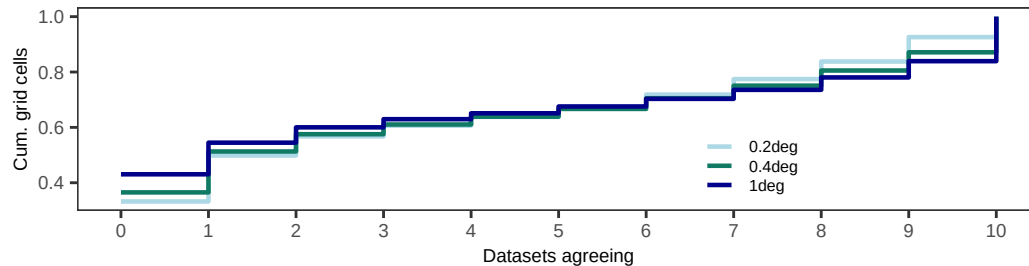
```
p2 <- ggplot(dt_ecdf, aes(n_pos, cdf, colour = resolution)) +
  geom_step(linewidth = 0.8) +
  scale_x_continuous(breaks = 0:10) +
  scale_color_manual(values = res_palette, name = "") +
  labs(x = "Datasets agreeing", y = "Cum. grid cells",
```

```

    colour = "Resolution") +
  theme_AP() +
  theme(legend.position = c(0.7, 0.3))

```

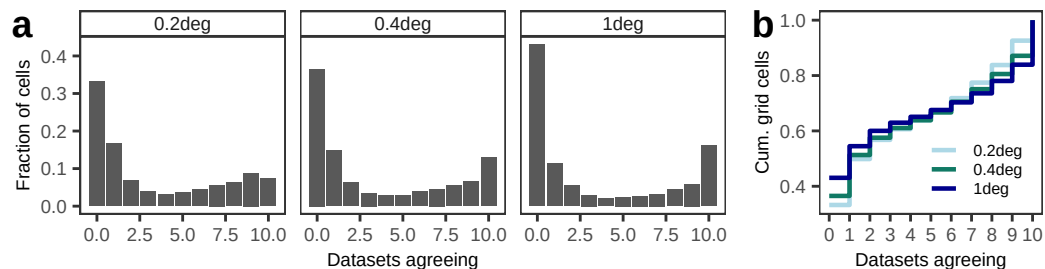
p2



```

plot_grid(p1, p2, ncol = 2, rel_widths = c(0.7, 0.3), labels = "auto")

```



Plot: consensus regime plot -----

```

dt_regime <- dt_k[, .(absence = sum(N[n_pos == 0]),
  contested = sum(N[n_pos >= 1 & n_pos <= 8]),
  presence = sum(N[n_pos >= 9])), resolution]

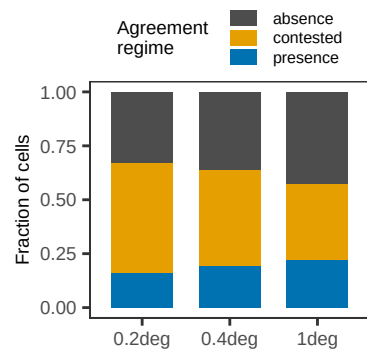
dt_regime <- melt(dt_regime, id.vars = "resolution", variable.name = "regime",
  value.name = "N")

dt_regime[, frac := N / sum(N), by = resolution]

plot_regime <- ggplot(dt_regime, aes(resolution, frac, fill = regime)) +
  geom_col(width = 0.7) +
  scale_fill_manual(values = c(absence = "#4D4D4D",
    contested = "#E69F00",
    presence = "#0072B2"),
    guide = guide_legend(nrow = 3),
    name = "Agreement \nregime") +
  labs(x = NULL, y = "Fraction of cells") +
  theme_AP() +
  theme(legend.position = "top")

```

plot_regime



5 Consequences for crop production

5.1 Extract data from EarthStat

```
# CODE TO RETRIEVE DATA FROM EARTHSTAT AND PRODUCE CROP DATASETS #####

# PATHS-----

earthstat_root <- "HarvestedAreaYield175Crops_Geotiff"
earthstat_geotiff <- file.path(earthstat_root, "GeoTiff")

agreement_file <- "./datasets/irrigated_areas_regridded/irrigated_areas_regridded_all.csv"

out_dir <- "./datasets/earthstat_regridded"
dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)

# TARGETS-----

targets <- data.table(
  res_tag = c("02", "04", "10"),
  resolution_label = c("0.2deg", "0.4deg", "1deg"),
  target_file = c(
    "mha_stack_ll_02_aligned_NEAR.tif",
    "mha_stack_ll_04_aligned_NEAR.tif",
    "mha_stack_ll_10_aligned_NEAR.tif"
  )
)

# FIND EARTHSTAT PRODUCTION FILES-----

prod_files <- list.files(
  path = earthstat_geotiff,
  pattern = "_Production\\.tif$",
  recursive = TRUE,
  full.names = TRUE
)

if (length(prod_files) == 0) stop("No *_Production.tif files found.")

earthstat_index <- data.table(
  crop = sub("_Production\\.tif$", "", basename(prod_files)),
  file = prod_files
)[order(crop)]

fwrite(earthstat_index, file.path(out_dir, "earthstat_production_file_index.csv"))

# LOAD AGREEMENT TABLE-----
```

```

agree_all <- dt_pres

# MAIN LOOP OVER TARGET RESOLUTIONS -----

ks <- 1:10

for (j in seq_len(nrow(targets))) {

  res_tag      <- targets$res_tag[j]
  res_label    <- targets$resolution_label[j]
  target_file  <- targets$target_file[j]

  message("\n=====")
  message("Processing resolution ", res_label, " (", res_tag, ")")
  message("=====\\n")

  out_dir_res <- file.path(out_dir, paste0("res_", res_tag))
  dir.create(out_dir_res, recursive = TRUE, showWarnings = FALSE)

  template_r <- rast(target_file)[[1]]
  values(template_r) <- NA_real_

  agree_dt <- copy(agree_all)[resolution == res_label, .(lon, lat, n_pos)]
  agree_dt <- unique(agree_dt)

  dup_check <- agree_dt[, .N, by = .(lon, lat)][N > 1]
  if (nrow(dup_check) > 0) {
    stop("Duplicated lon/lat rows in agreement table for resolution ", res_label)
  }

  setkey(agree_dt, lon, lat)

  #####
  # STEP 1: HARMONIZE EACH CROP TO THIS RESOLUTION
  #####

  qa_list <- lapply(seq_len(nrow(earthstat_index)), function(i) {

    crop <- earthstat_index$crop[i]
    file <- earthstat_index$file[i]

    out_file <- file.path(out_dir_res, paste0(crop, "_Production_ll_", res_tag, ".tif"))

    harmonize_one_production_to_file(
      file = file,
      crop = crop,
      template_r = template_r,

```

```

    out_file = out_file,
    method = "bilinear",
    max_valid_value = 1e9
  )
})

qa_dt <- rbindlist(qa_list)
fwrite(
  qa_dt,
  file.path(out_dir_res, paste0("earthstat_harmonized_files_", res_tag, ".csv"))
)

#####
# STEP 2: SUMMARISE PER CROP
#####

crop_results <- vector("list", length = nrow(earthstat_index))
global_results <- vector("list", length = nrow(earthstat_index))

for (i in seq_len(nrow(earthstat_index))) {

  crop <- earthstat_index$crop[i]
  file_ll <- file.path(out_dir_res, paste0(crop, "_Production_ll_", res_tag, ".tif"))

  message("Summarising ", crop, " at ", res_label, "°")

  r <- rast(file_ll)

  dt_crop <- as.data.table(as.data.frame(r, xy = TRUE, na.rm = FALSE))
  setnames(dt_crop, c("x", "y", crop), c("lon", "lat", "production_tons"))

  dt_crop <- dt_crop[!is.na(production_tons) & production_tons > 0]

  setkey(dt_crop, lon, lat)
  dt_crop <- agree_dt[dt_crop]
  dt_crop[is.na(n_pos), n_pos := 0L]

  total_all <- dt_crop[, sum(production_tons, na.rm = TRUE)]

  crop_out <- data.table(
    crop = crop,
    k = c(0L, ks),
    production_tons = NA_real_
  )

  crop_out[k == 0, production_tons := total_all]

```

```

for (kval in ks) {
  crop_out[
    k == kval,
    production_tons := dt_crop[n_pos >= kval, sum(production_tons, na.rm = TRUE)]
  ]
}

tons_k1 <- crop_out[k == 1, production_tons]
tons_k5 <- crop_out[k == 5, production_tons]

crop_out[, `:=`(
  rule = ifelse(k == 0, "all_cells", paste0("k_ge_", k)),
  resolution = res_label,
  res_tag = res_tag,
  share_of_all_cells = production_tons / total_all,
  share_of_k1 = if (!is.na(tons_k1) && tons_k1 > 0) production_tons / tons_k1 else NA_real,
  share_of_k5 = if (!is.na(tons_k5) && tons_k5 > 0) production_tons / tons_k5 else NA_real,
)]

crop_results[[i]] <- crop_out
global_results[[i]] <- crop_out[, .(k, production_tons)]

rm(r, dt_crop, crop_out)
gc()
}

#####
# STEP 3: SAVE PER-CROP RESULTS
#####

prod_k_crop <- rbindlist(crop_results)
setorder(prod_k_crop, crop, k)

fwrite(
  prod_k_crop,
  file.path(out_dir_res, paste0("earthstat_production_by_crop_k_agreement_", res_tag, ".csv")),
)

#####
# STEP 4: GLOBAL SUMMARY
#####

global_k <- rbindlist(global_results)[
  , .(production_tons = sum(production_tons, na.rm = TRUE)),
  by = k
][order(k)]

```

```

global_k[, `:=`(
  rule = ifelse(k == 0, "all_cells", paste0("k_ge_", k)),
  resolution = res_label,
  res_tag = as.integer(res_tag)
)]

global_all <- global_k[k == 0, production_tons]
global_k1 <- global_k[k == 1, production_tons]
global_k5 <- global_k[k == 5, production_tons]

global_k[, `:=`(
  share_of_all_cells = production_tons / global_all,
  share_of_k1 = if (!is.na(global_k1) && global_k1 > 0) production_tons / global_k1 else NA,
  share_of_k5 = if (!is.na(global_k5) && global_k5 > 0) production_tons / global_k5 else NA
)]

print(global_k)

fwrite(
  global_k,
  file.path(out_dir_res, paste0("earthstat_global_production_k_agreement_", res_tag, ".csv"))
)

rm(template_r, agree_dt, qa_dt, prod_k_crop, global_k, crop_results, global_results)
gc()
}

```

5.2 Plot crop heatmap

```

# PLOT HEATMAP OF CROPS #####

# Define files to retrieve -----

files <- c(
  "./datasets/earthstat_regridded/res_02/earthstat_production_by_crop_k_agreement_02.csv",
  "./datasets/earthstat_regridded/res_04/earthstat_production_by_crop_k_agreement_04.csv",
  "./datasets/earthstat_regridded/res_10/earthstat_production_by_crop_k_agreement_10.csv"
)

prod_dt <- rbindlist(lapply(files, fread), use.names = TRUE, fill = TRUE)

# TOP 25 CROPS BASED ON TOTAL PRODUCTION AT 0.2 (k = 0) -----

top25 <- prod_dt[
  resolution == "0.2deg" & k == 0,
  .(baseline_tons = production_tons),
  by = crop
]

```



```

][order(-baseline_tons)][1:25]

# PREPARE HEATMAP TABLE (LOSS VS k >= 1) -----

plot_dt <- prod_dt[
  crop %in% top25$crop & k %in% 1:10,
  .(crop, k, resolution, share_of_k1)
]

plot_dt[, loss_vs_1:= 1 - share_of_k1]

# Order crops -----

crop_levels <- rev(top25$crop)
plot_dt[, crop:= factor(crop, levels = crop_levels)]

# Order resolution -----

plot_dt[, resolution:= factor(
  resolution,
  levels = c("0.2deg", "0.4deg", "1deg")
)]

# Reverse k axis (10 → 1) -----

plot_dt[, k := factor(k, levels = rev(1:10))]

#####
# PLOT
#####

crop_labels <- c(
  "apple" = "Apple",
  "cabbage" = "Cabbage",
  "grape" = "Grape",
  "orange" = "Orange",
  "banana" = "Banana",
  "watermelon" = "Watermelon",
  "clover" = "Clover",
  "tomato" = "Tomato",
  "oilpalm" = "Oil palm",
  "grassnes" = "Grasses (NES)",
  "sweetpotato" = "Sweet potato",
  "barley" = "Barley",
  "cassava" = "Cassava",
  "soybean" = "Soybean",
  "vegetablenes" = "Vegetables (NES)",

```

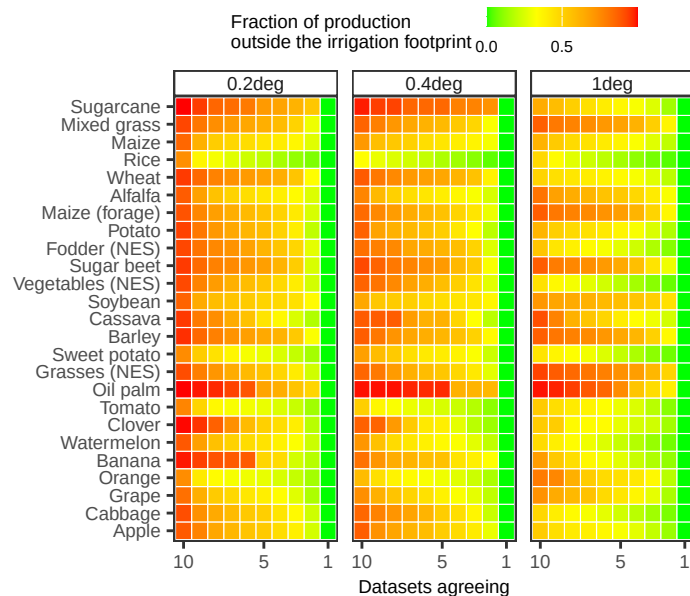
```

"sugarbeet" = "Sugar beet",
"fornes" = "Fodder (NES)",
"potato" = "Potato",
"maizefor" = "Maize (forage)",
"alfalfa" = "Alfalfa",
"wheat" = "Wheat",
"rice" = "Rice",
"maize" = "Maize",
"mixedgrass" = "Mixed grass",
"sugarcane" = "Sugarcane"
)

plot_heatmap <- ggplot(plot_dt, aes(k, crop, fill = loss_vs_1)) +
  geom_tile(color = "white") +
  scale_fill_gradientn(colours = c("green", "yellow", "orange", "red"),
    values = c(0, 0.5, 1), breaks = c(0, 0.5, 1),
    name = "Fraction of production \noutside the irrigation footprint") +
  scale_y_discrete(labels = crop_labels) +
  scale_x_discrete(breaks = c("10", "5", "1")) +
  facet_wrap(~resolution, ncol = 3) +
  labs(x = "Datasets agreeing", y = NULL) +
  theme_AP() +
  theme(legend.position = "top")

```

plot_heatmap



5.3 Plot breadbasket regions

```
# DEFINE BREADBASKET REGIONS AS NUMERIC BOUNDS #####

regions_dt <- data.table(
  region = c("Indo-Gangetic Plain",
             "North China Plain",
             "US High Plains",
             "Nile Delta/Basin",
             "Mekong Delta",
             "Mediterranean Basin"),
  xmin = c(68, 113, -104, 29.5, 104.5, -6),
  xmax = c(91, 121, -96, 33.0, 106.5, 36),
  ymin = c(21.5, 34, 32, 24.0, 8.5, 30),
  ymax = c(33.0, 41, 44, 31.5, 11.5, 46)
)

# LOCATION OF HARMONIZED RASTERS #####

earthstat_dirs <- list(
  "0.2deg" = "./datasets/earthstat_regridded/res_02",
  "0.4deg" = "./datasets/earthstat_regridded/res_04",
  "1deg" = "./datasets/earthstat_regridded/res_10")

res_tags <- c("0.2deg" = "02",
              "0.4deg" = "04",
              "1deg" = "10")

# CEREAL CROPS TO KEEP #####

grain_targets <- c("wheat", "rice", "maize", "barley", "millet", "sorghum")
grain_aliases <- c("othercereals", "cerealnes")

# RUN FOR ALL THREE RESOLUTIONS #####

template_files <- list(
  "0.2deg" = "mha_stack_ll_02_aligned_NEAR.tif",
  "0.4deg" = "mha_stack_ll_04_aligned_NEAR.tif",
  "1deg" = "mha_stack_ll_10_aligned_NEAR.tif")

results_regions_all_earthstat <- rbindlist(
  lapply(names(earthstat_dirs), function(res_label) {

    folder <- earthstat_dirs[[res_label]]
    res_tag <- res_tags[[res_label]]

    grain_dt <- build_cereal_sum_table(
      folder = folder,
```

```

    res_tag = res_tag,
    grain_targets = grain_targets,
    grain_aliases = grain_aliases
  )

  template_r <- rast(template_files[[res_label]])[[1]]
  values(template_r) <- NA_real_

  out <- compute_breadbaskets_earthstat(
    res_tag = res_label,
    grain_prod_dt = grain_dt,
    dt_pres = dt_pres,
    regions_dt = regions_dt,
    template_r = template_r
  )

  rm(grain_dt, template_r)
  gc()

  out
}),
use.names = TRUE,
fill = TRUE
)

## Using crops for 02: wheat, rice, maize, barley, millet, sorghum, cerealnes
## Using crops for 04: wheat, rice, maize, barley, millet, sorghum, cerealnes
## Using crops for 10: wheat, rice, maize, barley, millet, sorghum, cerealnes
# SAVE RESULTS #####

dir.create("./datasets/earthstat_regridded", recursive = TRUE, showWarnings = FALSE)

fwrite(results_regions_all_earthstat,
       "./datasets/earthstat_regridded/earthstat_breadbasket_grain_losses.csv")

# SUMMARY STATISTICS #####

print(results_regions_all_earthstat)

```

```

##           region resolution      k total_prod frac_vs_1  loss_vs_1
##           <char>      <char> <int>      <num>      <num>      <num>
##    1: Indo-Gangetic Plain    0.2deg    10  112799073 0.5417889 0.45821109
##    2: Indo-Gangetic Plain    0.2deg     9   188874014 0.9071869 0.09281306
##    3: Indo-Gangetic Plain    0.2deg     8   195009509 0.9366565 0.06334346
##    4: Indo-Gangetic Plain    0.2deg     7   198375705 0.9528248 0.04717518
##    5: Indo-Gangetic Plain    0.2deg     6   200251950 0.9618367 0.03816332

```

```
## ---
## 176: Mediterranean Basin      1deg      5    90599945 0.8922145 0.10778553
## 177: Mediterranean Basin      1deg      4    93932840 0.9250363 0.07496369
## 178: Mediterranean Basin      1deg      3    98418519 0.9692106 0.03078940
## 179: Mediterranean Basin      1deg      2    99155152 0.9764649 0.02353515
## 180: Mediterranean Basin      1deg      1   101545030 1.0000000 0.00000000
##      frac_vs_5  loss_vs_5
##      <num>      <num>
## 1: 0.5580268 0.441973153
## 2: 0.9343762 0.065623786
## 3: 0.9647290 0.035270958
## 4: 0.9813819 0.018618094
## 5: 0.9906639 0.009336147
## ---
## 176: 1.0000000 0.000000000
## 177:      NA      NA
## 178:      NA      NA
## 179:      NA      NA
## 180:      NA      NA
```

```
tmp_summary <- results_regions_all_earthstat[
  , .(
    min_frac_5 = min(frac_vs_5, na.rm = TRUE),
    max_frac_5 = max(frac_vs_5, na.rm = TRUE),
    min_loss_5 = min(loss_vs_5, na.rm = TRUE),
    max_loss_5 = max(loss_vs_5, na.rm = TRUE),
    min_frac_1 = min(frac_vs_1, na.rm = TRUE),
    max_frac_1 = max(frac_vs_1, na.rm = TRUE),
    min_loss_1 = min(loss_vs_1, na.rm = TRUE),
    max_loss_1 = max(loss_vs_1, na.rm = TRUE)
  ),
  by = .(region, k)
] %>%
  .[k %in% 5:10]

tmp_summary[k %in% 9:10 &
  region %in% c("Nile Delta/Basin", "Indo-Gangetic Plain",
    "Mekong Delta"),
  .(region, min_loss_1, max_loss_1)]
```

```
##      region min_loss_1 max_loss_1
##      <char>      <num>      <num>
## 1: Indo-Gangetic Plain 0.05489196 0.45821109
## 2: Indo-Gangetic Plain 0.02820065 0.09281306
## 3: Nile Delta/Basin 0.00265116 0.34278082
## 4: Nile Delta/Basin 0.00265116 0.03917718
## 5: Mekong Delta 0.05610146 0.47265794
## 6: Mekong Delta 0.00000000 0.12154759
```

```

# PLOT: LOSS RELATIVE TO k >= 1 #####

plot_dt <- copy(results_regions_all_earthstat)

# Keep only the main metric
plot_dt <- plot_dt[!is.na(loss_vs_1)]

# Order regions
region_levels <- c(
  "Indo-Gangetic Plain",
  "North China Plain",
  "US High Plains",
  "Nile Delta/Basin",
  "Mekong Delta",
  "Mediterranean Basin"
)

plot_dt[, region := factor(region, levels = region_levels)]

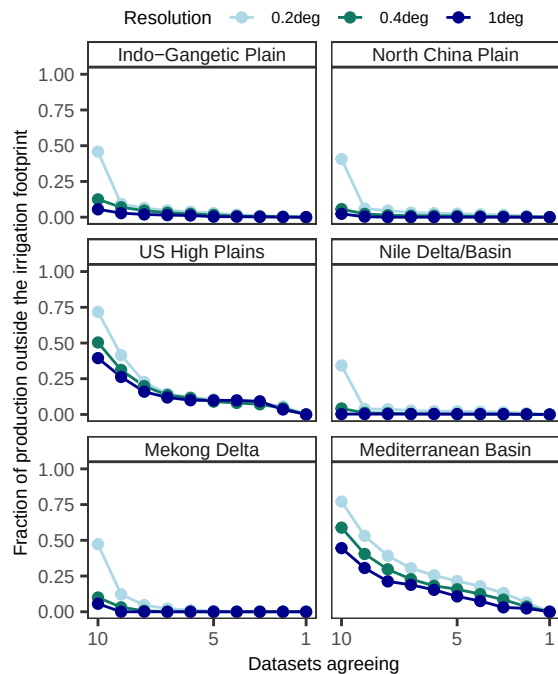
# Order resolutions
plot_dt[, resolution := factor(resolution, levels = c("0.2deg", "0.4deg", "1deg"))]

# Keep k numeric so reverse scale works cleanly
plot_dt[, k := as.integer(k)]

plot_breadbaskets <- ggplot(plot_dt, aes(k, loss_vs_1, colour = resolution,
                                         group = resolution)) +
  geom_line() +
  geom_point() +
  facet_wrap(~region, ncol = 2) +
  scale_x_reverse(breaks = c(10, 5, 1),
                 limits = c(10, 1)) +
  scale_y_continuous(limits = c(0, 1),
                    breaks = c(0, 0.25, 0.5, 0.75, 1)) +
  labs(x = "Datasets agreeing", y = "Fraction of production outside the irrigation footprint",
       colour = "Resolution") +
  scale_color_manual(values = res_palette) +
  theme_AP() +
  theme(legend.position = "top")

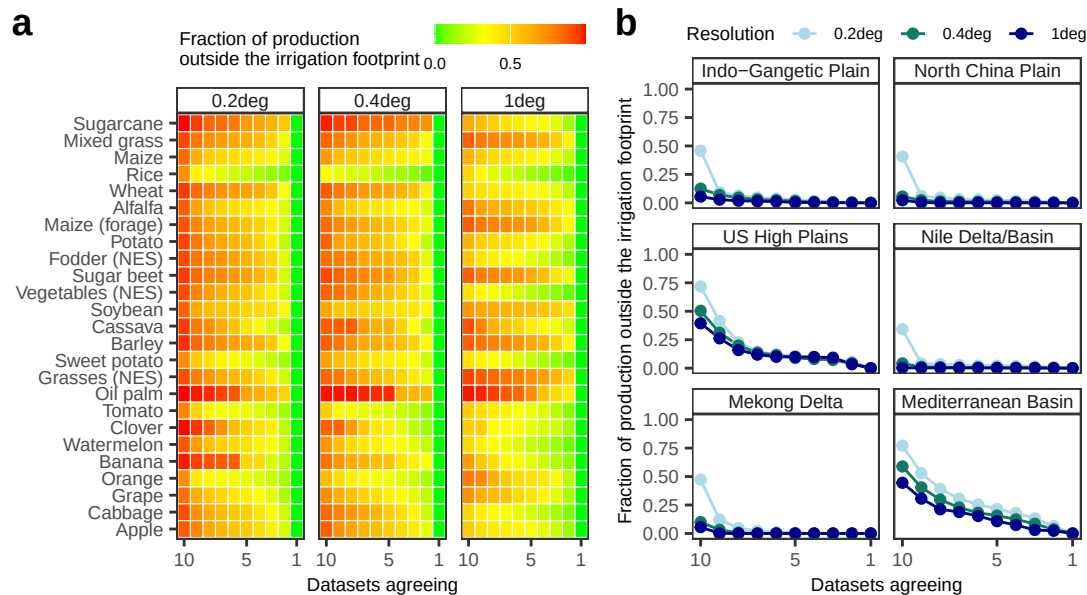
plot_breadbaskets

```



```
top2 <- plot_grid(plot_heatmap, plot_breadbaskets, ncol = 2, rel_widths = c(0.55, 0.45),
  labels = "auto")
```

top2



5.4 Plot global

```
# LOAD GLOBAL RESULTS #####
```

```
files <- c(
  "./datasets/earthstat_regridded/res_02/earthstat_global_production_k_agreement_02.csv",
```

```

    "./datasets/earthstat_regridded/res_04/earthstat_global_production_k_agreement_04.csv",
    "./datasets/earthstat_regridded/res_10/earthstat_global_production_k_agreement_10.csv"
  )

global_dt <- rbindlist(lapply(files, fread), use.names = TRUE, fill = TRUE)

# KEEP k = 10:1 #####

global_dt <- global_dt[k %in% 1:10]

# compute loss relative to k >= 1
global_dt[, loss_vs_1 := 1 - share_of_k1]

# order resolution
global_dt[, resolution := factor(resolution, levels = c("0.2deg", "0.4deg", "1deg"))]

# Print
global_dt[k %in% c(1, 5, 10), .(k, resolution, loss_vs_1)]

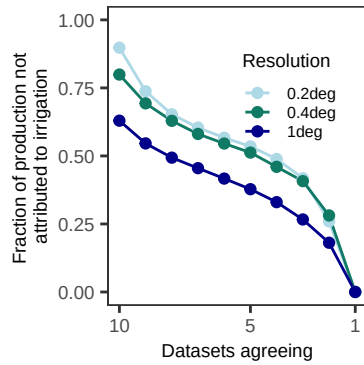
##           k resolution loss_vs_1
##    <int>      <fctr>      <num>
## 1:      1      0.2deg 0.0000000
## 2:      5      0.2deg 0.5349922
## 3:     10      0.2deg 0.8976727
## 4:      1      0.4deg 0.0000000
## 5:      5      0.4deg 0.5122408
## 6:     10      0.4deg 0.7990398
## 7:      1       1deg 0.0000000
## 8:      5       1deg 0.3777311
## 9:     10       1deg 0.6295084

# PLOT #####

plot_global <- ggplot(global_dt, aes(k, loss_vs_1, colour = resolution, group = resolution)) +
  geom_line() +
  geom_point() +
  scale_x_reverse(breaks = c(10, 5, 1),
                  limits = c(10, 1)) +
  scale_y_continuous(limits = c(0, 1),
                     breaks = c(0, 0.25, 0.5, 0.75, 1)) +
  scale_color_manual(values = res_palette) +
  labs(x = "Datasets agreeing", y = "Fraction of production not \nattributed to irrigation",
       colour = "Resolution") +
  theme_AP() +
  theme(
    legend.position = c(0.7, 0.7))

plot_global

```

6 Consequences for irrigation-induced evapotranspiration

6.1 Extract CESM data

```
# LOAD THE HARMONIZED GRIDS #####

tmpl_02 <- rast("./mha_stack_ll_02_aligned_NEAR.tif")[[1]]
tmpl_04 <- rast("./mha_stack_ll_04_aligned_NEAR.tif")[[1]]
tmpl_1 <- rast("./mha_stack_ll_10_aligned_NEAR.tif")[[1]]

templates <- list("0.2deg" = tmpl_02, "0.4deg" = tmpl_04, "1deg" = tmpl_1)

# Enforce lon/lat WGS84 on templates if needed-----

templates <- lapply(templates, function(t) {
  if (is.na(crs(t)) || crs(t) == "") crs(t) <- "EPSG:4326"
  t
})

# LOAD THE FILES #####
# Files come from the Yao et al 2025 paper (Nature Water)

# Year window -----

win <- "1985_2014_timmean"

# Base directory -----

base_dir <- "./yao_et_al_2025_datasets/water_fluxes/CESM2"

# Paste -----

irr_files <- paste0(base_dir, "/CESM2_IRR0", 1:3, "_1901_2014_QFLX_EVAP_TOT_yearmean_", win)
noi_files <- paste0(base_dir, "/CESM2_NOI0", 1:3, "_1901_2014_QFLX_EVAP_TOT_yearmean_", win)

# BUILD ET STACKS + ENSEMBLE MEAN PER RESOLUTION #####
```

```

d_list <- Map(function(irr_f, noi_f) delta_pair_resampled(irr_f, noi_f, templates),
              irr_files, noi_files)

# d_list length 3; each element is list("0.2deg"=rast, "0.4deg"=rast, "1deg"=rast)
dET_mean_rasters <- lapply(names(templates), function(res_name) {

  d_rasters <- lapply(d_list, function(one_pair) one_pair[[res_name]])
  s <- terra::rast(d_rasters) # 3-layer stack
  names(s) <- paste0("dET_IRRO", 1:3, "_mmday")

  # ensemble mean: cell-by-cell mean across 3 members
  m <- terra::app(s, fun = mean, na.rm = TRUE)
  names(m) <- paste0("CESM2_dET_mean_", win, "_", res_name, "_mmday")

  list(stack = s, mean = m)
})
names(dET_mean_rasters) <- names(templates)

# EXPORT TO DATA TABLE #####

dET_dt <- lapply(names(templates), function(res_name) {
  tml <- templates[[res_name]]
  out <- dET_mean_rasters[[res_name]]

  dt_mean <- rast_to_dt_with_cell(out$mean, tml, value_name = "dET_mean_mmday")
  dt_mean[, resolution:= res_name]

  dt_members <- rbindlist(lapply(1:terra::nlyr(out$stack), function(i) {
    rr <- out$stack[[i]]
    dt <- rast_to_dt_with_cell(rr, tml, value_name = "dET_mmday")
    dt[, member:= names(rr)] # dET_IRRO1_mmday / dET_IRRO2_mmday / dET_IRRO3_mmday
    dt[, resolution:= res_name]
    dt[]
  })))

  list(mean = dt_mean, members = dt_members)
})
names(dET_dt) <- names(templates)

# COMPUTE K-OF-10 AGREEMENT WITH CELL JOINS #####

resolutions <- c("0.2deg", "0.4deg", "1deg")
ks <- 1:10

# Compute -----

out_k_all <- rbindlist(lapply(resolutions, function(res) {

```

```

tpl <- templates[[res]]

# irrigation presence-----

dt_irrig_raw <- copy(dt_pres)[resolution == res]
dt_irrig <- pres_to_cell(dt_irrig_raw, tpl)

# add cell area-----

dt_area <- cell_area_dt(tpl)
setkey(dt_irrig, cell)
setkey(dt_area, cell)
dt_irrig <- dt_irrig[dt_area] # add area_m2

# increase ET mean table -----

dt_et <- copy(dET_dt[[res]]$mean)
valcol <- setdiff(names(dt_et), c("cell", "lon", "lat", "resolution"))[1]

# Join: we keep all ET cells and bring irrigation info when available:
# to summarise ONLY cells that exist in the irrigation table:
# dt_join <- dt_et[dt_irrig, on="cell"]
setkey(dt_et, cell)
dt_join <- dt_irrig[dt_et]

rbindlist(lapply(ks, function(k) {

  dt_sub <- dt_join[n_pos >= k]

  # restrict weights to cells with non-missing ET and non-missing area
  ok <- !is.na(dt_sub[[valcol]]) & !is.na(dt_sub$area_m2)
  w <- dt_sub$area_m2[ok]
  x <- dt_sub[[valcol]][ok]

  # area-weighted mean increase ET (mm/day)
  mean_aw <- if (length(x) == 0L) NA_real_ else sum(x * w) / sum(w)

  # area-weighted aggregate: mm/day * m^2 -> m^3/day
  # 1 mm = 0.001 m
  sum_m3day <- if (length(x) == 0L) NA_real_ else sum(x * w) * 1e-3

  data.table(resolution = res,
             k = k,
             n_cells_irrig_k = sum(dt_irrig$n_pos >= k, na.rm = TRUE),
             n_cells_join_k = nrow(dt_sub),
             n_cells_with_dET_k = sum(!is.na(dt_sub[[valcol]])),
             mean_dET_mmday = mean(dt_sub[[valcol]], na.rm = TRUE),

```

```

        sum_dET_mmday_unweighted = sum(dt_sub[[valcol]], na.rm = TRUE),
        mean_dET_mmday_areaweighted = mean_aw,
        sum_dET_m3day_areaweighted = sum_m3day,
        area_m2_with_dET_k = if (length(w) == 0L) 0 else sum(w)
    )
  })
})

out_k_all

```

##	resolution	k	n_cells_irrig_k	n_cells_join_k	n_cells_with_dET_k
##	<char>	<int>	<int>	<int>	<int>
## 1:	0.2deg	1	86551	86551	86146
## 2:	0.2deg	2	65026	65026	64695
## 3:	0.2deg	3	56056	56056	55792
## 4:	0.2deg	4	51038	51038	50806
## 5:	0.2deg	5	46920	46920	46723
## 6:	0.2deg	6	42296	42296	42144
## 7:	0.2deg	7	36510	36510	36396
## 8:	0.2deg	8	29235	29235	29158
## 9:	0.2deg	9	21006	21006	20964
## 10:	0.2deg	10	9590	9590	9575
## 11:	0.4deg	1	25244	25244	25135
## 12:	0.4deg	2	19367	19367	19283
## 13:	0.4deg	3	16881	16881	16807
## 14:	0.4deg	4	15488	15488	15422
## 15:	0.4deg	5	14369	14369	14312
## 16:	0.4deg	6	13251	13251	13200
## 17:	0.4deg	7	11715	11715	11675
## 18:	0.4deg	8	9910	9910	9878
## 19:	0.4deg	9	7735	7735	7718
## 20:	0.4deg	10	5118	5118	5112
## 21:	1deg	1	4595	4595	4582
## 22:	1deg	2	3673	3673	3663
## 23:	1deg	3	3226	3226	3220
## 24:	1deg	4	2988	2988	2983
## 25:	1deg	5	2816	2816	2811
## 26:	1deg	6	2618	2618	2613
## 27:	1deg	7	2396	2396	2391
## 28:	1deg	8	2133	2133	2129
## 29:	1deg	9	1772	1772	1769
## 30:	1deg	10	1298	1298	1297
##	resolution	k	n_cells_irrig_k	n_cells_join_k	n_cells_with_dET_k
##	<char>	<int>	<int>	<int>	<int>
##	mean_dET_mmday	sum_dET_mmday_unweighted	mean_dET_mmday_areaweighted		
##	<num>		<num>		<num>
## 1:	0.02529852		2179.36620		0.02558831

## 2:	0.03176582	2055.09005	0.03191648
## 3:	0.03538052	1973.95005	0.03538168
## 4:	0.03776344	1918.60923	0.03770233
## 5:	0.04012408	1874.71758	0.04011012
## 6:	0.04332301	1825.80479	0.04337120
## 7:	0.04821179	1754.71632	0.04832421
## 8:	0.05598796	1632.49706	0.05619227
## 9:	0.06818188	1429.36490	0.06853504
## 10:	0.08271255	791.97271	0.08345616
## 11:	0.02324782	584.33387	0.02345943
## 12:	0.02916753	562.43750	0.02922414
## 13:	0.03254787	547.03199	0.03244195
## 14:	0.03477842	536.35276	0.03462010
## 15:	0.03678079	526.40665	0.03666735
## 16:	0.03922764	517.80487	0.03914187
## 17:	0.04295393	501.48715	0.04292040
## 18:	0.04846748	478.76176	0.04850110
## 19:	0.05689614	439.12441	0.05710005
## 20:	0.07209324	368.54062	0.07260780
## 21:	0.02146453	98.35049	0.02162021
## 22:	0.02635598	96.54197	0.02637286
## 23:	0.02973174	95.73620	0.02956890
## 24:	0.03164255	94.38972	0.03144091
## 25:	0.03344324	94.00895	0.03322915
## 26:	0.03568753	93.25152	0.03550908
## 27:	0.03845227	91.93937	0.03830816
## 28:	0.04232406	90.10792	0.04219702
## 29:	0.04798235	84.88078	0.04799651
## 30:	0.05863885	76.05459	0.05891090
##	mean_dET_mmday	sum_dET_mmday_unweighted	mean_dET_mmday_areaweighted
##	<num>	<num>	<num>
##	sum_dET_m3day_areaweighted	area_m2_with_dET_k	
##	<num>	<num>	
## 1:	897374704	3.506972e+13	
## 2:	849120561	2.660446e+13	
## 3:	818354103	2.312931e+13	
## 4:	797228362	2.114534e+13	
## 5:	780279411	1.945343e+13	
## 6:	761085387	1.754817e+13	
## 7:	733047410	1.516936e+13	
## 8:	683881025	1.217038e+13	
## 9:	601363946	8.774547e+12	
## 10:	333696792	3.998468e+12	
## 11:	961188517	4.097238e+13	
## 12:	926238757	3.169431e+13	
## 13:	902973514	2.783352e+13	
## 14:	887304997	2.562976e+13	
## 15:	872410097	2.379256e+13	

```
## 16:                859248031        2.195214e+13
## 17:                833991565        1.943112e+13
## 18:                798030563        1.645387e+13
## 19:                734575092        1.286470e+13
## 20:                618966902        8.524799e+12
## 21:               1008992705        4.666896e+13
## 22:                990272480        3.754893e+13
## 23:                982727869        3.323518e+13
## 24:                970732988        3.087483e+13
## 25:                968022615        2.913173e+13
## 26:                961187451        2.706878e+13
## 27:                949610682        2.478873e+13
## 28:                932414251        2.209669e+13
## 29:                880927203        1.835398e+13
## 30:                792833208        1.345818e+13
##      sum_dET_m3day_areaweighted area_m2_with_dET_k
##                                <num>          <num>

# PLOT #####

# Condition ifesle -----

total_col <- if ("sum_dET_m3day_areaweighted" %in% names(out_k_all)) {
  "sum_dET_m3day_areaweighted"
} else if ("sum_dET_mmday_unweighted" %in% names(out_k_all)) {
  "sum_dET_mmday_unweighted"
} else {
  stop("out_k_all has neither sum_dET_m3day_areaweighted nor sum_dET_mmday_unweighted")
}

# Convert m3/day to km3/day -----

out_plot <- copy(out_k_all)

if (total_col == "sum_dET_m3day_areaweighted") {
  out_plot[, total_km3day := get(total_col) / 1e9]
  total_ylab <- "Total irrigation-\ninduced ET (km3 d-1)"
} else {
  out_plot[, total_km3day := get(total_col)]
  total_ylab <- "Total irrigation-induced \nET signal (sum of mm/day)"
}

# Keep only k = 5, ..., 10-----

out_plot <- out_plot[k %in% 1:10]
```

```

# Plot mean increase in irrigation-induced ET -----
plot_mean_ET <- ggplot(out_plot, aes(k, mean_dET_mmday, colour = resolution,
                                     group = resolution)) +
  geom_line() +
  geom_point() +
  labs(x = "Datasets agreeing",
       y = expression(atop("Mean " * Delta * ET[irr], "(mm d"^{-1} * ")")) +
  scale_x_reverse(breaks = c(10, 5, 1)) +
  scale_color_manual(values = res_palette) +
  theme_AP() +
  theme(legend.position = "none")

# Plot total irrigation induced ET -----
plot_total_ET <- ggplot(out_plot, aes(k, total_km3day, colour = resolution, group = resolution)) +
  geom_line() +
  geom_point() +
  labs(x = "Datasets agreeing", y = total_ylab) +
  scale_x_reverse(breaks = c(10, 5, 1)) +
  scale_color_manual(values = res_palette) +
  theme_AP() +
  theme(legend.position = "none")

# Plot area with irrigation-induced ET retained under k-----
out_plot <- copy(out_k_all)[k %in% 1:10]

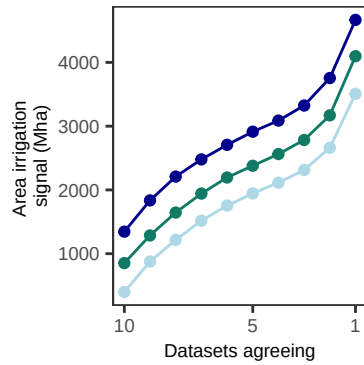
# convert m2 to Mha -----
out_plot[, area_Mha:= area_m2_with_dET_k / 1e10]

# also show contraction relative to k=5 within each resolution
out_plot[, area_frac_vs_k5:= area_m2_with_dET_k / area_m2_with_dET_k[k == 5], by = resolution]

# Total land area of grid cells with reported irrigated areas
plot_area <- ggplot(out_plot, aes(k, area_Mha, colour = resolution, group = resolution)) +
  geom_line() +
  geom_point() +
  labs(x = "Datasets agreeing", y = "Area irrigation \nsignal (Mha)") +
  scale_x_reverse(breaks = c(10, 5, 1)) +
  scale_color_manual(values = res_palette) +
  theme_AP() +
  theme(legend.position = "none")

plot_area

```

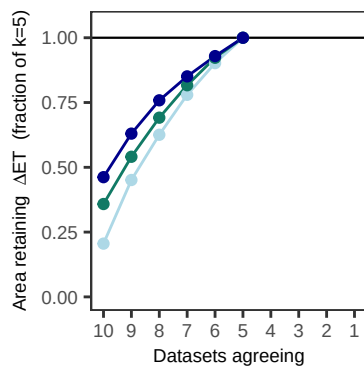


```
# Also in case: fraction of area retained vs k=5
plot_area_fraction <- ggplot(out_plot, aes(k, area_frac_vs_k5, colour = resolution, group = res)) +
  geom_hline(yintercept = 1, linewidth = 0.4) +
  geom_line() +
  geom_point() +
  labs(x = "Datasets agreeing",
       y = expression("Area retaining "~Delta*ET~" (fraction of k=5)")) +
  scale_x_reverse(breaks = 10:1) +
  scale_y_continuous(limits = c(0, 1.05)) +
  scale_color_manual(values = res_palette) +
  theme_AP() +
  theme(legend.position = "none")
```

```
plot_area_fraction
```

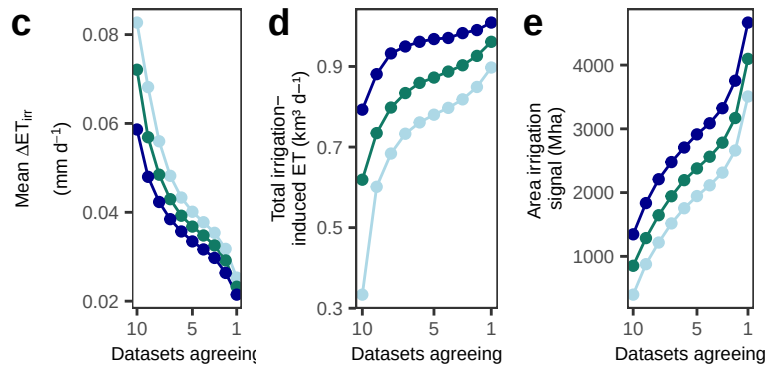
```
## Warning: Removed 12 rows containing missing values or values outside the scale range
## (`geom_line()`).
```

```
## Warning: Removed 12 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

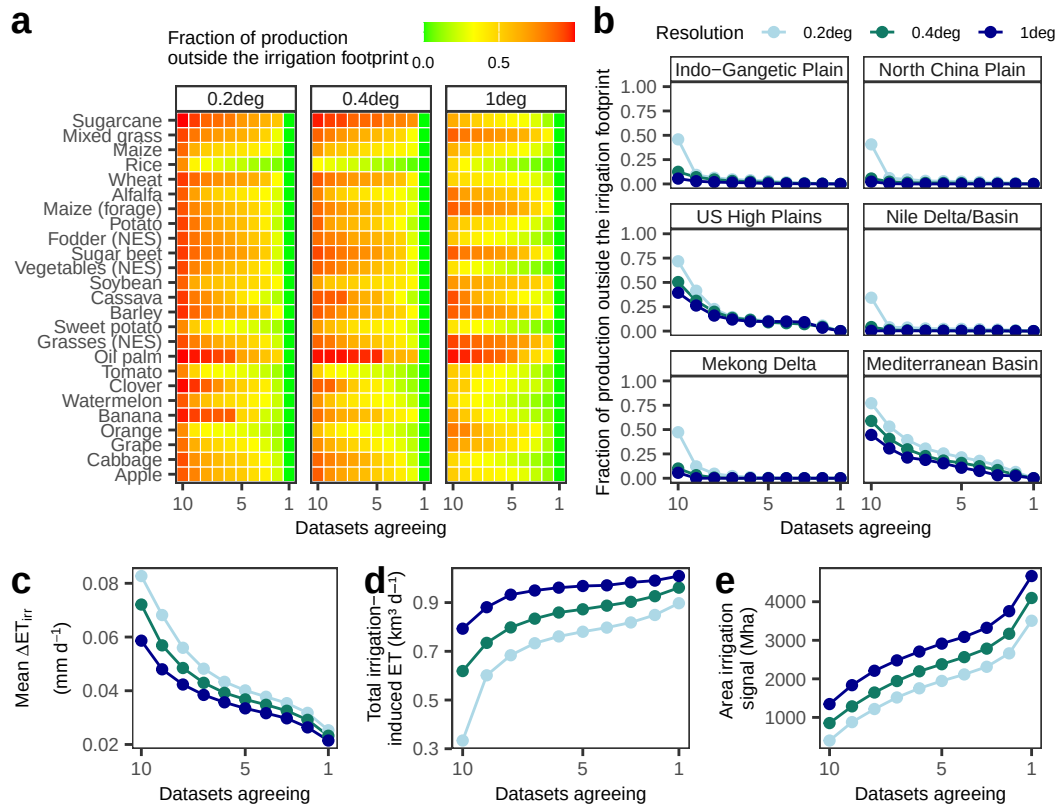


6.2 Merge figures

```
bottom <- plot_grid(plot_mean_ET, plot_total_ET, plot_area, labels = c("c", "d", "e"),
                    ncol = 3)
bottom
```

```
plot_grid(top2, bottom, ncol = 1, rel_heights = c(0.685, 0.325))
```



7 Final figures

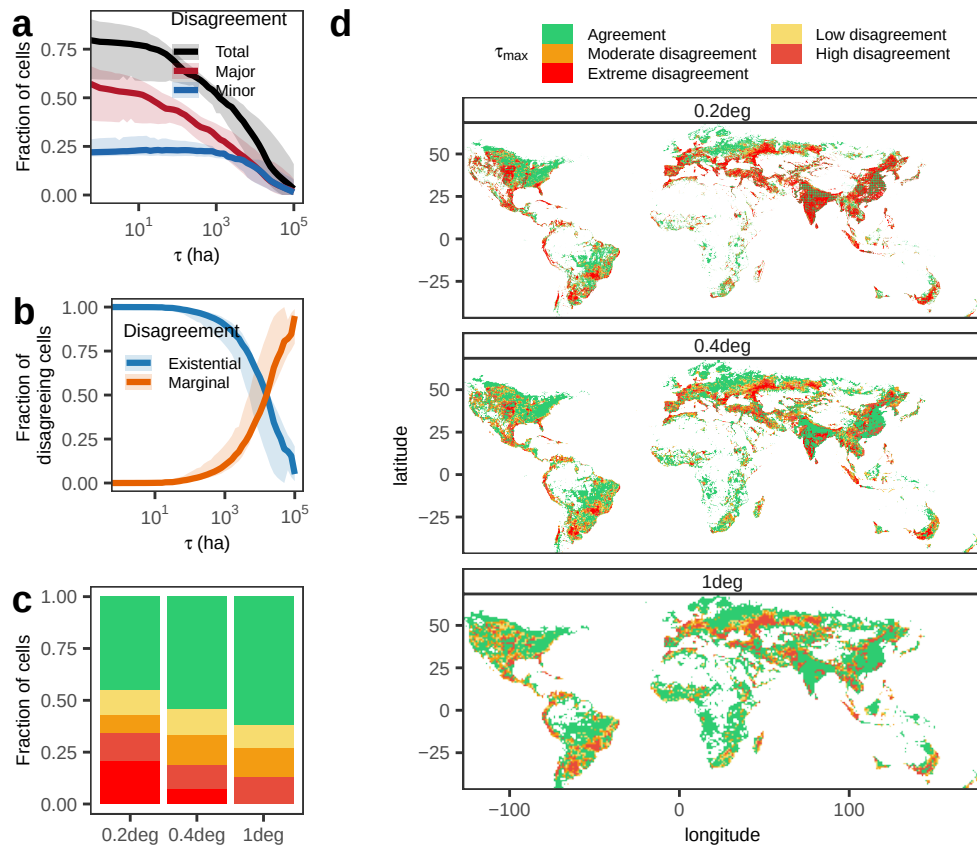
```
left <- plot_grid(plot_uncertainty1, plot_agreement1,
  plot_stacked +
  labs(y = "Fraction of cells"), ncol = 1, labels = "auto")
```

```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.
## log-10 transformation introduced infinite values.
## log-10 transformation introduced infinite values.
```

```
plot_grid(left, plot_raster, ncol = 2, rel_widths = c(0.3, 0.7), labels = c("", "d"))
```

```
## Warning: Raster pixels are placed at uneven horizontal intervals and will be shifted
## i Consider using `geom_tile()` instead.
```

```
## Warning: Raster pixels are placed at uneven horizontal intervals and will be shifted
## i Consider using `geom_tile()` instead.
```



```
left <- plot_grid(plot_uncertainty1, plot_agreement1,
  plot_regime, ncol = 1, labels = c("a", "b", "d"))
```

```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced :
## log-10 transformation introduced infinite values.
## log-10 transformation introduced infinite values.
## log-10 transformation introduced infinite values.
```

```

middle <- plot_grid(p1 + facet_wrap(~resolution, ncol = 1),
  plot_stacked +
    labs(y = "Fraction of cells"),
  ncol = 1, rel_heights = c(0.7, 0.3), labels = c("c", "e"))
plot_grid(left, middle, plot_raster, ncol = 3, rel_widths = c(0.26, 0.26, 0.48), labels = c("a", "b", "f"))

```

```

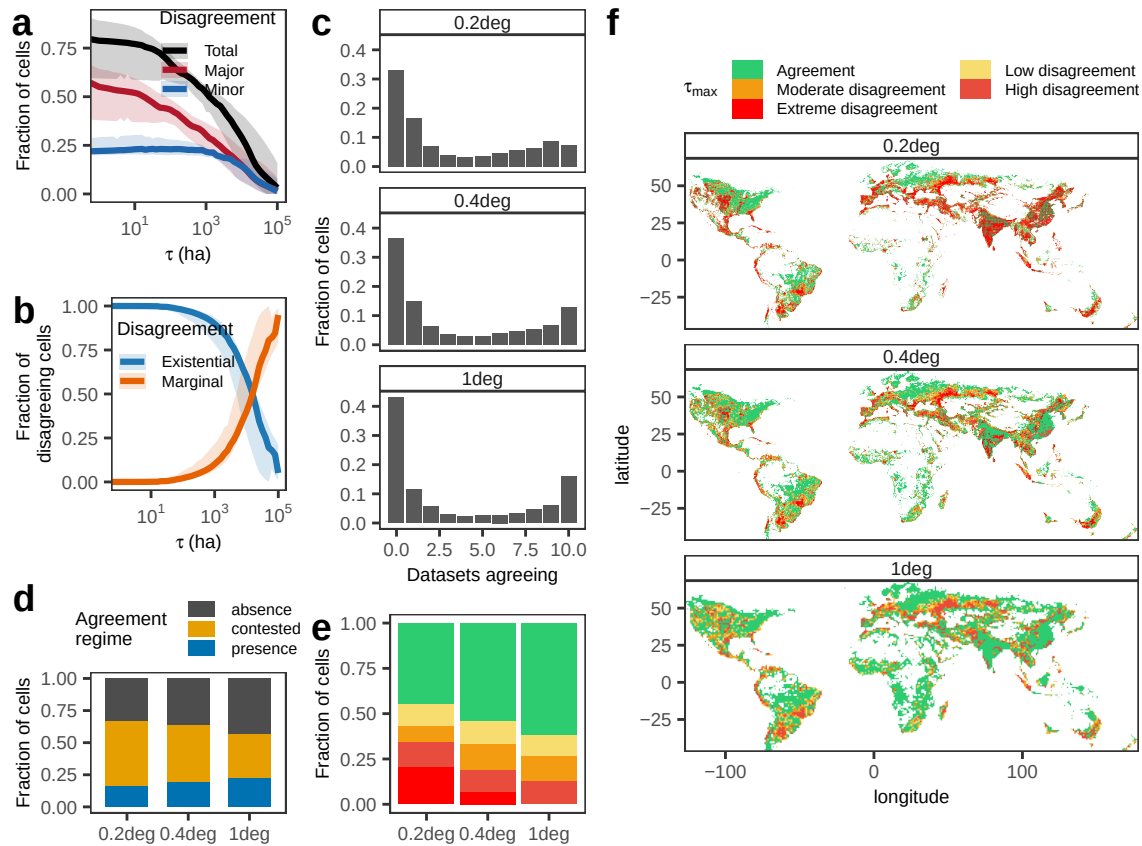
## Warning: Raster pixels are placed at uneven horizontal intervals and will be shifted
## i Consider using `geom_tile()` instead.

```

```

## Warning: Raster pixels are placed at uneven horizontal intervals and will be shifted
## i Consider using `geom_tile()` instead.

```



```

top <- plot_grid(plot_indices1 +
  labs(x = "", y = "Fraction \n variance") +
  scale_fill_discrete(name = "Sobol' \n indices") +
  theme(legend.position = c(0.4, 0.7)),
  a1 + labs(x = "exclusion", y = "Fraction \n disagree"),
  b1, c1, ncol = 4, labels = c("a", "", "", ""),
  rel_widths = c(0.23, 0.37, 0.2, 0.2))

```

```

## Scale for fill is already present.

```

```

## Adding another scale for fill, which will replace the existing scale.

```

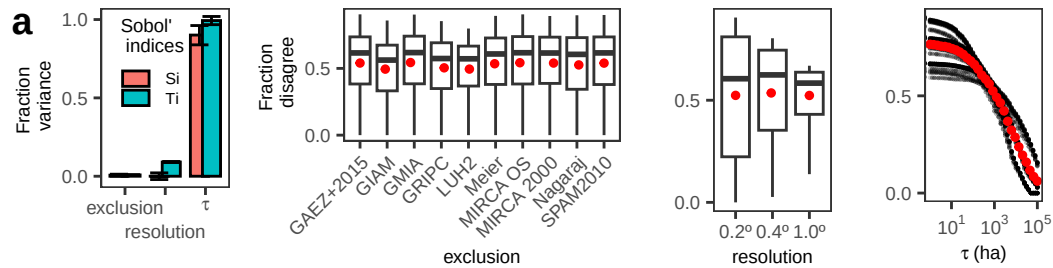
```

## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation introduced
## log-10 transformation introduced infinite values.

```

```
## Warning: Removed 157 rows containing non-finite outside the scale range
## (`stat_summary_bin()`).
```

top



```
middle <- plot_grid(plot_indices2 +
  labs(x = "", y = "Fraction \n variance") +
  scale_fill_discrete(name = "") +
  theme(legend.position = "none"),
  a2 + labs(x = "exclusion", y = "Fraction \n disagree"),
  b2, c2, ncol = 4, labels = c("b", "", "", ""),
  rel_widths = c(0.23, 0.37, 0.2, 0.2))
```

```
## Scale for fill is already present.
```

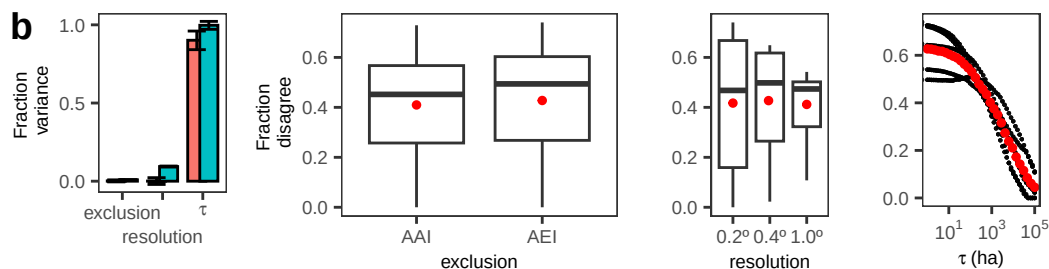
```
## Adding another scale for fill, which will replace the existing scale.
```

```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation
## introduced infinite values.
```

```
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation
## introduced infinite values.
```

```
## Warning: Removed 157 rows containing non-finite outside the scale range
## (`stat_summary_bin()`).
```

middle



```
bottom <- plot_grid(plot_indices3 +
  labs(x = "", y = "Fraction \n variance") +
  scale_fill_discrete(name = "") +
  theme(legend.position = "none"),
  a3 + labs(x = "exclusion", y = "Fraction \n disagree"),
  b3, c3, d3, ncol = 5, labels = c("c", "", "", "", ""),
  rel_widths = c(0.22, 0.34, 0.16, 0.15, 0.15))
```

```
## Scale for fill is already present.
```

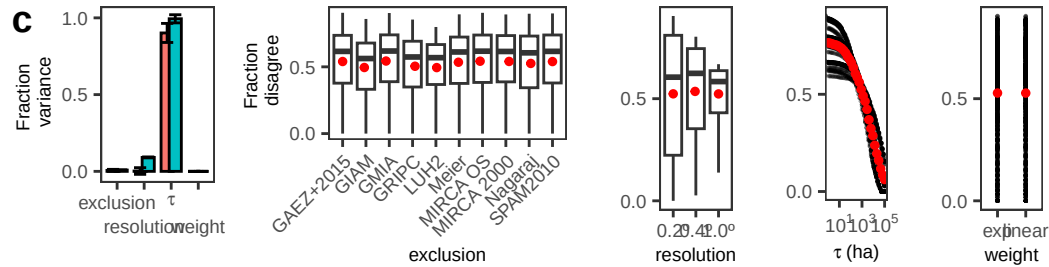
```
## Adding another scale for fill, which will replace the existing scale.

## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation
## introduced infinite values.

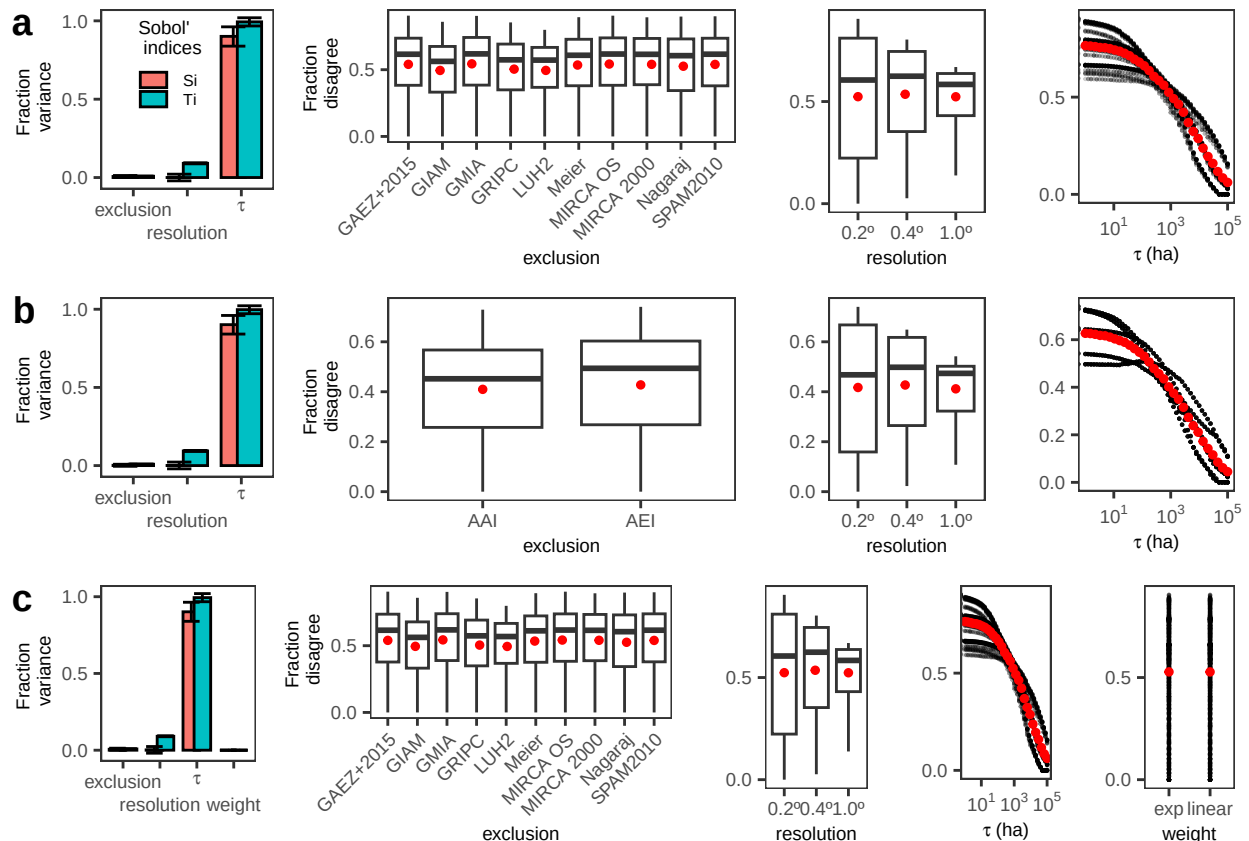
## Warning in scale_x_log10(labels = label_log(digits = 2)): log-10 transformation
## introduced infinite values.

## Warning: Removed 157 rows containing non-finite outside the scale range
## (`stat_summary_bin()`).
```

bottom



```
plot_grid(top, middle, bottom, ncol = 1)
```



```
dt_pres[, classification := fifelse(
  n_pos == 0, "absence",
  fifelse(n_pos <= 8, "contested", "presence")
)]
```

```

plot_classification <- ggplot(dt_pres, aes(lon, lat, fill = classification)) +
  geom_raster() +
  coord_equal(expand = FALSE) +
  facet_wrap(~ resolution, ncol = 1) +
  scale_fill_manual(values = c(absence = "#4D4D4D",
                               contested = "#E69F00",
                               presence = "#0072B2")) +
  scale_x_continuous(expand = c(0, 0)) +
  scale_y_continuous(expand = c(0, 0)) +
  theme_AP() +
  theme(panel.grid = element_blank(),
        strip.text = element_text(size = 11),
        legend.position = "top") +
  labs(x = "longitude", y = "latitude") +
  guides(fill = guide_legend(nrow = 3, byrow = TRUE))

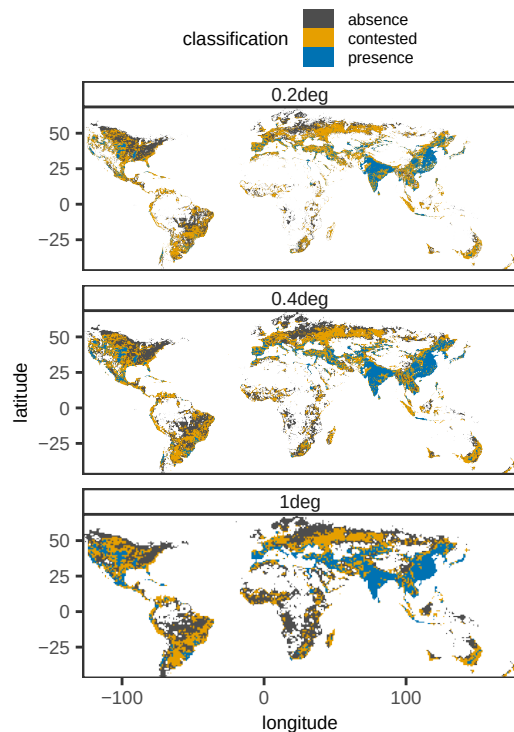
```

plot_classification

```

## Warning: Raster pixels are placed at uneven horizontal intervals and will be shifted
## i Consider using `geom_tile()` instead.
## Raster pixels are placed at uneven horizontal intervals and will be shifted
## i Consider using `geom_tile()` instead.

```



```

plot_grid(plot_raster, plot_classification, ncol = 2, labels = "auto")

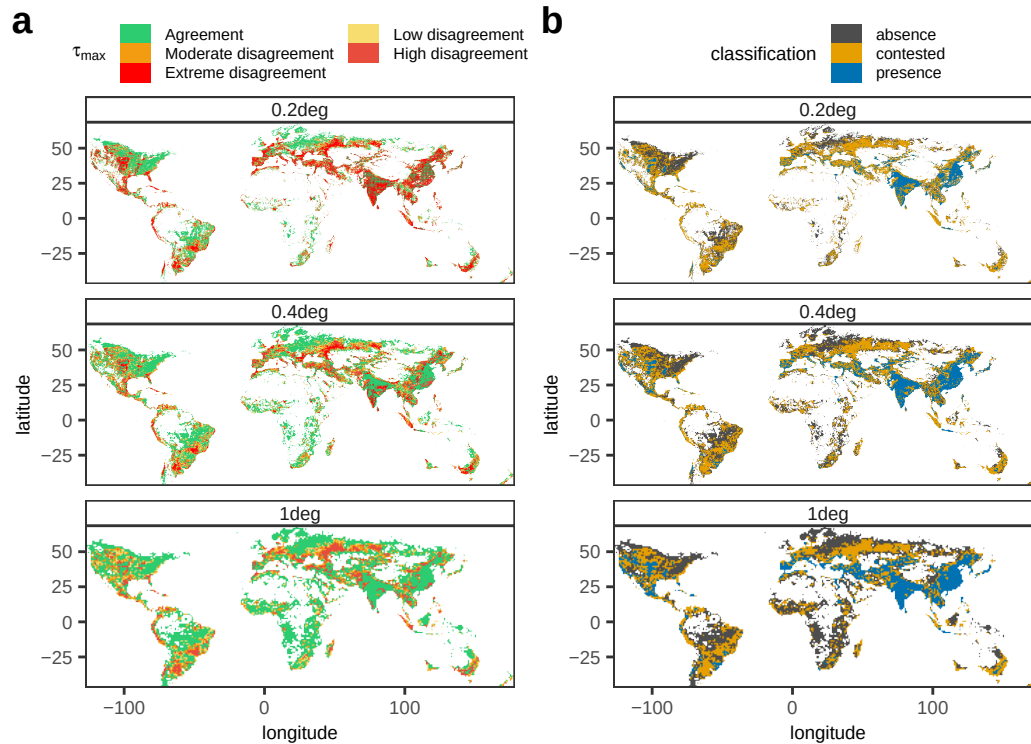
```

```

## Warning: Raster pixels are placed at uneven horizontal intervals and will be shifted
## i Consider using `geom_tile()` instead.

```

```
## Raster pixels are placed at uneven horizontal intervals and will be shifted
## i Consider using `geom_tile()` instead.
## Raster pixels are placed at uneven horizontal intervals and will be shifted
## i Consider using `geom_tile()` instead.
## Raster pixels are placed at uneven horizontal intervals and will be shifted
## i Consider using `geom_tile()` instead.
```



8 Is disagreement caused by different irrigation conceptualizations?

```
# STUDY OF DEFINITION HETEROGENEITY #####
#####

# Read the survey spreadsheet -----

dt_definitions <- as.data.table(readxl::read_excel("dataset_survey_definitions.xlsx"))

setnames(dt_definitions, c("dataset", "citation_full", "paddy_rice_raw",
                           "temporal_raw", "justification", "irrigation_target"))

# Homogenise paddy rice treatment -----

dt_definitions[, paddy_rice:= fcase(grepl("^Included", paddy_rice_raw), "included",
                                     grepl("^Separated", paddy_rice_raw), "separated",
                                     grepl("^Unclear", paddy_rice_raw), "unclear")]
```

```

# Homogenise temporal concept -----

dt_definitions[, temporal:= fcase(grepl("^Annual", temporal_raw), "annual",
                                   grepl("^5-year", temporal_raw), "multi_year",
                                   grepl("^Reference", temporal_raw), "reference_period",
                                   grepl("^Seasonal", temporal_raw), "seasonal",
                                   grepl("^Unclear", temporal_raw), "unclear")]

# Keep clean columns -----

dt_clean <- dt_definitions[, .(dataset, paddy_rice, temporal,
                               irrigation_target, justification)]

# SETUP #####

attrs <- c("irrigation_target", "paddy_rice", "temporal")
ds_names <- dt_clean$dataset
n_perm <- 10000

# Base pairwise disagreement (pooled)-----

pairs <- CJ(ds1 = ds_names, ds2 = ds_names)[ds1 < ds2]
dt_sub <- dt_pres[n_pos > 0]
pairs[, disagreement:= mean(dt_sub[[ds1]] != dt_sub[[ds2]]), by = .(ds1, ds2)]

# Resolution-stratified pairwise disagreement-----

res_pairs <- dt_pres[n_pos > 0, {
  p <- CJ(ds1 = ds_names, ds2 = ds_names)[ds1 < ds2]
  sub <- .SD
  p[, disagreement:= mean(sub[[ds1]] != sub[[ds2]]), by = .(ds1, ds2)]
  p
}, by = resolution]

# RUN ALL INDIVIDUAL TESTS #####

results <- lapply(attrs, run_test,
                  pairs_dt = pairs, res_dt = res_pairs,
                  dt_meta = dt_clean, n_perm = n_perm)

## Warning in wilcox.test.default(d, s, alternative = "greater"): cannot compute
## exact p-value with ties
## Warning in wilcox.test.default(d, s, alternative = "greater"): cannot compute
## exact p-value with ties
## Warning in wilcox.test.default(d, s, alternative = "greater"): cannot compute
## exact p-value with ties

```



```
names(results) <- attr$
```

```
# Summary table-----
```

```
summary_tbl <- rbindlist(lapply(results, function(r) {
  data.table(attribute = r$attribute,
    pooled_W = r$pooled_W,
    pooled_p = r$pooled_p,
    obs_delta = r$obs_delta,
    perm_p = r$perm_p)
}))
print(summary_tbl)
```

```
##           attribute pooled_W pooled_p  obs_delta perm_p
##           <char>    <num>    <num>      <num>  <num>
## 1: irrigation_target      233 0.6538862 -0.004393000 0.4709
## 2:      paddy_rice      263 0.4154462 -0.008059414 0.5144
## 3:      temporal      253 0.5044833 -0.012013015 0.5300
```

```
# Resolution-stratified table -----
```

```
res_summary <- rbindlist(lapply(results, function(r) {
  r$res_wilcox[, attribute := r$attribute]
}))
print(res_summary)
```

```
## resolution n_same n_diff median_same median_diff      W  p_value
##           <char> <int> <int>      <num>      <num> <num>    <num>
## 1: 0.2deg      20     25  0.3419198  0.3457730 234.0 0.6454975
## 2: 0.4deg      20     25  0.2913762  0.2564966 248.0 0.5225355
## 3: 1deg        20     25  0.2454842  0.1923830 243.5 0.5635208
## 4: 0.2deg      22     23  0.2879978  0.3551894 275.0 0.3143284
## 5: 0.4deg      22     23  0.2764419  0.2643797 240.0 0.6192544
## 6: 1deg        22     23  0.2581066  0.2082699 214.0 0.8151239
## 7: 0.2deg      22     23  0.3504350  0.3287426 265.0 0.3979971
## 8: 0.4deg      22     23  0.3325741  0.2564966 238.0 0.6362766
## 9: 1deg        22     23  0.2806311  0.2047878 217.0 0.7963956
##           attribute
##           <char>
## 1: irrigation_target
## 2: irrigation_target
## 3: irrigation_target
## 4:      paddy_rice
## 5:      paddy_rice
## 6:      paddy_rice
## 7:      temporal
## 8:      temporal
## 9:      temporal
```

```

# OMNIBUS: any mismatch + dose-response #####

# Tag resolution pairs with all match columns -----

omni <- copy(res_pairs)
for (a in attrs) {
  lookup <- dt_clean[, .(dataset, val = get(a))]
  omni[lookup, on = .(ds1 = dataset), v1:= i.val]
  omni[lookup, on = .(ds2 = dataset), v2:= i.val]
  omni[, (paste0(a, "_match")):= fifelse(v1 == v2, "same", "different")]
  omni[, c("v1", "v2"):= NULL]
}

match_cols <- paste0(attrs, "_match")

# Number of mismatches (0-3)-----

omni[, n_mismatches:= Reduce(`+`, lapply(.SD, function(x)
  as.integer(x == "different"))),
  .SDcols = match_cols]

# Any mismatch-----

omni[, any_mismatch:= fifelse(n_mismatches > 0, "different", "same")]

# Omnibus Wilcoxon-----

s_omni <- omni[any_mismatch == "same", disagreement]
d_omni <- omni[any_mismatch == "different", disagreement]
wt_omni <- wilcox.test(d_omni, s_omni, alternative = "greater")

# Spearman dose-response-----

cor_omni <- cor.test(omni$n_mismatches, omni$disagreement,
  method = "spearman", alternative = "greater")

## Warning in cor.test.default(omni$n_mismatches, omni$disagreement, method =
## "spearman", : Cannot compute exact p-value with ties
cat("\n-- Omnibus Wilcoxon (any mismatch) --\n")

##
## -- Omnibus Wilcoxon (any mismatch) --
cat(sprintf("  W = %.1f, p = %.4f\n", wt_omni$statistic, wt_omni$p.value))

##  W = 1069.5, p = 0.4588

```

```

cat("\n-- Spearman dose-response --\n")

##
## -- Spearman dose-response --
cat(sprintf("  rho = %.4f, p = %.4f\n", cor_omni$estimate, cor_omni$p.value))

##  rho = -0.0741, p = 0.8033
# Omnibus permutation test (resolution-blocked)-----

obs_omni <- omni[, .(
  delta = mean(.SD[any_mismatch == "different", disagreement]) -
    mean(.SD[any_mismatch == "same", disagreement])
), by = resolution][, mean(delta, na.rm = TRUE)]

null_omni <- replicate(n_perm, {
  tmp <- copy(omni)
  for (a in attrs) {
    pl <- data.table(dataset = ds_names, val = sample(dt_clean[[a]]))
    tmp[pl, on = .(ds1 = dataset), v1:= i.val]
    tmp[pl, on = .(ds2 = dataset), v2:= i.val]
    tmp[, (paste0(a, "_match")):= fifelse(v1 == v2, "same", "different")]
    tmp[, c("v1", "v2"):= NULL]
  }
  tmp[, n_m:= Reduce(`+`, lapply(.SD, function(x) as.integer(x == "different"))),
    .SDcols = match_cols]
  tmp[, am:= fifelse(n_m > 0, "different", "same")]
  tmp[, .(delta = mean(.SD[am == "different", disagreement]) -
    mean(.SD[am == "same", disagreement])
  ), by = resolution][, mean(delta, na.rm = TRUE)]
})

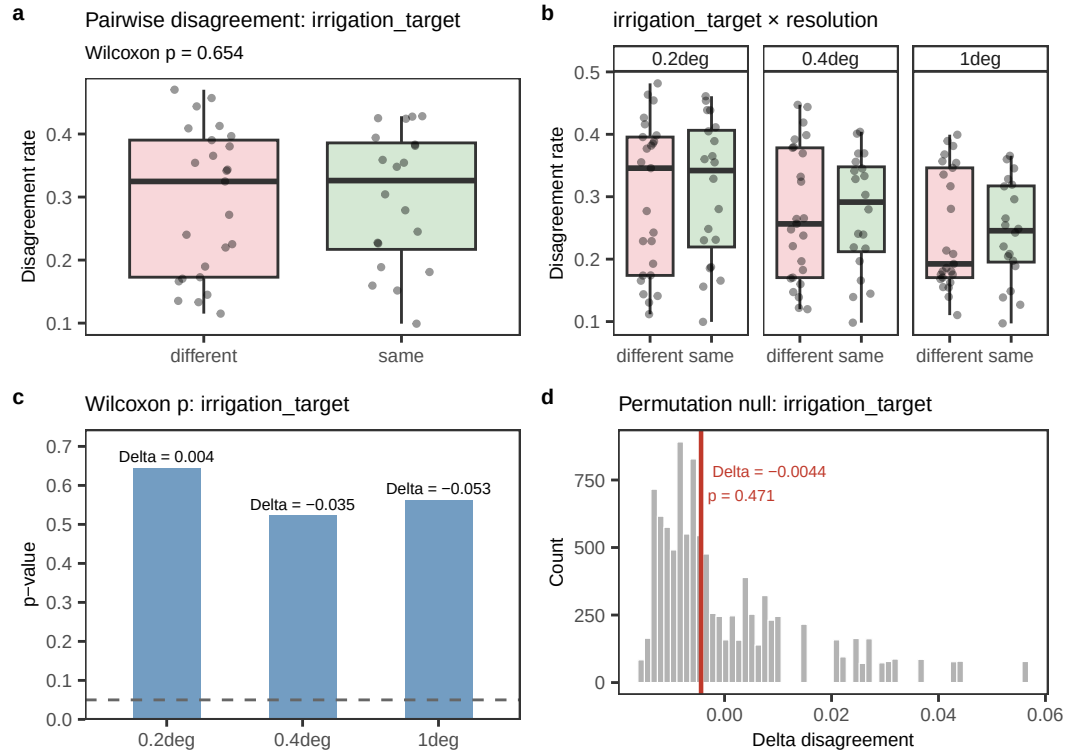
perm_p_omni <- mean(null_omni >= obs_omni)
cat(sprintf("\n-- Omnibus permutation --\n  obs Delta = %.4f, p = %.4f\n",
  obs_omni, perm_p_omni))

##
## -- Omnibus permutation --
##  obs Delta = 0.0024, p = 0.4478
# Plot results test -----

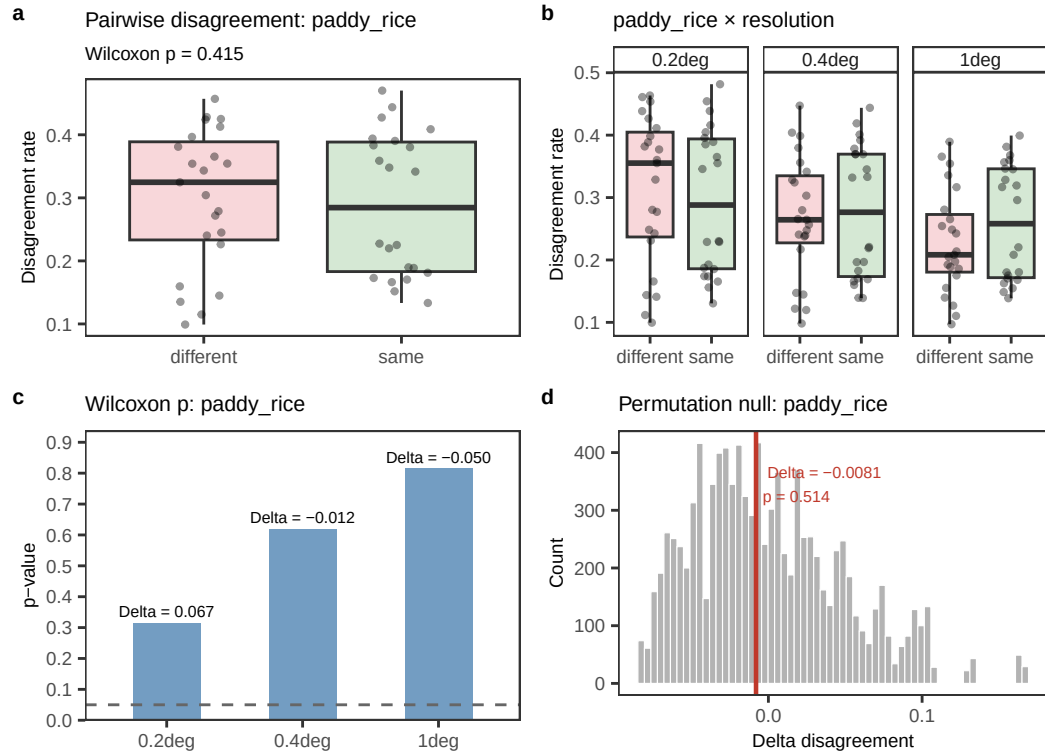
lapply(results, plot_attr)

## $irrigation_target

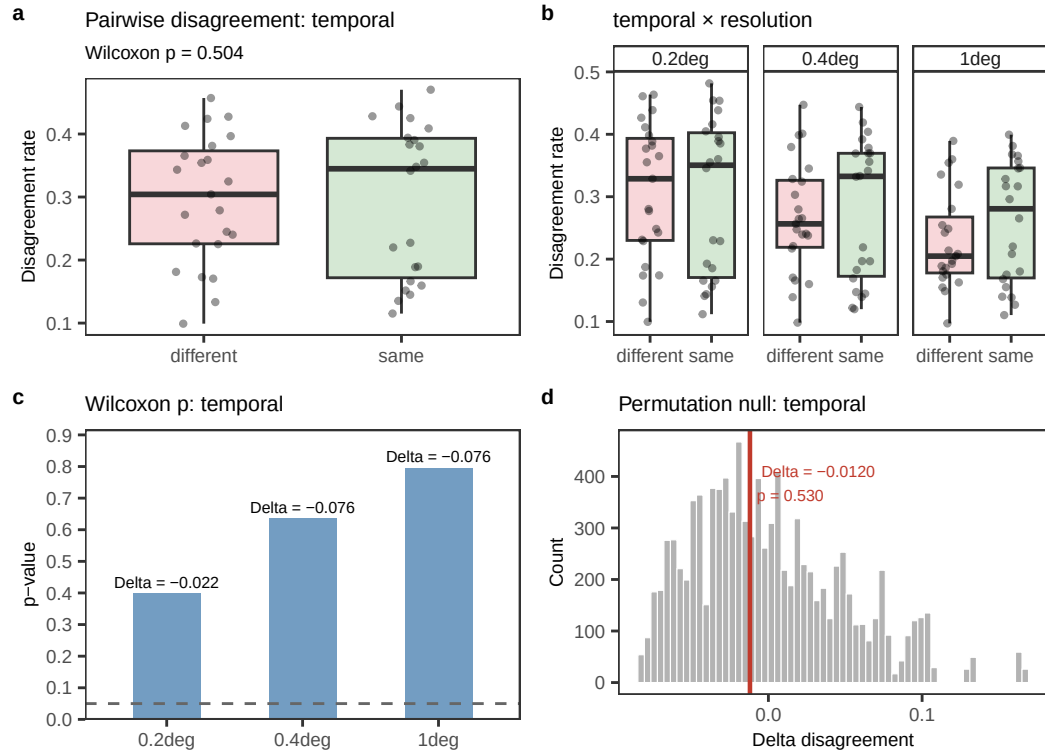
```



```
##
## $paddy_rice
```



```
##
## $temporal
```



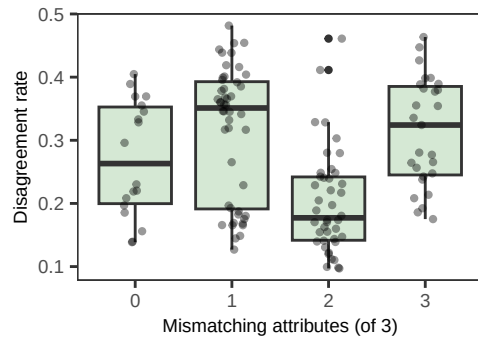
Omnibus plots -----

```
p_dose <- ggplot(omni, aes(x = factor(n_mismatches), y = disagreement)) +
  geom_boxplot(outlier.size = 0.8, fill = "#D5E8D4") +
  geom_jitter(width = 0.15, alpha = 0.4, size = 0.8) +
  labs(title = "Disagreement vs. number of \ndefinitional mismatches",
       subtitle = sprintf("Spearman rho = %.3f, p = %.3f", cor_omni$estimate, cor_omni$p.value),
       x = "Mismatching attributes (of 3)", y = "Disagreement rate") +
  theme_AP() + theme(plot.subtitle = element_text(size = 7))

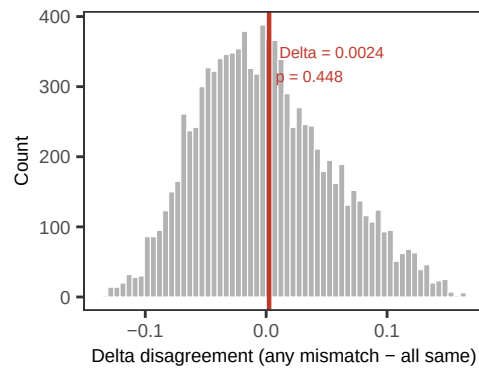
p_omni_null <- ggplot(data.table(delta = null_omni), aes(x = delta)) +
  geom_histogram(bins = 60, fill = "grey70", colour = "white", linewidth = 0.2) +
  geom_vline(xintercept = obs_omni, colour = "#C0392B", linewidth = 0.8) +
  annotate("text", x = obs_omni, y = Inf,
         label = sprintf("Delta = %.4f\np = %.3f", obs_omni, perm_p_omni),
         vjust = 2, hjust = -0.1, colour = "#C0392B", size = 2.2) +
  labs(title = "Omnibus permutation null \n(all attributes, resolution-blocked)",
       x = "Delta disagreement (any mismatch - all same)", y = "Count") +
  theme_AP()

plot_grid(p_dose, p_omni_null, ncol = 2, labels = "auto", label_size = 8)
```

a Disagreement vs. number of definitional mismatches
Spearman rho = -0.074, p = 0.803



b Omnibus permutation null
(all attributes, resolution-blocked)



9 Assessment of non-identifiability

```
# INSTABILITY UNDER SMALL PERTURBATIONS #####

# From long to wide dataset -----

dt_wide <- dcast(dt, lon + lat + country + code + continent +
               resolution ~ dataset, value.var = "mha")

# Run analysis -----
# 1% and 2% of the grid cell

inst_res <- run_instability_analysis(dt = dt_wide, dataset_names = dataset_names,
                                   tau_vals = c(1, 2), include_full = TRUE,
                                   include_loo = TRUE)

instability_dt <- inst_res$instability
classification_dt <- inst_res$classifications

instability_sum_all <- summarize_instability(instability_dt)
instability_sum_rel <- summarize_instability_relevant(instability_dt)

print(instability_sum_all)
```

```
## resolution n_cells frac_always_absent frac_always_present frac_unstable
##          <char>   <int>             <num>             <num>             <num>
## 1:      0.2deg  129628             0.3548770           0.3902243           0.2548986
## 2:      0.4deg   39758             0.3156597           0.4543237           0.2300166
## 3:       1deg   8068              0.2791274           0.5249132           0.1959593
## mean_flip_rate median_flip_rate q05_flip_rate q25_flip_rate q75_flip_rate
##          <num>             <num>             <num>             <num>             <num>
## 1:      0.05682169           0              0              0              0.04545455
## 2:      0.05049070           0              0              0              0.00000000
## 3:      0.04826024           0              0              0              0.00000000
## q95_flip_rate
##          <num>
## 1:      0.4545455
## 2:      0.4545455
## 3:      0.4545455
```

```
print(instability_sum_rel)
```

```
## resolution n_cells frac_unstable mean_flip_rate median_flip_rate
##          <char>   <int>             <num>             <num>             <num>
## 1:      0.2deg   83626             0.3951164           0.08807885           0
## 2:      0.4deg   27208             0.3361144           0.07378010           0
## 3:       1deg   5816              0.2718363           0.06694698           0
## q05_flip_rate q25_flip_rate q75_flip_rate q95_flip_rate
```

```
##           <num>           <num>           <num>           <num>
## 1:         0             0       0.09090909       0.4545455
## 2:         0             0       0.04545455       0.4545455
## 3:         0             0       0.04545455       0.5000000

# CONVERGENCE TEST #####

# Run on exact combinations -----

tau_vals <- c(1, 2)

res_exact_all <- rbindlist(lapply(tau_vals, function(tt) {
  run_by_resolution(dt = dt_wide, dataset_names = dataset_names,
    tau_percent = tt, mode = "exact")
}), fill = TRUE)

res_exact_all_sum <- summarize_agreement_vs_information(res_exact_all)

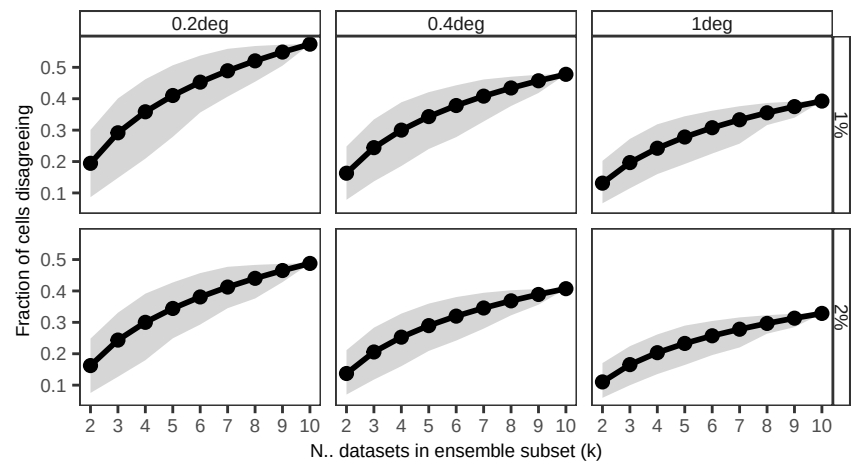
# Prettier labels-----

res_exact_all_sum[, tau_percent:= paste(tau_percent, "%", sep = "")]

# Plot results -----

p_disagree_exact <- ggplot(res_exact_all_sum, aes(k, frac_disagree_mean)) +
  geom_ribbon(aes(ymin = frac_disagree_q05, ymax = frac_disagree_q95), alpha = 0.2) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  facet_grid(tau_percent~ resolution) +
  scale_x_continuous(breaks = 2:length(dataset_names)) +
  labs(x = "N° datasets in ensemble subset (k)",
    y = "Fraction of cells disagreeing") +
  theme_AP()

p_disagree_exact
```

10 Irrigation masks

```
# PRODUCE AND EXPORT IRRIGATION MASKS #####
```

```
irrigation_mask_dt <- dt_pres[, .(lon, lat, country, code,  
                                continent, resolution, n_pos)]
```

```
setnames(irrigation_mask_dt, "n_pos", "k")
```

```
dir.create(here("datasets", "output"), recursive = TRUE, showWarnings = FALSE)
```

```
res_labels <- c("0.2deg" = "02", "0.4deg" = "04", "1deg" = "1")
```

```
irrigation_mask_dt[, {  
  fwrite(.SD, here("datasets", "output", paste0("irrigation_mask_", res_labels[resolution], ".  
}, by = resolution]
```

```
## Empty data.table (0 rows and 1 cols): resolution
```

11 Session information

```
# SESSION INFORMATION #####
```

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.3.1
##
## Matrix products: default
## BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods    base
##
## other attached packages:
## [1] benchmarkme_1.0.8   wesanderson_0.3.7   countrycode_1.6.1
## [4] sf_1.1-0            rnatralearnth_1.1.0 terra_1.8-93
## [7] here_1.0.2          sensobol_1.1.6      lubridate_1.9.4
## [10] forcats_1.0.1       stringr_1.6.0       dplyr_1.1.4
## [13] purrr_1.2.1         readr_2.2.0         tidyr_1.3.2
## [16] tibble_3.3.1        ggplot2_4.0.1.9000  tidyverse_2.0.0
## [19] cowplot_1.2.0.9000 scales_1.4.0         data.table_1.18.2.1
##
## loaded via a namespace (and not attached):
## [1] benchmarkmeData_1.0.4 gtable_0.3.6          xfun_0.55
## [4] lattice_0.22-7        tzdb_0.5.0            vctrs_0.7.1
## [7] tools_4.5.2           Rdpack_2.6.4          generics_0.1.4
## [10] parallel_4.5.2        proxy_0.4-29          pkgconfig_2.0.3
## [13] Matrix_1.7-4          KernSmooth_2.23-26    RColorBrewer_1.1-3
## [16] S7_0.2.1              lifecycle_1.0.5       compiler_4.5.2
## [19] farver_2.1.2          tinytex_0.58          codetools_0.2-20
## [22] htmltools_0.5.9       class_7.3-23          yaml_2.3.12
## [25] pillar_1.11.1         classInt_0.4-11       iterators_1.0.14
## [28] foreach_1.5.2         tidyselect_1.2.1      digest_0.6.39
## [31] stringi_1.8.7         labeling_0.4.3        rprojroot_2.1.1
## [34] fastmap_1.2.0         grid_4.5.2           cli_3.6.5
## [37] magrittr_2.0.4        e1071_1.7-17          withr_3.0.2
## [40] timechange_0.3.0      httr_1.4.8            rmarkdown_2.30
## [43] otel_0.2.0            hms_1.1.4            evaluate_1.0.5
```

```
## [46] knitr_1.51          rbibutils_2.4          doParallel_1.0.17
## [49] rlang_1.1.7           Rcpp_1.1.1            glue_1.8.0
## [52] DBI_1.3.0             rstudioapi_0.17.1     R6_2.6.1
## [55] units_1.0-0
```

```
## Return the machine CPU -----
```

```
cat("Machine:      "); print(get_cpu())$model_name)
```

```
## Machine:
```

```
## [1] "Apple M1 Max"
```

```
## Return number of true cores -----
```

```
cat("Num cores:    "); print(parallel::detectCores(logical = FALSE))
```

```
## Num cores:
```

```
## [1] 10
```

```
## Return number of threads -----
```

```
cat("Num threads: "); print(parallel::detectCores(logical = FALSE))
```

```
## Num threads:
```

```
## [1] 10
```